

STUDY GUIDE

Operating Systems

Chapters 6–9: deadlock, memory, virtual memory and scheduling

This printable guide collects the notes, key terms and review questions (with answers and explanations) from the interactive guide. Use it to review after working through the simulations.

1. Concurrency: Deadlock and Starvation
2. Memory Management
3. Virtual Memory
4. Uniprocessor Scheduling

Contents

1. Chapter 6: Concurrency: Deadlock and Starvation

- 6.1 Principles of Deadlock
- 6.2 Deadlock Prevention
- 6.3 Deadlock Avoidance
- 6.4 Deadlock Detection
- 6.5 An Integrated Deadlock Strategy
- 6.6 Dining Philosophers Problem
- 6.7 UNIX Concurrency Mechanisms
- 6.8 Linux Kernel Concurrency Mechanisms
- 6.9 Solaris Thread Synchronization Primitives
- 6.10 Windows Concurrency Mechanisms
- 6.11 Android Interprocess Communication

2. Chapter 7: Memory Management

- 7.1 Memory Management Requirements
- 7.2 Memory Partitioning
- 7.3 Paging
- 7.4 Segmentation
- 7A Loading and Linking

3. Chapter 8: Virtual Memory

- 8.1 Hardware and Control Structures
- 8.2 Operating System Software
- 8.3 UNIX and Solaris Memory Management
- 8.4 Linux Memory Management
- 8.5 Windows Memory Management
- 8.6 Android Memory Management

4. Chapter 9: Uniprocessor Scheduling

- 9.1 Types of Processor Scheduling
- 9.2 Scheduling Algorithms
- 9.3 Traditional UNIX Scheduling

Chapter 6 · Concurrency: Deadlock and Starvation

Locks keep shared data safe. Used carelessly, they can freeze everything. Here is how systems prevent, avoid, detect and recover from that.

Chapter 5 made processes take turns on shared data. This chapter looks at the failure that turn-taking can cause: a deadlock, where a group of processes each hold something another one needs and all of them wait forever. It explains the four conditions that make a deadlock possible, then the three ways an operating system can respond: rule one condition out (**prevention**), check every request against a safe plan (**avoidance**), or let deadlocks happen and then find and break them (**detection and recovery**). The classic dining philosophers puzzle ties the ideas together, and the chapter closes with the synchronization tools real systems offer: UNIX, Linux, Solaris, Windows and Android.

Learning objectives

- List the four conditions for deadlock and use a resource allocation graph to spot one.
- Explain how deadlock prevention removes one of the conditions, and what each choice costs.
- Run the banker's algorithm to decide whether a state is safe and whether a request may be granted.
- Detect a deadlock with a detection algorithm and choose a recovery strategy.
- Solve the dining philosophers problem without deadlock or starvation.
- Compare the concurrency mechanisms of UNIX, Linux, Solaris, Windows and Android.

How Chapter 6 fits together

Chapter 5's locks bring a new danger. A set of processes is in **deadlock** when each is blocked waiting for something only another blocked member can provide. In the overview, P locks the printer then the scanner and Q the opposite order; if each gets its first device, both wait forever (3 random schedules in 8, on average). With one agreed order, none freezes. **Starvation** differs: the system keeps working, but one ready process is passed over again and again.

The principles (6.1)

Resources are **reusable** (devices, memory, files) or **consumable** (messages, signals). Deadlock needs four conditions: **mutual exclusion**, **hold and wait** and **no preemption** make it possible, and a **circular wait** (a closed chain of processes, each waiting for a resource the next one holds) makes it happen. A resource allocation graph draws request edges (process → resource) and assignment edges (resource → process); a cycle among single-unit resources means deadlock.

Three ways to respond (6.2 to 6.4)

- **Prevention (6.2)** designs one condition out in advance: request everything at once (no hold and wait), make a waiting process give up what it holds (no preemption), or number the resources and request them in increasing order (no circular wait). No future knowledge is needed, but resources are often idle or held early.
- **Avoidance (6.3)** rules out no condition in advance. Knowing each process's maximum claim, the **banker's algorithm** grants a request only if the result is a **safe state** (one with an order in which every

process can finish), so some grantable requests wait.

- **Detection and recovery (6.4)** grants freely, runs a detection algorithm now and then, and breaks any deadlock by aborting processes, rolling them back to a checkpoint or preempting resources. The cost is lost work.

Mixing strategies and a classic test (6.5, 6.6)

An **integrated strategy** (6.5) sorts resources into ordered classes and uses the best method inside each class. The **dining philosophers** (6.6) share five forks; the obvious solution deadlocks, while seating at most four, one philosopher reaching right first, or a monitor each break circular wait. A deadlock-free solution can still allow starvation.

Real systems (6.7 to 6.11)

UNIX (6.7): pipes, message queues, shared memory, semaphores, signals. Linux (6.8): real-time signals, atomic operations, spinlocks, semaphores, barriers, RCU. Solaris (6.9): mutexes, semaphores, readers/writer locks, condition variables. Windows (6.10): dispatcher objects, critical sections, slim reader-writer locks, interlocked operations. Android (6.11): Binder.

Why it matters next

Main memory is the resource every process needs; Chapter 7 shows how the OS divides it.

6.1 Principles of Deadlock

You should be able to

- Define deadlock precisely and recognise it in traffic, devices, memory and message passing.
- Read and draw a joint progress diagram, and find its fatal region and deadlock point.
- Tell reusable resources from consumable ones and explain how each kind can deadlock.
- Build a resource allocation graph and decide, by reduction, whether a cycle really is a deadlock.
- State the four conditions for deadlock and compare prevention, avoidance and detection.

What a deadlock is

A set of processes is **deadlocked** when every process in the set is blocked, each waiting for an event (usually the release of a resource) that only another blocked member of the same set can cause. Because all of them are asleep, the event never happens: the blocking is **permanent** unless the operating system intervenes from outside, for example by aborting a process or taking a resource back. No efficient solution works in every case, so each strategy an OS can choose has a cost.

- Deadlock is a property of a **group** of processes that wait on each other, usually two or more.
- It is not the same as a long wait (the wait ends by itself), starvation (others keep progressing while one is passed over) or livelock (processes keep running but make no progress).

Picture: gridlock at a crossing

Four cars reach a four-way crossing together. Each car going straight needs two of the four quadrants, one after the other. If every car enters its first quadrant, each holds one quadrant and waits for the next, which

the next car holds. The waits form a closed loop and nobody can move. Only an outside action, such as an officer making one car reverse (taking its quadrant back), breaks the loop.

Joint progress diagram

For two processes P and Q on one processor, the x axis is P's progress and the y axis is Q's. Only one process runs at a time, so a run is a **staircase** path: right when P runs, up when Q runs (with two processors the path could also go diagonally). Example: P does get A, get B, free A, free B; Q does get B, get A, free B, free A.

- **"A needed by both"** and **"B needed by both"** are boxes where both processes would hold the same resource. The path can never enter them.
- The **fatal region** is the corner where P already holds A and Q already holds B. From any point there, every path ends at the **deadlock point**, where P waits for B and Q waits for A. Deadlock is decided on entering the region, before anyone blocks.
- Strict turn-taking (P, Q, P, Q, ...) walks straight into the fatal region; other schedules pass around it. Same code, different timing.
- If P instead frees A **before** it asks for B, P never holds both. The two boxes no longer overlap, there is no fatal region, and both processes finish in every interleaving.

Reusable resources

A **reusable resource** is used by one process at a time and is not used up: processors, I/O channels, main and secondary memory, devices, files, databases and semaphores. A deadlock over reusable resources has one shape: each process holds some and waits for more that another holds.

- **Two devices, opposite orders.** P locks the scanner then the printer; Q locks the printer then the scanner. If each takes its first lock before the other takes its second, both block forever. Locking in the same order in both programs removes the danger.
- **A memory pool.** Pool = 200 KB. A asks for 80 KB, later 70 KB more; B asks for 70 KB, later 80 KB more. If both first requests are granted, $200 - 150 = 50$ KB remain, less than either second request, so both block. Worked check: the smallest safe pool is $150 + 70 = 220$ KB, because then A's second request can be met, A finishes and frees 150 KB, and B can finish.

Consumable resources

A **consumable resource** is created by a producer and destroyed when a consumer takes it; there is no fixed supply. Examples: interrupts, signals, messages and data in I/O buffers.

- If P1 does a blocking Receive from P2 before sending, and P2 does a blocking Receive from P1 before sending, both block in every interleaving: the messages each needs are never created. Sending first fixes it.
- Such deadlocks come from design errors and can be rare: a client sends a request and waits for the answer, while the server, only when a request arrives damaged, waits for a resend that the client never sends. Tests that rarely produce damaged requests almost never reveal it.

Resource allocation graphs

Processes are circles; each resource type is a square with one dot per unit. A **request edge** points from a process to a resource square (the process is waiting). An **assignment edge** points from a unit dot to the process holding it.

- If every resource has a single unit, a **cycle means deadlock**.
- With multi-unit resources, a cycle is **necessary but not sufficient**: if a unit on the cycle is free or held by a process that can still finish, the cycle will break.
- **Graph reduction** decides it: (1) find a process whose outstanding requests can all be met by the free units; (2) assume it runs to completion; (3) return every unit it holds; (4) repeat while someone can be picked; (5) any process left over is deadlocked.
- Example: R1 (1 unit), R2 (2 units). P1 holds R1 and wants R2; P2 holds one R2 and wants R1; P3 holds the other R2 and wants nothing. Cycle $P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$, but P3 finishes, then P1, then P2: no deadlock. If P3 also waited for R1, nobody could be picked and all three would be deadlocked.

The four conditions

1. **Mutual exclusion**: only one process at a time may use a resource unit.
2. **Hold and wait**: a process keeps the resources it holds while waiting for more.
3. **No preemption**: a resource cannot be taken away from its holder; it is released only voluntarily.
4. **Circular wait**: a closed chain of processes exists, each holding a resource the next one is waiting for.

Conditions 1–3 are **policy** conditions: each is **necessary** (without it there can be no deadlock) but even all three together are not **sufficient**. Condition 4 is the **event** that may happen as processes run; given 1–3, a circular wait that cannot be broken is a deadlock. All four together are necessary and sufficient. Ruling out any one makes deadlock impossible, for example: shared read-only access (no mutual exclusion), asking for everything at once (no hold and wait), giving everything back when refused or letting the OS take memory frames back (no preemption condition), numbered resources requested in increasing order (no circular wait).

Three ways to respond

Approach	Policy	Scheme	Main advantage	Main disadvantage
Prevention	Conservative; undercommits resources. Rule out a condition by design.	Request all at once	Suits a single burst of work; no preemption needed	Inefficient; delays starting; needs known in advance
		Preemption	Convenient when state is easy to save and restore	Preempts more often than necessary
		Resource ordering	Checkable at compile time; no run-time cost	Resources cannot be requested in whatever order the work needs
Avoidance	Midway. Grant a request only if every process can still finish afterwards.	Safe-state check per request	No preemption necessary	Needs future maximum claims; processes may wait long
Detection and recovery	Very liberal. Grant whatever can be granted; check periodically.	Periodic detection check	Never delays process start; handled on-line	Recovery loses work (aborts, preemption)

Prevention changes the rules so a condition can never hold; avoidance rules out no condition in advance and judges each request against declared future needs; detection lets deadlocks happen and then breaks them.

Key terms

Deadlock

A permanent standstill of a set of processes: each one is blocked waiting for an event, usually the release of a resource, that only another blocked member of the same set can cause.

Joint progress diagram

A graph of two processes running on one processor: the x axis is how far one process has got, the y axis how far the other has got, and the run is drawn as a staircase path.

Fatal region

The part of a joint progress diagram where both processes already hold a resource the other will need, so every possible path from there ends in deadlock.

Reusable resource

A resource that one process at a time can use and that is not used up by using it, such as a processor, memory, a device, a file or a semaphore. It is released for others afterwards.

Consumable resource

A resource that is created by one process and destroyed when another process takes it, such as a message, a signal, an interrupt or data in an I/O buffer. There is no fixed supply.

Resource allocation graph

A directed graph with a circle for each process and a square for each resource type (one dot per unit). Its edges show which process holds which unit and which process is waiting for what.

Request edge

An arrow from a process to a resource square in a resource allocation graph: the process has asked for a unit of that resource and is waiting for it.

Assignment edge

An arrow from a unit dot inside a resource square to a process: that unit has been given to (is held by) the process.

Graph reduction

A way to test a resource allocation graph for deadlock: repeatedly pick a process whose outstanding requests could all be granted now, pretend it finishes and frees what it holds, and repeat. Processes that can never be picked are deadlocked.

Hold and wait

The deadlock condition in which a process keeps the resources it already holds while it waits for more.

No preemption

The deadlock condition in which a resource cannot be taken away from the process holding it; it is only released voluntarily.

Circular wait

A closed chain of processes in which each process holds at least one resource that the next process in the chain is waiting for.

Necessary condition

Something that must be true for an outcome to happen. It may be true without the outcome happening.

Sufficient condition

Something that, when true, guarantees the outcome happens.

Deadlock prevention

A strategy that designs the system so that one of the four deadlock conditions can never hold, making deadlock impossible.

Deadlock avoidance

A strategy that checks each request as it is made and refuses or delays any grant that could lead to deadlock, using knowledge of each process's maximum future needs.

Deadlock detection

A strategy that grants requests freely, checks from time to time whether a deadlock has formed, and then recovers by breaking it.

Review questions with answers

1. Which of these situations is a **deadlock?**

- A. P1 waits for the disk, which is busy with P2's request and will finish in a few milliseconds.
- B. P1 holds lock A and waits for lock B, while P2 holds lock B and waits for lock A.
- C. A low-priority process keeps being passed over because higher-priority work keeps arriving.
- D. Two processes keep reacting to each other and changing state, but neither makes progress.

Answer: B. P1 holds lock A and waits for lock B, while P2 holds lock B and waits for lock A.

Deadlock needs a set of processes that are all blocked, each waiting for an event only another blocked member can cause. P1 and P2 each wait for a lock the other holds, so neither will ever release anything.

2. In a joint progress diagram, once the execution path enters the fatal region, deadlock is certain even though neither process is blocked yet.

Answer: True

In the fatal region each process already holds a resource the other will need. Every continuation of the path ends at the deadlock point, so the outcome is decided before anyone actually blocks.

3. If a cycle in a resource allocation graph passes through a resource that has several units, the processes on that cycle must be deadlocked.

Answer: False

With multi-unit resources a cycle is necessary but not sufficient. If a unit on the cycle is held by a process that can still finish, that process will release it and the cycle breaks.

4. Sort each resource: reusable or consumable?

Reusable	Consumable
Main memory An I/O channel A semaphore used as a lock A file	A message An interrupt Data waiting in an I/O buffer A signal

Reusable resources are used by one process at a time and then released, unchanged. Consumable resources are created by a producer and destroyed when a consumer takes them.

5. Match each deadlock condition to its meaning.

Mutual exclusion	Only one process at a time may use a resource unit
Hold and wait	A process keeps what it holds while it waits for more
No preemption	A resource cannot be taken away from the process holding it
Circular wait	A closed chain of processes, each waiting for a resource the next one holds

The first three describe how resources are managed (policies); the fourth is a pattern that may arise as processes run.

6. Which of the four conditions are **policy** conditions: necessary for deadlock, yet not enough on their own to cause one?

- A. Mutual exclusion
- B. Hold and wait
- C. No preemption
- D. Circular wait

Answer: A, B, C

Mutual exclusion, hold and wait and no preemption can all hold for a long time without any deadlock. Circular wait is the event that actually happens; given the other three, an unbreakable circular wait is a deadlock.

7. A memory pool holds 200 KB. Process A asks for 80 KB and later for 70 KB more; process B asks for 70 KB and later for 80 KB more. Neither releases anything until it finishes. If both first requests are granted, how many KB are left free?

Answer: 50 KB

$200 - 80 - 70 = 50$ KB. That is less than either second request (70 or 80 KB), so both processes block for good: a deadlock over a reusable resource.

8. Process A asks for 80 KB and later 70 KB more; process B asks for 70 KB and later 80 KB more; neither releases memory until it finishes. What is the smallest pool size, in KB, for which no order of requests can deadlock?

Answer: 220 KB

After both first grants, 150 KB are in use. If at least 70 KB remain, A's second request can be met; A finishes and frees 150 KB, so B can finish too. $150 + 70 = 220$ KB. With any less, both second requests would fail.

9. R1 has 1 unit and R2 has 2 units. P1 holds R1 and requests a unit of R2. P2 holds one unit of R2 and requests R1. P3 holds the other unit of R2 and requests nothing. How many processes are deadlocked?

Answer: 0

There is a cycle ($P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$), but P3 is not waiting. P3 finishes and frees its unit of R2; P1 gets it, finishes and frees R1 and R2; then P2 gets R1 and finishes. Nobody is left over, so 0 are deadlocked.

10. Two processes exchange messages with a **blocking** Receive. P1 runs: Receive from P2, then Send to P2. P2 runs: Receive from P1, then Send to P1. What happens?

- A. Both block forever, whatever order the scheduler picks.
- B. They deadlock only if P2 happens to run first.
- C. Nothing goes wrong: messages are consumable, so they never run out.
- D. They deadlock only on a machine with more than one processor.

Answer: A. Both block forever, whatever order the scheduler picks.

Each process sends only after it has received, so neither message is ever created. This design error is a deadlock over consumable resources.

11. An operating system grants every request it can. Every few minutes it runs an algorithm that looks for a set of processes that can never finish, and if it finds one it aborts a member. Which approach is this?

- A. Deadlock prevention
- B. Deadlock avoidance
- C. Detection and recovery
- D. Ignoring the problem

Answer: C. Detection and recovery

Granting freely, checking periodically and then recovering (by aborting or preempting) is detection and recovery. It never delays a process from starting, but recovery loses work.

12. Put the steps of reducing a resource allocation graph in order.

Correct order:

1. Find a process whose outstanding requests can all be met by the free units
2. Assume it runs to completion
3. Return every unit it holds to the free pool
4. Repeat while some other process can be picked
5. Report any processes left over as deadlocked

Reduction plays out a best-case future: whoever can finish does, and frees what it held. Processes that can never be picked are exactly the deadlocked ones.

6.2 Deadlock Prevention

You should be able to

- Explain how deadlock prevention works by ruling out one of the four deadlock conditions, and tell the indirect methods from the direct method.
- Explain why mutual exclusion usually cannot be given up, and recognise resources that can safely be shared.
- Describe the all-at-once rule for hold and wait and the two ways to remove no preemption, with the price of each.
- Apply a numbered resource ordering to a set of programs and explain why it makes a circular wait impossible.
- Compare the prevention rules by what they cost: idle resources, longer waits, redone work and extra effort for programmers.

6.2 Deadlock Prevention

A deadlock needs four conditions at once: **mutual exclusion** (one process at a time per resource), **hold and wait** (a process keeps what it holds while waiting for more), **no preemption** (nobody can take a resource from its holder) and **circular wait** (a closed chain in which each process waits for a resource the next one holds). **Deadlock prevention** designs the system, before anything runs, so that at least one of them can never hold. One is enough: then no schedule can produce a deadlock. **Indirect methods** rule out one of the first three conditions, removing what a circular wait needs in order to freeze the system; the **direct method** rules out the fourth, the circular wait itself.

Compare: avoidance (6.3) rules out no condition in advance but checks each request at run time; detection (6.4) lets deadlocks happen, then finds and breaks them. Analogy: a fire needs fuel, heat, oxygen and its chain reaction; removing any one (say, the oxygen) puts it out.

1. Mutual exclusion: usually cannot be ruled out

A **shareable resource** can be used by many processes at once without harm: a read-only file, a record everyone only reads, read-only program code. Nobody waits for it, so it is never part of a deadlock. A **non-shareable resource** needs exclusive access: a printer, a DVD burner, a record or counter being updated. The OS must provide that exclusive access, so prevention cannot drop this condition in general. Sharing anyway trades deadlock for corrupted results: two print jobs mix their lines on one page, and two updates

to one record lose one (balance \$800; P1 adds \$150 while P2 takes \$200; both read \$800; P1 writes \$950, then P2 writes \$600; the right answer was \$750). Keep exclusive access only where it is really needed.

2. Hold and wait: ask for everything at once

All-at-once request: a process asks for every resource it will need in one request and is blocked, holding nothing, until the whole set can be granted together. So hold and wait cannot happen, and a waiting process, holding nothing, can never be part of a circular wait. **Variant:** a process may request resources only while it holds none, giving back everything before it asks for more.

The price: (1) a process may wait long for its whole set when it could have started with part of it; (2) resources stay allocated but unused for long periods, lowering **resource utilization** (the share of time a resource does useful work); (3) a process may not know in advance all it will need (it can depend on the input); (4) in a modular program, every module's needs must be known and added up before it starts.

Worked example: P1 arrives in tick 0 (disk 3 ticks, compute 2, then disk + printer 2); P2 in tick 1 (printer 5); P3 in tick 4 (disk 3). "Done" is a process's last tick of work.

Rule	P1 done	P2 done	P3 done	Ticks waiting	Held but idle	Hold and wait?
No rule	7	5	10	5	3	yes (P1, tick 5)
All at once	6	11	9	9	7	never
Release first	8	5	6	2	0	never

All at once leaves the printer held and idle while P1 is still reading, so P2 finishes much later. Release first makes P1 give the disk back; it loses it to P3 and finishes later, and a process that keeps losing that race could starve. Idle share = (time held – time used) / time held: a plotter held 12 minutes but used 3 is idle $(12 - 3) / 12 = 75\%$ of the time.

3. No preemption: let resources be taken back

- **(a) Give back when refused.** A process that holds resources and is refused a further request must release everything it holds, and later request it all again plus the new one. It waits holding nothing.
- **(b) Take it from the holder.** When a process requests a resource another holds, the OS may force the holder to release it. This works only if no two processes have the same **priority**, so it is always clear who gives way; with equal priorities two processes could keep snatching it back and forth.

Practical only for a **preemptible resource**, whose state can be saved and restored later: the processor (save its registers) or memory (copy a page to disk and back). A printer halfway through a page cannot be un-printed: the half page is wasted and the work redone.

4. Circular wait: resource ordering (the direct method)

Number every resource type $R_1, R_2, \dots$. A process holding type R_i may afterwards request only types R_j with $j > i$ (several units of one type are requested together). Example: tape 1, disk 2, scanner 3, printer 4, plotter 5; holding the scanner, a process may still request only the printer and the plotter: 2 types.

The numbering game (scanner 1, disk 2, printer 3, network 4; P1 scanner → disk → network; P2 disk → printer; P3 printer → scanner): P3 breaks the rule, and the three can close a circle (P1 holds the scanner, waits for the disk; P2 holds the disk, waits for the printer; P3 holds the printer, waits for the scanner). The

natural orders already form a loop, so no numbering makes all three legal; some program must change its order, e.g. P3 locks the scanner first, before it needs it.

Why no cycle can form (proof by contradiction): (1) assume processes that all follow the rule form a circular wait; (2) each holds a resource and requests one held by the next, so the numbers rise at every link: $a < b < c < \dots$; (3) going around the chain leads back to the first process's resource, number a ; (4) so $a < \dots < a$, a number larger than itself. Impossible, whatever the timing.

The price: requests follow the number order, not the program's natural order, so resources are grabbed early and held longer; this can be inefficient, slow processes down and deny access unnecessarily.

5. Comparing the rules

Rule lab: P1 locks the disk, reads, locks the printer, prints, unlocks both; P2 locks the printer, prints a banner, locks the disk, prints, unlocks both (under the numbered rule, disk #1 and printer #2, P2 locks the disk first). Every possible interleaving, then alternating turns (blocked: a process blocked after a turn; idle: a work step with a held resource unused; given back: a refusal that forced a release):

Rule	Rules out	Orders that deadlock	Alternating turns	Blocked	Idle	Given back
No rule	nothing	18 of 24	deadlock at tick 6	3	0	0
All at once	hold and wait	0 of 8	both finish after 8 ticks	3	2	0
Give back when refused	no preemption	0 of 24	both finish after 10 ticks	3	0	1
Numbered order	circular wait	0 of 10	both finish after 10 ticks	4	1	0

Condition ruled out (kind)	Main price
Mutual exclusion (indirect)	Possible only for truly shareable resources
Hold and wait (indirect)	Long waits, idle resources, needs known in advance
No preemption (indirect)	Only for saveable state; work may be redone
Circular wait (direct)	Unnatural request order; resources held early

Overall: prevention is simple to reason about and guarantees no deadlock, but it is conservative: it lowers resource utilization and slows processes, or burdens programmers.

Key terms

Deadlock prevention

Designing the system in advance so that one of the four deadlock conditions can never hold. With one condition gone, deadlock cannot happen at all, whatever the timing.

Indirect method

A prevention method that rules out one of the first three conditions (mutual exclusion, hold and wait, or no preemption). It removes what a circular wait needs in order to freeze the system, instead of

forbidding the circle itself.

Direct method

The prevention method that rules out the fourth condition, the circular wait itself, usually by numbering the resource types.

Mutual exclusion

The rule that only one process at a time may use a resource; any other process that asks for it must wait. It is the first of the four deadlock conditions.

Hold and wait

The deadlock condition in which a process keeps the resources it already holds while it waits for more.

No preemption

The deadlock condition in which a resource cannot be taken away from the process holding it; it is released only when the holder chooses to give it up.

Circular wait

A closed chain of processes in which each one holds at least one resource that the next one in the chain is waiting for.

Shareable resource

A resource that many processes can use at the same moment without harm, such as a read-only file. Nobody ever waits for it, so it can never be part of a deadlock.

Non-shareable resource

A resource that only one process may use at a time, such as a printer or a record that is being updated. Letting two processes use it at once would corrupt the work.

All-at-once request

A rule that removes hold and wait: a process asks for every resource it will need in one request and is blocked, holding nothing, until the whole set can be granted together.

Preemption

Taking a resource away from the process that holds it before that process is finished with it, and giving it back later.

Preemptible resource

A resource whose state can be saved and restored later, such as the processor (save its registers) or a page of memory (copy it to disk), so it can be taken away without ruining the work.

Priority

A ranking given to each process. When two processes compete, the one with the higher priority is served first, and a lower-priority process gives way to it.

Resource ordering

A rule that removes circular wait: every resource type gets a number, and a process that holds a resource may afterwards request only resources with higher numbers.

Resource utilization

The share of time a resource spends doing useful work, rather than sitting idle or being held by a process that is not using it.

Review questions with answers

1. What does a deadlock prevention strategy do?

- A. It designs the system so that at least one of the four deadlock conditions can never hold
- B. It checks each request as it is made and refuses any grant that could lead to deadlock
- C. It lets deadlocks happen, then finds the processes involved and breaks the deadlock
- D. It removes all four deadlock conditions at once, so that no process ever has to wait

Answer: A. It designs the system so that at least one of the four deadlock conditions can never hold

A deadlock needs all four conditions at the same time, so a rule that makes any one of them impossible rules deadlock out completely, whatever the timing.

2. Which of these is the **direct** method of deadlock prevention?

- A. Requiring each process to request all of its resources at once
- B. Numbering the resource types and requiring requests in increasing order
- C. Letting the OS take a resource away from a lower-priority holder
- D. Letting many processes share a read-only file

Answer: B. Numbering the resource types and requiring requests in increasing order

Indirect methods rule out one of the first three conditions (mutual exclusion, hold and wait, no preemption). The direct method rules out the fourth, circular wait, usually with a numbered order of resource types.

3. Can several processes use each resource at the same moment without harm?

Shareable	One process at a time
A read-only file A database record that several processes only read Read-only program code used by many processes	A printer A database record that a process is updating A DVD burner writing a disc

Reading changes nothing, so readers can share. Output devices and data being written need one process at a time, which is why mutual exclusion cannot be ruled out in general.

4. Match each prevention rule with the deadlock condition it makes impossible.

Request every resource in a single request	Hold and wait
Release everything you hold when a request is refused	No preemption
Request resource types only in increasing number order	Circular wait
Let many readers share a read-only file at once	Mutual exclusion

All-at-once requests stop a process from holding while it waits; giving resources back on refusal makes them preemptible; numbering stops any chain from closing into a circle; sharing removes the need for exclusive use, but only where sharing is safe.

5. A system makes every process request all of its resources at once and blocks it until the whole set can be granted. Which are costs of this rule? Select all that apply.

- A. A process may wait a long time for its whole set, even though it could have started with part of it
- B. Resources may stay allocated for long periods without being used
- C. A process may not know in advance every resource it will need
- D. A circular wait can still form among processes that each hold part of their set
- E. In a program built from modules, the needs of every module must be known before it starts

Answer: A, B, C, E

The real costs all come from asking early and for everything. A circular wait cannot form, because under this rule a waiting process holds nothing at all.

6. Under an all-at-once rule, a process holds a plotter for its whole 12-minute run, but it uses the plotter only during its last 3 minutes. For what percentage of the holding time does the plotter sit idle?

Answer: 75 %

Held for 12 minutes, used for 3, so idle for $12 - 3 = 9$ minutes: $9 / 12 = 0.75$, or 75%. That idle time is lost resource utilization, the main price of the rule.

7. Process P holds the disk and asks for the printer, which another process holds. The system removes the no-preemption condition with the rule in which the **refused** process gives way, not the holder. What happens next?

- A. P keeps the disk and waits in line until the printer's holder releases it
- B. The OS takes the printer from its holder and gives it to P right away
- C. P releases the disk, and later requests the disk and the printer again
- D. P is terminated, and the system restarts it later from the beginning

Answer: C. P releases the disk, and later requests the disk and the printer again

Refused while holding something, a process must release all it holds and later ask again for everything, the new resource included. Waiting while holding nothing, it cannot be part of a circular wait.

8. A system prevents deadlock by letting the OS force a process to release a resource that another process asks for. What must be true for this method to work?

- A. Every process must request all of its resources at once
- B. Every resource type must be numbered and requested in increasing order
- C. Only processes that have finished their work may be asked to give resources up
- D. No two processes may have the same priority, so it is always clear which one gives way

Answer: D. No two processes may have the same priority, so it is always clear which one gives way

The OS uses priority to decide who must release. With equal priorities there is no tie-breaker, and two processes could keep taking the same resource from each other.

9. Preempting a resource is practical for the processor and for pages of memory, but not for a printer partway through printing a page.

Answer: True

The processor's state (its registers) can be saved and reloaded, and a memory page can be copied to disk and back. Half a printed page cannot be taken back, so preempting the printer would ruin the work.

10. Resource types are numbered: tape drive 1, disk 2, scanner 3, printer 4, plotter 5. Under resource ordering, a process holds the scanner and nothing with a higher number. How many of the other resource types may it still request?

Answer: 2

Holding type 3, it may request only types numbered above 3: the printer (4) and the plotter (5). That is 2. The tape drive and the disk had to be requested before the scanner, if at all.

11. Resources are numbered disk = 1, printer = 2, network = 3. Which request order obeys the resource ordering rule?

- A. `lock(printer); lock(network); lock(disk);`
- B. `lock(disk); lock(printer); lock(network);`
- C. `lock(disk); lock(network); lock(printer);`
- D. `lock(network); lock(printer); lock(disk);`

Answer: B. `lock(disk); lock(printer); lock(network);`

The rule: while holding resource i , request only resources numbered above i . Only $1 \rightarrow 2 \rightarrow 3$ climbs at every step.

12. Put the steps of the argument that resource ordering makes a circular wait impossible in order.

Correct order:

1. Assume a circular wait has formed among processes that all follow the ordering rule
2. Each process in the chain holds a resource and requests one held by the next process, so the numbers rise at every link
3. Following the chain all the way around leads back to the resource held by the first process
4. That would make the first number larger than itself, which is impossible, so no circular wait can form

It is a proof by contradiction: suppose the circle exists, notice that the numbers must strictly rise along it, and see that coming back to the start would need a number bigger than itself.

6.3 Deadlock Avoidance

You should be able to

- Explain how deadlock avoidance differs from prevention and detection, and what it must know in advance.
- Apply process initiation denial and explain why it admits far fewer processes than it could.
- Run the safety test on a state with several resource types and give a safe sequence when one exists.
- Handle a request with the banker's algorithm: report an error, make the process wait, or grant it.
- State what avoidance gains over the other strategies and the four restrictions on its use.

What deadlock avoidance is

Deadlock needs four conditions: mutual exclusion, hold and wait, no preemption and circular wait.

Prevention designs one away. **Avoidance** allows the first three (policies the system chooses) and decides carefully so that deadlock never becomes unavoidable. Before each decision it asks: "If I do this, can every process still finish?"

- It must know, in advance, each process's **maximum claim**: the most units of each resource type the process may ever hold at once.
- It allows more concurrency than prevention.
- Two approaches: **process initiation denial** (do not start a process whose claims might lead to deadlock) and **resource allocation denial** (do not grant a request that might lead to deadlock), which is Dijkstra's **banker's algorithm**.

Prevention acts when the rules are designed; avoidance when a process starts or requests; detection grants freely, looks for deadlocks later, and may have to abort processes or take resources back.

The idea with one resource type: a bank vault

A vault holds 12 coins. Four firms with credit limits Alder 8, Birch 4, Cedar 7, Dune 5 have borrowed 3, 2, 2 and 2, so the vault holds 3. A customer who reaches its limit finishes and repays everything.

- **Safe**: at least one order (a **safe sequence**) lets every customer be topped up, finish and repay. Here: Birch needs $2 \leq 3$ (vault then 5), Alder $5 \leq 5$ (then 8), Cedar $5 \leq 8$ (then 10), Dune $3 \leq 10$ (then 12).
- **Unsafe**: no such order. Lend Cedar two more coins: the vault holds 1, the needs are 5, 2, 3 and 3, none fits, so it is unsafe although a coin is left.

Unsafe is not the same as deadlocked

From that unsafe state, if every customer insists on its full limit, everyone ends up waiting forever: deadlock. But if Birch needs only the 2 coins it holds and repays early, the vault holds 3 and everyone can finish. Unsafe means deadlock can no longer be *ruled out*. Every deadlocked state is unsafe, but not every unsafe state leads to deadlock. Avoidance simply never enters unsafe states.

The bookkeeping for n processes and m resource types

Name	Meaning
Resource vector R	$R[j]$ = total units of type j the system owns
Available vector V	$V[j]$ = units of type j not allocated to anyone
Claim matrix C	$C[i][j]$ = most units of type j process i may need (declared in advance)
Allocation matrix A	$A[i][j]$ = units of type j process i holds now
Need matrix C – A	how many more units each process may still request

Three facts always hold: for every j, $R[j] = V[j] + \sum_i A[i][j]$ (every unit is free or held); $C[i][j] \leq R[j]$ (nobody claims more than exists); $A[i][j] \leq C[i][j]$ (nobody holds more than it claimed). For vectors, $x \leq y$ means every entry of x is at most the matching entry of y.

Running example (4 processes, 3 types), $R = (9, 4, 8)$:

Process	Claim C	Allocation A	Need C – A
P1	(6, 2, 4)	(0, 1, 1)	(6, 1, 3)
P2	(2, 1, 2)	(1, 0, 2)	(1, 1, 0)
P3	(6, 3, 1)	(3, 0, 1)	(3, 3, 0)
P4	(4, 2, 5)	(3, 2, 1)	(1, 0, 4)

Column sums of A are (7, 3, 5), so $V = (2, 1, 3)$.

Approach 1: process initiation denial

Start a new process P_{n+1} only if, for every resource type j : $R[j] \geq C[n+1][j] + \sum_{i=1..n} C[i][j]$. Only claims are added; current allocations do not matter.

Example: P1 then P2 gives claim totals (8, 3, 6), which fit in (9, 4, 8). Adding P3 makes (14, 6, 7), so P3 is refused. At most two of the four can run together (P1 + P2, P2 + P3 or P2 + P4), yet the banker runs all four at once in a safe state.

It is far from optimal because it assumes every process makes its maximum claim at the same moment. That rarely happens, so reserved capacity sits idle and processes that could run safely are kept out.

The safety test

A **safe state** has at least one order in which every process can get its remaining need, finish and release everything. The **safety test** searches for one:

```

work = V;                // units free right now
rest = all processes;    // nobody placed yet
while (rest not empty) { // someone is still left
    find Pk in rest with // scan rest from the front
        C[k] - A[k] <= work; // its whole need fits, every column
    if (none found) break; // stuck: stop
    work = work + A[k];    // Pk finishes and returns what it holds
    remove Pk from rest;  // Pk is next in the safe sequence
}                          // search again
safe = (rest is empty);  // safe only if everyone was placed

```

- **Why add A[k]? Pk takes its need from work, then returns its whole claim $C[k] = A[k] + \text{need}$: a net gain of A[k].**
- **Why the first fit is enough:** a finishing process only adds units to work, so picking any fitting process never spoils an order that exists. If the scan gets stuck, no order works.

Worked example: work = (2, 1, 3). P1 (6, 1, 3) is short in R1; P2 (1, 1, 0) fits, so work = (2, 1, 3) + (1, 0, 2) = (3, 1, 5). Scan again: P1 short in R1, P3 (3, 3, 0) short in R2, P4 (1, 0, 4) fits: work = (6, 3, 6). P1 fits: (6, 4, 7). P3 fits: (9, 4, 8) = R. **Safe sequence P2 → P4 → P1 → P3.**

Approach 2: the banker's algorithm (resource allocation denial)

When process i requests a vector req:

1. If $A[i] + \text{req} > C[i]$ in any column: **error** (more than it declared).
2. Else if $\text{req} > V$ in any column: **wait** (the units are not free).
3. Else make a **tentative allocation**: $A[i] = A[i] + \text{req}$ and $V = V - \text{req}$, then run the safety test.
4. If the new state is safe, keep the allocation. If it is unsafe, undo it ($A[i] = A[i] - \text{req}$, $V = V + \text{req}$) and make the process **wait**.

Requests against the example state, each judged on its own:

Request	What happens
P4 asks (1, 0, 1)	Within its claim and free. On paper $A[P4] = (4, 2, 2)$, $V = (1, 1, 2)$. Safety test: $P2 \rightarrow P4 \rightarrow P1 \rightarrow P3$, so granted .
P1 asks (0, 1, 0)	Within its claim and free. On paper $A[P1] = (0, 2, 1)$, $V = (2, 0, 3)$. Needs P1 (6, 0, 3), P2 (1, 1, 0), P3 (3, 3, 0), P4 (1, 0, 4): none fits. Unsafe, so it is undone and P1 waits although the unit was free.
P3 asks (0, 0, 1)	$A + \text{req} = (3, 0, 2)$ exceeds the claim (6, 3, 1) in R3: error .
P1 asks (3, 0, 0)	Within its claim, but only 2 units of R1 are free: P1 waits , no safety test needed.

A waiting process may succeed later, once other processes finish and release their units.

What avoidance buys and what it demands

- **Advantages:** no preemption or rollback (which detection and recovery may need), and fewer restrictions than prevention, so more processes run at once.
- **Restrictions:** (1) maximum claims stated in advance; (2) independent processes, whose order of execution is not constrained by synchronization; (3) a fixed number of resources to allocate; (4) no process may exit while holding resources.

Each restriction protects an assumption of the safety test: it needs C, assumes any finishing order is possible, trusts R, and counts on every allocation coming back. Real systems often break one, so general-purpose operating systems rarely use the banker's algorithm for all their resources.

Key terms

Deadlock avoidance

A strategy that lets processes ask for resources freely but checks each decision against the future: a request is granted only if, afterwards, every process can still be guaranteed to finish.

Deadlock prevention

Designing the system so that one of the conditions for deadlock can never hold. Deadlock becomes impossible, but resources are often used poorly.

Deadlock detection

Granting requests whenever units are free, then searching from time to time for processes that are deadlocked and breaking any deadlock that is found.

Maximum claim

The most units of each resource type that a process may ever hold at one time. The process declares it before it starts and may never go beyond it.

Process initiation denial

An avoidance rule used when a process asks to start: it is admitted only if its claim plus the claims of every process already running fit within the system's total, for every resource type.

Resource allocation denial

An avoidance rule used on every request while processes run: a request is refused for now if granting it would leave the system in an unsafe state.

Banker's algorithm

Dijkstra's resource allocation denial method. It grants a request on paper, runs the safety test on the result, and keeps the grant only if the result is safe.

Safe state

A state in which there is at least one order in which every process can be given everything it may still need, run to completion and give its resources back.

Unsafe state

A state with no safe sequence: no order is guaranteed to let every process finish. It is not necessarily deadlocked (it may already be, or everyone may still finish), but the system can no longer promise that a deadlock will not happen.

Safe sequence

An order of the processes in which each one can be satisfied from the units free at that moment plus the units returned by every process earlier in the order.

Resource vector (R)

One number per resource type: the total number of units of that type the system owns.

Available vector (V)

One number per resource type: the units of that type that no process holds right now.

Claim matrix (C)

One row per process, one column per resource type. $C[i][j]$ is the most units of type j that process i declared it may need.

Allocation matrix (A)

One row per process, one column per resource type. $A[i][j]$ is the number of units of type j that process i holds right now.

Need matrix (C – A)

The claim matrix minus the allocation matrix: how many more units of each type each process may still ask for before it reaches its maximum claim.

Safety test (safety algorithm)

The check inside the banker's algorithm: start from the free units, repeatedly find a process whose remaining need fits, pretend it finishes and returns what it holds, and call the state safe only if every process is placed.

Work vector

The safety test's running count of units it can rely on. It starts as a copy of V and grows each time a process is assumed to finish and hand back its allocation.

Tentative allocation

Granting a request on paper only (adding it to A and taking it out of V) so that the safety test can judge the result. It is undone if the result is unsafe.

Review questions with answers

1. What must deadlock avoidance know in advance that deadlock detection does not need?

- A. The set of processes that are already deadlocked when each request arrives
- B. The maximum units of each resource type that each process may ever need
- C. A fixed numbering of resource types that all requests must follow in order
- D. Exactly how long each process will run before it releases what it holds

Answer: B. The maximum units of each resource type that each process may ever need

The safety test compares each process's remaining need, claim minus allocation, with the free units. Without declared maximum claims there is no need to compare.

2. An unsafe state is the same as a deadlocked state.

Answer: False

In an unsafe state no order is *guaranteed* to let everyone finish, but processes may still release resources early or ask for less than their claim, so deadlock may never happen. Unsafe only means it can no longer be ruled out.

3. Every deadlocked state is an unsafe state.

Answer: True

The deadlocked processes can never finish, so no order exists in which every process finishes. A state with no safe sequence is unsafe by definition.

4. A system owns $R = (8, 5, 6)$. Three processes hold $(2, 1, 0)$, $(1, 2, 3)$ and $(3, 0, 1)$. How many units of the **second** resource type are available?

Answer: 2 units

Add up the second column of the allocations: $1 + 2 + 0 = 3$. Then V for that type $= 5 - 3 = 2$. Every unit is either free or held by someone.

5. During the safety test, $work = (1, 2, 0)$. The test finds that process Q, which holds $(3, 0, 2)$ and still needs $(1, 1, 0)$, can finish. After Q is assumed to finish, how many units of the **third** resource type does work hold?

Answer: 2 units

work becomes $work + A[Q] = (1, 2, 0) + (3, 0, 2) = (4, 2, 2)$. Q takes its need and then returns its whole claim, so the net change is exactly what it held.

6. $R = (6, 4)$. Two running processes have claims $(2, 1)$ and $(3, 2)$. A new process with claim $(2, 1)$ asks to start. What does process initiation denial do?

- A. Admits it: the second column of claims totals 4, which just fits
- B. Admits it, provided enough units of each type are free right now
- C. Refuses it: the second column of claims would total more than 4
- D. Refuses it: the first column of claims would total 7, more than 6

Answer: D. Refuses it: the first column of claims would total 7, more than 6

Claims add up to $(2+3+2, 1+2+1) = (7, 4)$. Since $7 > 6$, the newcomer is refused, even though the processes might never all reach their claims at once.

7. Process P has claim (3, 2), holds (1, 1) and requests (2, 0). Available V = (1, 3). What does the banker's algorithm do?

- A. Make P wait: not enough units are free
- B. Report an error: the request exceeds its claim
- C. Grant on paper and run the safety test
- D. Grant at once, because its claim covers the request

Answer: A. Make P wait: not enough units are free

Test 1 passes: $(1, 1) + (2, 0) = (3, 1) \leq (3, 2)$. Test 2 fails: P wants 2 units of the first type and only 1 is free, so P waits.

8. A system owns R = (5, 4). Claims: P1 (3, 2), P2 (2, 3), P3 (4, 1). Allocations: P1 (1, 1), P2 (1, 2), P3 (2, 0). Which of these is a safe sequence?

- A. P1 → P2 → P3
- B. P3 → P2 → P1
- C. P2 → P1 → P3
- D. None: the state is unsafe

Answer: C. P2 → P1 → P3

V = (1, 1). Needs: P1 (2, 1), P2 (1, 1), P3 (2, 1). Only P2 fits; it returns (1, 2), making work (2, 3). Now P1 fits, work becomes (3, 4), and then P3 fits.

9. Put the banker's handling of one request in order.

Correct order:

1. Check that $A[i] + \text{req}$ does not exceed the claim $C[i]$
2. Check that req does not exceed the available vector V
3. Tentatively grant: $A[i] = A[i] + \text{req}$ and $V = V - \text{req}$
4. Run the safety test on the new state
5. Keep the grant if safe; otherwise undo it and make the process wait

An over-claim is an error, so it is caught first. Then the units must actually be free. Only then does the banker grant on paper, test, and keep or undo.

10. Which are requirements for using the banker's algorithm?

- A. Each process declares its maximum claim in advance
- B. Processes are independent: no synchronization fixes their order
- C. The number of resources to allocate is fixed
- D. No process exits while holding resources
- E. Resources are numbered and must be requested in increasing order
- F. Each process requests all its resources at once

Answer: A, B, C, D

The first four are the restrictions on avoidance. Numbered ordering and all-at-once requests are prevention rules; avoidance needs neither.

11. Match each symbol of the banker's bookkeeping with its meaning.

R	Total units of each type the system owns
V	Units of each type free right now
C	Most units each process declared it may need
A	Units each process holds right now
$C - A$	Units each process may still ask for

R and V are vectors (one number per type). C, A and $C - A$ are matrices (one row per process). Always $R = V +$ the column sums of A.

12. Which strategy does each statement describe?

Prevention	Avoidance	Detection
Resources must be requested in a fixed numeric order A process must request all its resources at once	A request whose tentative result is unsafe is refused Maximum claims must be declared before a process starts	Every so often, search for processes stuck waiting on each other Processes may be aborted to break a deadlock

Prevention bans a condition by rule. Avoidance looks ahead at each decision using declared claims. Detection lets deadlocks happen, finds them, and recovers.

6.4 Deadlock Detection

You should be able to

- Explain how deadlock detection differs from prevention and avoidance, and weigh how often to run it.
- Run the detection algorithm by hand on an allocation matrix, a request matrix and an available vector, and name the deadlocked processes.
- Explain how the detection algorithm differs from the banker's safety test, and why it is called optimistic.
- Find a deadlock with a wait-for graph when every resource type has a single unit.
- Compare the four recovery strategies and choose a victim using the standard criteria.

Deadlock detection: grant now, look later

Prevention and avoidance pay a price all the time so that deadlock can never happen. **Deadlock detection** makes the opposite bet. The operating system puts no limits on what processes request or hold: if the units a process asks for are free, it gets them at once, and if not, it waits as usual. Every so often the OS runs a **detection algorithm** over the current state to find processes caught in a circular wait. If it finds any, it breaks the deadlock (**deadlock recovery**). The loop is: grant, detect, recover if needed, carry on.

Approach	When it acts	Needs in advance	Main cost
Prevention	by design: a condition is ruled out	a policy all obey	idle resources, constrained programs
Avoidance	at each request: refuse risky ones	maximum claims	waits even when units are free
Detection	afterwards: find, then recover	nothing	the checks, plus work lost in recovery

How often to run the check

- **At every request.** A deadlock is caught by the very request that creates it, and because only that one request changed the state since the last check, the algorithm can be incremental and simple. The price is processor time on every request.
- **Less often:** on a timer, or when processor utilization drops (deadlocked processes stop working and others pile up behind them). Far cheaper, but a deadlock sits unnoticed until the next check, freezing its processes and their resources. Innocent utilization dips (a burst of disk transfers) trigger checks that find nothing.

Worked example. A full check costs 6 ms. Run every 10 s, it uses $6 / 10,000 = 0.06\%$ of the processor, and a deadlock waits on average half an interval (5 s), at worst a whole one (10 s). Checking each of 400 requests a second at 0.05 ms each costs 20 ms per second, 2% of the processor, but finds the deadlock at once.

The data the algorithm reads

- **Allocation matrix A:** one row per process, one column per resource type; $A[i][j]$ = units of resource j that process i holds.
- **Request matrix Q:** $Q[i][j]$ = units of resource j that process i has asked for and is still waiting to receive (all zeros means it is not waiting).
- **Available vector V:** free units of each type, which is the total minus everything held.
- **Work vector W:** a scratch copy of V that grows as the algorithm pretends processes finish.

The detection algorithm

The algorithm marks every process it can show is **not** deadlocked.

1. Mark every process whose row of A is all zeros. It holds nothing, so nobody can be waiting on it, and it cannot be part of a deadlock.
2. Set $W = V$.
3. Find an unmarked process i whose row of Q is $\leq W$ in every column. If there is none, stop.
4. Mark i and add its row of A to W (assume it gets what it asked for, finishes and releases everything). Go back to step 3.

A deadlock exists if and only if some processes are still unmarked at the end, and the unmarked processes are exactly the deadlocked ones. The order in which fitting processes are chosen does not change the verdict, because W only ever grows.

Worked example (5 processes, 5 resource types)

Totals (1, 1, 2, 1, 2), so $V = (0, 0, 0, 0, 1)$.

	A R1	A R2	A R3	A R4	A R5	Q R1	Q R2	Q R3	Q R4	Q R5
P1	1	0	1	0	0	0	1	0	0	0
P2	0	1	0	0	0	1	0	0	0	1
P3	0	0	0	0	1	0	0	0	1	1
P4	0	0	1	1	0	0	0	0	0	0
P5	0	0	0	0	0	0	0	1	0	0

1. Step 1: mark P5 (row of A all zeros).
2. Step 2: $W = V = (0, 0, 0, 0, 1)$.
3. Step 3: P1 (needs R2), P2 (needs R1) and P3 (needs R4) do not fit; P4 asks for nothing and fits.
4. Step 4: mark P4; $W = (0, 0, 0, 0, 1) + (0, 0, 1, 1, 0) = (0, 0, 1, 1, 1)$.
5. Step 3: P1 (needs R2) and P2 (needs R1) do not fit; P3 asks for $(0, 0, 0, 1, 1)$ and fits.
6. Step 4: mark P3; $W = (0, 0, 1, 1, 1) + (0, 0, 0, 0, 1) = (0, 0, 1, 1, 2)$.
7. Step 3: P1 (needs R2) and P2 (needs R1) do not fit, so the algorithm stops.

Result: P1 and P2 are unmarked, so deadlocked (P1 waits for R2, held by P2; P2 waits for R1, held by P1).

P5 waits too, but holds nothing, so it is not part of the cycle. A waiting process's request never fits in V (the OS would simply have granted it), so it can be marked only after others add their units to W . A row of Q that is all zeros means the process is running.

Detection versus the banker's safety test

Both start from what is free and pretend processes finish one at a time, growing W . They differ in what they assume each process still needs:

- **Banker's safety test (avoidance)** compares each process's remaining claim $C - A$ (the worst case: everything it may still ask for) with W . It runs **before** a request is granted and answers "is the state safe?".
- **Detection** compares each process's current request Q with W . It is **optimistic**: it assumes a process whose request can be met will finish without asking for more. It runs **afterwards** and answers "which processes are deadlocked now?". It needs no maximum claims, and it marks processes holding nothing outright.

Example. Totals $(4, 2, 2)$, held $A = (2, 0, 1), (1, 1, 0), (0, 1, 1)$, so $V = (1, 0, 0)$. Remaining claims $C - A = (1, 1, 0), (1, 1, 1), (3, 1, 0)$: none fits in V , so the banker's test says **unsafe**. Current requests $Q = (0, 0, 0), (0, 0, 1), (2, 0, 0)$: detection marks P1, P2 and P3 in turn, so there is **no deadlock**. Unsafe does not mean deadlocked. If P1 then asks for the rest of its claim instead of finishing, P1, P2 and P3 become deadlocked, and the next detection run catches it. When every process asks for its whole remaining claim ($Q = C - A$) and no row of A is all zeros, the two tests make exactly the same moves.

Special case: one unit of each resource type

When every resource type has a single unit, a waiting process can be released only by the one process holding that unit. Collapse the resource allocation graph into a **wait-for graph**: one node per process, an arrow $P_i \rightarrow P_j$ when P_i waits for a resource that P_j holds. **A deadlock exists exactly when the wait-for**

graph contains a cycle; the processes on the cycle are deadlocked, and a process whose arrows lead into a cycle is stuck as well until the cycle is broken. A depth-first search finds cycles (reaching a process already on the current path closes one); it handles each process and arrow once, at most about n^2 work for n processes. With several units per type a cycle alone does not prove deadlock, so the general case needs the matrix algorithm.

Recovery, from crude to refined

1. **Abort all deadlocked processes.** Needs nothing special; all their work is lost. The most common choice in real operating systems.
2. **Roll back each deadlocked process to a checkpoint and restart it.** Needs checkpoint and rollback support. Only work since the checkpoint is lost, but the same deadlock may recur; timing differences usually prevent it.
3. **Abort deadlocked processes one at a time**, cheapest first, rerunning the detection algorithm after each abort, until no deadlock remains.
4. **Preempt resources one at a time**, rerunning detection after each preemption. The victim must be rolled back to a point before it acquired the resource. If the same process is always chosen it may starve, so a fair policy counts how often each has been a victim.

Choosing a victim (for options 3 and 4)

- Least processor time consumed so far (easy: the OS accounts it).
- Least output produced so far (fairly easy to count).
- Most estimated time remaining (hard: only an estimate; spares nearly finished work).
- Fewest total resources allocated so far (easy, but freeing little may not break the cycle).
- Lowest priority (easy where priorities exist; ignores how much work is lost).

Lab results. Four deadlocked processes (P1 and P2 wait for each other, P4 waits for P2, P3 waits for P4), choosing victims greedily and rerunning detection after each abort:

Strategy	Victims	CPU time lost	Output lost
Abort all	P1, P2, P3, P4	155 s	920 lines
Roll back all	P1, P2, P3, P4	42 s	180 lines
Least processor time used so far	P4, P1	50 s	800 lines
Least output produced so far	P2	75 s	40 lines
Most estimated time remaining	P1	40 s	300 lines
Fewest resources held	P3, P4, P1	80 s	880 lines
Lowest priority	P3, P2	105 s	120 lines

No criterion wins on every measure, and a cheap victim off the cycle only adds aborts. Rerunning detection after each abort tells the OS when to stop.

Key terms

Deadlock detection

An approach to deadlock in which the operating system grants every request it can, runs an algorithm from time to time to find deadlocked processes, and then recovers.

Detection algorithm

The procedure that looks at who holds what and who is waiting for what, marks every process it can show is not deadlocked, and reports the processes left unmarked as deadlocked.

Allocation matrix (A)

A table with one row per process and one column per resource type. Entry $A[i][j]$ is how many units of resource j process i holds right now.

Request matrix (Q)

A table with one row per process and one column per resource type. Entry $Q[i][j]$ is how many units of resource j process i has asked for and is still waiting to receive.

Available vector (V)

One number per resource type: how many units of that resource are free right now, held by no process.

Work vector (W)

A scratch copy of the available vector that the detection algorithm grows as it pretends that marked processes finish and hand back what they hold.

Marked process

A process the detection algorithm has shown is not deadlocked: it holds nothing, or everything it is waiting for could be supplied.

Wait-for graph

A graph with one node per process and an arrow from P_i to P_j whenever P_i is waiting for a resource that P_j holds. When every resource type has one unit, a cycle in it means deadlock.

Circular wait

A closed chain of processes in which each one holds a resource that the next one in the chain is waiting for.

Resource allocation graph

A drawing of processes (circles) and resource types (boxes with one dot per unit), with an arrow for every request and for every unit held.

Deadlock recovery

What the operating system does after it finds a deadlock: abort processes, roll them back, or take resources away until the circular wait is broken.

Checkpoint

A saved copy of a process's state at one moment, so the process can later be restarted from that point instead of from the beginning.

Rollback

Returning a process to an earlier checkpoint: everything it did after that point is thrown away and it starts again from there.

Victim

The process chosen to be aborted, rolled back, or stripped of a resource so that a deadlock can be broken.

Resource preemption

Taking a resource away from the process holding it before that process has finished with it.

Deadlock prevention

Designing the system so that one of the conditions for deadlock can never hold, which makes deadlock impossible.

Deadlock avoidance

Deciding at each request, from the current state and each process's declared maximum needs, whether granting it could lead to deadlock, and making the process wait if it could.

Banker's algorithm

The classic avoidance method: grant a request only if the state afterwards is still safe, judged against each process's maximum claim.

Claim matrix (C)

In deadlock avoidance, the table of the largest number of units of each resource type that each process may ever ask for.

Review questions with answers

1. An operating system handles deadlock by detection. A process asks for two units of a resource, and three units are free. What does the OS do?

- A. First checks whether the resulting state would be safe
- B. Refuses unless the process has declared its maximum needs
- C. Grants the two units immediately
- D. Runs the detection algorithm before deciding

Answer: C. Grants the two units immediately

Detection puts no limits on requests: if the units are free, they are granted. The OS looks for deadlocks later and recovers if it finds one.

2. Which statements about running the detection algorithm at **every** resource request are true?

- A. A deadlock is caught by the very request that creates it
- B. The check can be incremental, which keeps the algorithm simple
- C. It costs processor time on every request
- D. It requires every process to declare its maximum claim
- E. A deadlock can sit unnoticed until the next timer tick

Answer: A, B, C

Checking at each request finds a deadlock as soon as it forms, and since only one request changed the state, the check can be incremental. The price is work on every request. Maximum claims belong to avoidance, and timer delays belong to periodic checking.

3. Put the steps of the deadlock detection algorithm in order.

Correct order:

1. Mark every process whose row of the allocation matrix is all zeros
2. Set a temporary vector W equal to the available vector V
3. Find an unmarked process whose row of the request matrix is $\leq W$; if there is none, stop
4. Mark that process, add its row of the allocation matrix to W, and search again
5. Report every process that is still unmarked as deadlocked

Processes holding nothing are marked first. W starts as the free units, grows each time a process is marked (it is assumed to finish and release what it holds), and whoever is left unmarked when no request fits is deadlocked.

4. The detection algorithm runs on this state (four processes, three resource types). How many processes are deadlocked?

	A = held			Q = requested		
	R1	R2	R3	R1	R2	R3
P1	1	0	0	0	1	0
P2	0	1	0	1	0	0
P3	0	0	1	0	0	0
P4	0	0	0	0	0	2
Available V = (0, 0, 1)						

Answer: 2

P4 holds nothing and is marked in step 1. $W = (0, 0, 1)$. P3 asks for nothing, so it is marked and W becomes $(0, 0, 2)$. Neither P1 (needs R2) nor P2 (needs R1) fits, so 2 processes, P1 and P2, are deadlocked: each waits for what the other holds.

5. When the deadlock detection algorithm stops, the processes left unmarked are exactly the deadlocked processes.

Answer: True

Every marked process can be shown to finish in some order. A process left unmarked waits for units that no sequence of finishing processes can ever free, so it is deadlocked.

6. If the banker's safety test calls a state unsafe, the detection algorithm will report a deadlock in that same state.

Answer: False

Unsafe only means a deadlock could happen if processes asked for everything they may still claim. Detection looks only at the requests that exist now, and in an unsafe state it may still find that every process can finish.

7. Why is the deadlock detection algorithm called **optimistic**?

- A. It assumes any process whose current request fits will finish without asking for more
- B. It assumes deadlocks are impossible, so it can never report a deadlocked process
- C. It grants requests even when the units are not free, expecting them to be freed soon
- D. It assumes every process will soon ask for its whole maximum claim before it finishes

Answer: A. It assumes any process whose current request fits will finish without asking for more

When a request fits in W, the algorithm marks the process and adds everything it holds to W, as if it will run to completion. If the process later asks for more, any new deadlock is caught by a later run.

8. Every resource type in a system has exactly one unit. Which finding proves that a deadlock exists?

- A. A process with two outgoing arrows in the wait-for graph
- B. A process with no outgoing arrows
- C. More arrows than processes
- D. A cycle in the wait-for graph

Answer: D. A cycle in the wait-for graph

With one unit per type, $P_i \rightarrow P_j$ means only P_j can release what P_i needs. A cycle is a closed loop of such waits, so nobody in it can ever proceed.

9. Match each recovery strategy to the description that fits it.

Abort all deadlocked processes	The simplest and most common choice; all their work is lost
Roll back to checkpoints and restart	Needs saved states, and the same deadlock may form again
Abort processes one at a time	Reruns the detection algorithm after each abort
Preempt resources one at a time	The victim is rolled back to before it acquired the resource

The strategies run from crude to refined: abort everyone, roll everyone back, abort the cheapest victims one by one, or take resources away one by one.

10. Choosing a victim: for each measure, does the usual criterion pick the process with the smallest value or the largest?

Pick the smallest	Pick the largest
Processor time consumed so far Output produced so far Resources allocated so far Priority	Estimated time remaining

The aim is to lose the least: least computing time, least output, fewest resources, lowest priority. Time remaining is the exception: pick the process furthest from finishing, so nearly-done work is spared.

11. One full run of the detection algorithm costs 5 ms of processor time. The OS runs it once every 2 seconds. What percentage of the processor's time goes to detection?

Answer: 0.25 %

5 ms out of every 2000 ms is $5 / 2000 = 0.0025$, which is 0.25%. Checking less often lowers this cost, but a deadlock then waits up to 2 s to be found.

12. Recovery by aborting deadlocked processes one at a time: why does the OS rerun the detection algorithm after each abort?

- A. To restart the aborted process from its most recent saved checkpoint
- B. To find out whether the deadlock is gone or another victim is needed
- C. To recompute each remaining process's maximum claim after the abort
- D. To give the aborted process its resources back once the others finish

Answer: B. To find out whether the deadlock is gone or another victim is needed

Freeing one victim's resources may or may not break the cycle. Rerunning detection tells the OS whether it can stop or must choose the next victim.

6.5 An Integrated Deadlock Strategy

You should be able to

- Explain why no single deadlock method suits every kind of resource, and what an integrated strategy does instead.
- Name the four resource classes in the order a process acquires them, with examples of each.
- Choose the best method for each class and justify it from how its resources behave: known in advance, cheap to take back, or held only briefly.
- Explain why numbering the classes makes a deadlock that crosses from one class to another impossible.
- Compare the integrated design with one-method-for-everything designs on waiting time, idle resources, checking overhead and deadlock risk.

Why mix methods?

Each deadlock approach works but has a price. **Prevention** rules out one of the four conditions by design (price: idle resources, longer waits, redone work). **Avoidance** grants a request only if the state stays safe (price: needs declared in advance, a test per request). **Detection** lets deadlocks happen, then finds and breaks them (price: frozen processes, aborted or rolled-back work).

No approach is cheapest for every kind of resource, so an **integrated deadlock strategy**:

1. groups resources that behave alike into a few **resource classes**;
2. puts the classes in a fixed order (**class ordering**): a process holding resources from one class may afterwards ask only for that class or a higher-numbered one (resource ordering, applied to whole classes);
3. uses, inside each class, the method that suits that class best.

Using one method for everything (a **uniform strategy**) is a poor fit for at least one class.

The four classes, in the order they are assigned

Class	Best method	Why it fits
1. Swappable space: disk blocks that hold a process's memory image while it is swapped out	Prevention: all the space in one request (no hold and wait). Avoidance also possible.	The size is known when the process is created, and the space is needed for the whole run anyway.
2. Process resources: assignable devices such as tape drives, and files opened for exclusive use	Avoidance: the banker's test on each request. Ordering inside the class also possible.	Jobs declare their devices and files in advance, exactly the maximum claim avoidance needs. Requests are few, so tests are cheap.
3. Main memory, handed out in pages or segments	Prevention by preemption: swap a process out to secondary storage, freeing its memory.	Memory contents can be copied out and restored exactly, so no work is lost.
4. Internal resources, such as I/O channels	Prevention by resource ordering.	Uses are brief and very frequent; a fixed order written into the system's code costs nothing at run time.

Three questions that decide the method

- **Is the need known in advance?** Reserve it all at once, or use avoidance with the declared claim.
- **Can it be taken back cheaply?** Preempt it (memory: swap the process out).
- **Is it grabbed briefly and very often?** Use a fixed number order; any run-time check or long reservation costs far more than the use.

Why the wrong pairings fail

- Preempting a tape drive mid-job ruins the half-written tape (a half-updated file may be left inconsistent). Preempting swap space leaves a swapped-out image nowhere to go.
- Reserving a channel for a whole run locks others out while it sits idle almost all the time; a safety test on each of thousands of brief channel uses costs more than the transfers.
- Swap blocks and memory frames are interchangeable: a process needs a number of them, not particular ones, so ranking them gives no useful order.
- Detection means frozen processes and lost work, a needless cost where prevention or avoidance is cheap.

Common mistake: asking which method is best *overall*. None is.

Why ordering the classes prevents deadlock across classes

1. In a deadlock, processes form a loop: each holds a resource and waits for one held by the next.
2. With class ordering each wait goes to the same class or a higher one, so class numbers never go down around the loop.
3. A loop ends where it began, so numbers that never go down never went up either: every resource in the loop is in one class.
4. Inside one class, its own method rules the loop out: all-at-once means nobody holds swap space while waiting for more; avoidance never enters an unsafe state, and a deadlock is unsafe; preemption means nobody waits for memory forever; numbered channels give the same argument one level down.

This is the argument for numbering single resources, used on whole classes. It forbids only requests back down, not a second request in the same class (that is the class method's business). Its cost: a process must take resources in class order, for example its tape drive before its memory (or give the memory back first).

Worked example: three jobs through the four classes

System: 10 GB of swap space, 3 tape drives, 7 GB of memory, channels CH1 and CH2. Jobs (priority $A > B > C$): A: 4 GB swap, claim 3 drives, 4 GB memory, uses CH1 then CH2; B: 3 GB swap, claim 2 drives, 3 GB memory, uses CH2 then CH1; C: 5 GB swap, claim 0 drives, 5 GB memory, uses CH1.

- **Class 1.** C asks for 5 GB of swap space with only 3 GB free. The request is refused as a whole; C waits holding nothing.
- **Class 2.** A asks for 2 drives: on paper 1 would be left, A could finish with it and then B, so the state is safe and the grant stands. B then asks for 1: on paper none would be left while A still needs 1 and B still needs 1, an unsafe state, so B waits although a drive is free.
- **Class 3.** Later B needs 3 GB of memory with only 2 GB free. C, the lowest-priority job in memory, is swapped out into the 5 GB of swap space it reserved in class 1, and swapped back in when B finishes.
- **Class 4.** B's program uses CH2 before CH1, but the rule makes it ask for CH1 first.

Whole run: 5 safety tests (all in class 2), 2 requests made to wait, 1 swap-out, 1 swap-in, and no deadlock possible at any moment.

One rule for everything vs. the integrated design

Three jobs (each needs one of 2 tape drives, plus spells of I/O on CH1, CH2 or both) run minute by minute under four designs:

Design	All done at (min)	Channels busy / held (min)	Safety tests	Deadlock
Integrated: one method per class	12	17 / 17	3	impossible
All at once, for everything	26	17 / 52	0	impossible
The banker, for everything	13	17 / 17	20	impossible
Detect and recover, for everything	18	20 / 24	0	1 found, 7 min of work lost

- **All at once** is safe but slow: each job holds both channels and its drive for its whole run, so the channels are busy only 33% of the time they are held. The rest is **held-but-idle time**.
- **The banker** finishes close behind but runs many more safety tests, and every job must declare its maximum need for every resource, channels included.
- **Detection** lets a channel deadlock form (one job holds CH1 and waits for CH2, another holds CH2 and waits for CH1); recovery aborts a job and throws its work away.
- **The integrated design** pays each class only its own cheapest price. In this workload it finishes first, though not best in every column (the class order makes a job take its tape before its memory, so a drive can sit idle).

Key terms

Integrated deadlock strategy

A design that sorts resources into classes, puts the classes in a fixed order, and inside each class uses whichever deadlock method suits that class best.

Resource class

A group of resources that behave alike (how long they are held, whether needs are known ahead, whether they can be taken back cheaply) and are therefore handled by one deadlock method.

Class ordering

The rule that numbers the resource classes: a process that holds resources from one class may afterwards ask only for resources from that class or a higher-numbered one, never a lower-numbered one.

Uniform strategy

Using one deadlock method, such as all-at-once requests or the banker's algorithm, for every resource in the system, whatever its kind.

Swappable space

Blocks of secondary storage (usually disk) set aside to hold a process's memory image while the process is swapped out of main memory.

Process resources

Devices and files that are assigned to one process at a time for long stretches, such as tape drives and files opened for exclusive use.

Assignable device

A device that the operating system hands to one process for as long as that process needs it, instead of sharing it request by request. A tape drive is the classic example.

Tape drive

A device that reads and writes a reel or cartridge of magnetic tape in order, from start to end. A job mounts its tape and keeps the drive for a long time. Tapes are still used for backups and archives.

Internal resources

Resources the system itself uses for a moment at a time while doing work for processes, such as I/O channels. Each use is short, but uses are very frequent.

Preemption by swapping

Taking main memory back from a process by copying its memory image to its swappable space and suspending it. Later it is swapped back in and continues exactly where it stopped.

Held-but-idle time

Time during which a resource is assigned to a process that is not actually using it, so nobody else can use it either. It is the main waste of asking for everything at once.

Review questions with answers

1. In the integrated strategy, which method handles **main memory**?

- A. Avoidance: run the banker's safety test on every allocation
- B. All at once: each process reserves its largest memory need for its whole run
- C. Preemption: swap a process out to its swap space and bring it back later
- D. Detection: let memory deadlocks happen, then abort a process

Answer: C. Preemption: swap a process out to its swap space and bring it back later

Memory is cheap to preempt: its contents can be copied to swap space and restored exactly, so no work is lost. Taking it back removes the no-preemption condition for this class.

2. Put the four resource classes in the order in which a process is assigned them.

Correct order:

- 1. Swappable space
- 2. Process resources (devices and files)
- 3. Main memory
- 4. Internal resources (I/O channels)

Swap space first, then devices and files, then memory, then internal resources. One useful effect of this order: a process holding memory already owns the swap space it would be swapped out into.

3. Match each class to the method that suits it best.

Swappable space	Allocate all of it in one request
Process resources	Avoidance with declared claims
Main memory	Preemption by swapping out
Internal resources	Resource ordering

Each method fits what is cheap for the class: known sizes for swap space, declared claims for devices and files, cheap save-and-restore for memory, and brief, frequent use for channels.

4. Sort each resource into its class.

Swappable space	Process resources	Main memory	Internal resources
Blocks of the disk's swap area	A tape drive A file opened for exclusive update	Page frames	An I/O channel

Swap blocks hold swapped-out images; drives and exclusively used files are long-held process resources; page frames are main memory; channels are used briefly by the system itself.

5. Under class ordering, a process that holds main memory may then ask for a tape drive.

Answer: False

Tape drives are in class 2 and memory is in class 3. Asking for a drive while holding memory would go back down to a lower class, which the ordering forbids. The process must get its drive before its memory, or give its memory back first.

6. Ordering the classes is enough, by itself, to prevent every deadlock, including one among resources of a single class.

Answer: False

The ordering only rules out waiting loops that cross from one class to another. A loop inside a single class is stopped by that class's own method, which is why each class still needs one.

7. Why is preemption a poor method for tape drives?

- A. Tape drives cannot be numbered, so the OS cannot pick one to take back
- B. A half-written tape is ruined, so the job's tape work must be redone
- C. Preemption works only for the processor, never for devices like drives
- D. Jobs cannot know in advance which drives they will need while they run

Answer: B. A half-written tape is ruined, so the job's tape work must be redone

A resource is a good candidate for preemption only if its state can be saved and restored cheaply. A tape drive in mid-job cannot, so its class uses avoidance instead.

8. Which reasons explain why all-at-once allocation suits swappable space?

- A. The amount a process needs is known when it is created
- B. Swap space can be taken back cheaply
- C. The space is needed for the whole run anyway, so reserving it early wastes little
- D. Swap space is used for only a few milliseconds at a time
- E. A process waiting for swap space holds none of it, so hold and wait cannot occur in this class

Answer: A, C, E

Known size plus whole-run use makes the all-at-once rule cheap, and it removes hold and wait. Swap space cannot be taken back cheaply (it holds swapped-out images), and it is held for a long time, not milliseconds.

9. A system has 12 GB of swap space. Running processes hold 4 GB and 3 GB of it. A new process needs 6 GB and, following the integrated strategy, asks for all of it in one request. How many GB does it receive right away?

Answer: 0 GB

$12 - 4 - 3 = 5$ GB are free, less than the 6 GB requested. The request is refused as a whole, so the process gets 0 GB and waits holding nothing: no hold and wait.

10. A job keeps an I/O channel reserved for its entire 50-minute run, but it actually transfers data on the channel for a total of 30 seconds. What percentage of the reserved time is the channel busy?

Answer: 1 %

50 minutes is 3,000 seconds, and $30 / 3,000 = 1\%$. The channel sits held but idle 99% of the time, which is why all-at-once is a poor fit for internal resources.

11. A system has 3 tape drives, managed by avoidance. Job A (claim 3) holds 2 drives; job B (claim 2) holds none; 1 drive is free. B asks for 1 drive. What happens?

- A. B gets the drive at once, because one drive is still free at the moment it asks
- B. The system preempts a drive from A and hands it to B, since B holds no drive yet
- C. B gets the drive, because the request stays within its declared claim of 2 drives
- D. B waits: granting would leave an unsafe state (0 free; A and B each still need 1)

Answer: D. B waits: granting would leave an unsafe state (0 free; A and B each still need 1)

Grant on paper: 0 drives free, A needs 1 more, B needs 1 more. Neither can be sure to finish, so the state would be unsafe and B waits, even though a drive is free.

12. Why does ordering the classes make a deadlock involving resources from two different classes impossible?

- A. Each class runs its own detector that breaks any loop it finds
- B. A process may hold resources from only one class at any time
- C. Class numbers can never go down around a waiting loop, so a loop cannot leave its class
- D. The system preempts a resource from the higher class whenever a loop appears

Answer: C. Class numbers can never go down around a waiting loop, so a loop cannot leave its class

Each request goes to the same or a higher class. A loop must end where it began, so numbers that never go down can never have gone up either: every resource in the loop is in one class, where that class's method takes over.

6.6 Dining Philosophers Problem

You should be able to

- Describe the dining philosophers setup and the three things any solution must guarantee.
- Trace the one-semaphore-per-fork solution and show exactly how it deadlocks, pointing to all four deadlock conditions at the frozen table.
- Explain why a room semaphore that admits four diners, or an asymmetric pick-up order, makes deadlock impossible.
- Follow the monitor solution with cwait and csignal and explain why its circle of waiting can never close.
- Tell deadlock freedom apart from starvation freedom using a concrete schedule.

The dining philosophers problem

Five philosophers sit at a round table and alternate between thinking and eating. There is a bowl of spaghetti in the middle, a plate at each seat and **one fork between each pair of neighbours**, five forks in total. To eat, a philosopher needs **both** neighbouring forks: the one on its left and the one on its right (from its own seat). A fork can be held by only one philosopher at a time, so neighbours compete for the fork they share.

Number the seats 0 to 4. Philosopher i 's left fork is fork i and its right fork is fork $(i + 1) \bmod 5$, so fork k is shared by philosophers $k - 1$ and k . Each meal uses two of the five forks, so **at most two philosophers can eat at once**, and never two neighbours.

A solution is a ritual every philosopher follows. It must guarantee:

1. **Mutual exclusion:** no fork is used by two philosophers at the same time.
2. **No deadlock:** the table never reaches a state where everyone waits forever.
3. **No starvation:** every hungry philosopher eventually eats.

Why it matters: the philosophers are threads and the forks are locks. Any thread that needs two shared resources at once (two bank accounts for a transfer, two files for a copy) faces the same danger.

First attempt: one semaphore per fork

```
semaphore fork[5] = {1, 1, 1, 1, 1}; // 1 means "on the table"
void philosopher(int i) {
    while (true) {
        think();
        semWait(fork[i]);           // left fork
        semWait(fork[(i + 1) % 5]); // right fork
        eat();
        semSignal(fork[(i + 1) % 5]); // right fork back
        semSignal(fork[i]);         // left fork back
    }
}
```

Each fork semaphore starts at 1, so only one philosopher can hold a fork: mutual exclusion holds. But if all five become hungry together and each picks up its left fork, every right fork is already in a neighbour's hand. Each philosopher blocks on its right fork while holding its left one, and every fork semaphore ends at -1 (one waiter each). P0 waits for P1, P1 for P2, P2 for P3, P3 for P4 and P4 for P0: **deadlock**. Nobody broke the rules; the timing alone caused it.

The four deadlock conditions at the frozen table

Condition	How it appears	How it could be broken
Mutual exclusion	Each fork is in exactly one hand.	Cannot be dropped: two people cannot share one fork.
Hold and wait	Everyone holds its left fork while waiting for the right one.	Take both forks in one step or none; or need only one fork.
No preemption	Nobody may take a fork out of a neighbour's hand.	Put the left fork back if the right one is busy, and retry. Danger: livelock, all five picking up and putting down in step forever.
Circular wait	The wait-for graph is a loop $P0 \rightarrow P1 \rightarrow P2 \rightarrow P3 \rightarrow P4 \rightarrow P0$.	Room semaphore, asymmetric order, or a monitor.

Two quick fixes change the problem rather than solve it: **more forks** (everyone owns a pair, so nothing is shared; real systems often have only one copy of a resource) and **eat with one fork** (no hold and wait, but the job changes).

The standard fix: a room semaphore of 4

```
semaphore fork[5] = {1, 1, 1, 1, 1};
semaphore room = 4;    // at most four seated
void philosopher(int i) {
    while (true) {
        think();
        semWait(room);    // sit down
        semWait(fork[i]);
        semWait(fork[(i + 1) % 5]);
        eat();
        semSignal(fork[(i + 1) % 5]);
        semSignal(fork[i]);
        semSignal(room);  // stand up
    }
}
```

Why it works (the **pigeonhole principle**): at most four philosophers sit at the table, but there are five forks. If every fork were taken, one of the four would hold two forks and be eating, so it will finish and release them. And the seated philosopher whose right-hand neighbour is not seated never waits for its right fork, because nobody else can be holding it. Someone can always make progress, so a full circle of waiting cannot form: **no deadlock**. With first-in-first-out (strong) semaphores every waiter is served in turn, so there is **no starvation** either. If all five call `semWait(room)` before anyone leaves, `room` goes `4` $\rightarrow$ `-1`: four sit, one waits at the door. A room of 5 admits everybody and brings the deadlock back; smaller values also avoid deadlock but allow less parallelism (a room of 1 lets only one philosopher eat at a time). A model check of every order of moves confirms: rooms 1 to 4 never freeze; 5 can.

Asymmetric order (resource ordering)

Let at least one philosopher pick up its **right** fork first. The classic version: number the forks and always pick up the lower-numbered fork first. For philosophers 0 to 3 the lower number is the left fork; for philosopher 4 (forks 4 and 0) it is the right fork, fork 0. A right-hander and its right-hand neighbour (a left-hander) now reach first for the same fork, so the loser holds nothing while it waits, and the waits cannot all point the same way round the table. This is **resource ordering** from deadlock prevention: it breaks circular wait. Any mix of left- and right-first philosophers works; all left or all right can freeze.

The monitor solution

```
monitor dining_controller;
cond ForkReady[5];           // a waiting line per fork
boolean fork[5];             // true = on the table (all start true)
void get_forks(int pid) {
    int left = pid, right = (pid + 1) % 5;
    if (!fork[left]) cwait(ForkReady[left]); // wait, holding nothing
    fork[left] = false;
    if (!fork[right]) cwait(ForkReady[right]); // wait, holding left
    fork[right] = false;
}
void release_forks(int pid) {
    int left = pid, right = (pid + 1) % 5;
    if (empty(ForkReady[left])) fork[left] = true; // back on the table
    else csignal(ForkReady[left]);                // hand it to the waiter
    if (empty(ForkReady[right])) fork[right] = true;
    else csignal(ForkReady[right]);
}
void philosopher(int k) {
    while (true) { think(); get_forks(k); eat(); release_forks(k); }
}
```

Only one philosopher is active inside the monitor at a time, so taking the left fork and checking the right one happen in one visit with nobody slipping in between. A philosopher may still wait in `ForkReady[right]` while holding its left fork, but the circle cannot close: in an all-left state, whoever took its left fork first would have found its right fork free and taken it too. `csignal` performs a **handoff**: the waiter resumes at once (classic monitor semantics) and gets the fork directly, so no neighbour can grab it first. Only two philosophers share a fork, so each `ForkReady` queue holds at most one waiter, served at that fork's next release: as long as meals end, **nobody starves**.

Deadlock freedom is not starvation freedom

Deadlock freedom: some process can always make progress. **Starvation freedom**: every process eventually does. The rule "pick up both forks in one step, only when both are free" never deadlocks (nobody holds and waits), but if P1 and P3 overlap their meals, f2 and f3 are each free at times and never at the same time, so P2 can wait forever. Adding fairness (defer to a neighbour that has been hungry longer, much as a FIFO queue would) lets P2 eat, at the price of forks sometimes lying idle.

Testing is not proof: random schedules reveal the first attempt's deadlock only sometimes (16 of 100 runs of 300 moves in this guide's lab), while an exhaustive model check proves the room, asymmetric and monitor solutions can never freeze.

Key terms

Dining philosophers problem

A classic coordination puzzle: five philosophers around a round table share five forks, one between each pair of neighbours, and each needs both of its neighbouring forks to eat. A solution must keep every fork exclusive and be free of deadlock and starvation.

Hold and wait

A deadlock condition: a process keeps the resources it already has while it waits for more.

No preemption

A deadlock condition: a resource cannot be taken away from the process that holds it; it is only ever handed back voluntarily.

Circular wait

A deadlock condition: a closed chain of processes in which each one waits for a resource held by the next one in the chain.

Wait-for graph

A drawing with one node per process and an arrow from each blocked process to the process it is waiting for. With single-unit resources, a cycle in it means deadlock.

Room semaphore

In the dining philosophers solution, a counting semaphore that starts at 4 and limits how many philosophers may sit at the table at once, so a full circle of waiting can never form.

Pigeonhole principle

If more items than boxes are shared out, some box must get at least two. If all five forks were in the hands of four seated diners, one diner would hold two forks and could eat.

Resource ordering

A deadlock-prevention rule: number the resources and make every process request them in increasing order, so no chain of waits can loop back to its start.

Asymmetric solution

A dining philosophers fix in which at least one philosopher picks up its forks in the opposite order from the others, which breaks the circle of waiting.

Handoff

Passing a released resource straight to a process that is waiting for it, instead of putting it back where anyone could grab it first.

Deadlock freedom

The guarantee that the system never reaches a state where a group of processes waits forever for each other: some process can always make progress.

Starvation freedom

The stronger guarantee that every process that wants to proceed eventually does. A deadlock-free scheme can still starve one unlucky process.

Model checking

Letting a computer try every possible order of moves from a starting state to prove a property, such as "no deadlock is ever reachable".

Review questions with answers

1. Each philosopher runs `semWait(fork[i]); semWait(fork[(i + 1) mod 5]);` before eating, with every fork semaphore starting at 1. Which timing produces a deadlock?

- A. Two neighbours become hungry at the same moment and reach for their shared fork
- B. A philosopher finishes eating and puts its right fork down before its left one
- C. All five pick up their left fork before any of them reaches for the right one
- D. One philosopher thinks for a very long time while its neighbours keep eating

Answer: C. All five pick up their left fork before any of them reaches for the right one

If all five hold their left fork, every right fork is in a neighbour's hand. Each philosopher waits for the next one around the table: a circular wait, and none of them will ever let go.

2. In the dining philosophers problem, five forks lie on the table and every meal needs two forks. What is the largest number of philosophers that can be eating at the same moment?

Answer: 2 philosophers

Each eater uses two forks, so three eaters would need six. With five forks at most two can eat at once, and they must not be neighbours.

3. In the one-semaphore-per-fork solution, all five philosophers pick up their left fork and then each calls `semWait` on its right fork. What value does each fork semaphore end with?

Answer: -1

Fork k drops from 1 to 0 when philosopher k takes it as its left fork, then to -1 when the other neighbour, philosopher $k - 1$, blocks on it as its right fork. A value of -1 means exactly one process waits in that semaphore's queue.

4. In the dining philosophers solution with a room semaphore, room starts at 4. All five philosophers get hungry and call `semWait(room)` before anyone leaves the table. What is the value of room afterwards?

Answer: -1

Five `semWait` calls take the value from 4 down to -1 . Four philosophers sit down; the fifth sees a negative value and waits in the room queue.

5. If the room semaphore starts at 5 instead of 4, the dining philosophers solution is still free of deadlock.

Answer: False

A room of 5 admits everybody, so nothing stops all five from holding a left fork at once. It behaves exactly like the first attempt and can freeze.

6. In the monitor solution to the dining philosophers problem, a philosopher can be waiting in `ForkReady[right]` while it still holds its left fork.

Answer: True

`get_forks` takes the left fork and then waits on the right one if it is busy. What the monitor prevents is the circle: the right fork's holder took it earlier, so the waits can never loop all the way round.

7. Which of these make the five dining philosophers deadlock-free?

- A. A room semaphore that starts at 4
- B. Making philosopher 4 pick up its right fork first
- C. Making every philosopher pick up its right fork first
- D. A room semaphore that starts at 5
- E. A monitor whose get_forks checks the right fork in the same visit that takes the left

Answer: A, B, E

A room of 4 and a single right-hander both break circular wait, and so does the monitor. Everyone reaching right first is the mirror image of the first attempt, and a room of 5 seats everyone, so both can still freeze.

8. Match each deadlock condition with how it shows up at the frozen table.

Mutual exclusion	Each fork is in exactly one hand
Hold and wait	A philosopher keeps its left fork while waiting for the right
No preemption	Nobody may pull a fork out of a neighbour's hand
Circular wait	P0 waits for P1, P1 for P2, ... and P4 waits for P0

All four conditions hold at once at the frozen table. Remove any one of them and the deadlock cannot happen.

9. In the one-semaphore-per-fork dining philosophers solution, put one pass of philosopher i's loop in order.

Correct order:

1. think()
2. semWait(fork[i])
3. semWait(fork[(i + 1) mod 5])
4. eat()
5. semSignal(fork[(i + 1) mod 5])
6. semSignal(fork[i])

Think, take the left fork, take the right fork, eat, then put the right fork and the left fork back.

10. Which deadlock condition does each fix remove?

Hold and wait	Circular wait
Pick up both forks in one step, or neither Change the meal so one fork is enough	Let at most four philosophers sit down Have one philosopher pick up its right fork first Number the forks and always take the lower number first

A fix that stops anyone from holding one fork while waiting for another removes hold and wait. A fix that keeps the waits from closing into a loop (a seat limit, a different pick-up order) removes circular wait.

11. In the dining philosophers monitor solution, a philosopher calls `release_forks` while a neighbour is waiting in `ForkReady[left]`. What does `csignal(ForkReady[left])` do?

- A. Sets `fork[left]` to true and lets every hungry philosopher race to grab it
- B. Hands the fork straight to the waiting neighbour, which resumes and uses it
- C. Wakes every philosopher waiting in the monitor, in every condition queue
- D. Blocks the releasing philosopher until the neighbour hands the fork back

Answer: B. Hands the fork straight to the waiting neighbour, which resumes and uses it

`csignal` performs a handoff: the waiter resumes and the fork never lies on the table, so no other philosopher can take it first.

12. A scheme lets a hungry philosopher pick up its two forks only in a single step, and only when both are free. Which statement is true?

- A. It can deadlock, because each philosopher may hold one fork while waiting for its second
- B. It cannot starve anyone, because nobody ever holds one fork while waiting for another
- C. It needs a room semaphore that seats at most four philosophers to avoid deadlock
- D. It cannot deadlock, but a philosopher whose two neighbours' meals keep overlapping can starve

Answer: D. It cannot deadlock, but a philosopher whose two neighbours' meals keep overlapping can starve

Deadlock freedom says someone can always make progress; starvation freedom says everyone eventually does. This scheme gives the first but not the second unless a fairness rule is added.

6.7 UNIX Concurrency Mechanisms

You should be able to

- Name the five UNIX concurrency mechanisms and say which ones move data and which ones trigger actions.
- Trace a pipe as a circular buffer, predict when its writer or reader blocks, and tell unnamed pipes from named pipes (FIFOs).
- Send and receive typed messages on a message queue, and explain why shared memory is the fastest mechanism yet needs its own mutual exclusion.
- Apply the four semop rules to a semaphore set and explain why a list of operations succeeds or waits as one indivisible unit.
- Follow a signal from sender to pending bit to default action, handler or ignore, and explain why repeated signals of one type collapse into one.

Five UNIX tools for cooperating processes

Processes have separate address spaces, so they cooperate through the kernel. **Pipes, message queues and shared memory move data; semaphores and signals trigger actions.**

Tool	Carries	Who waits	Mutual exclusion
Pipe	a byte stream, first in, first out	writer if no room; reader if nothing to read	provided by the kernel
Message queue	whole messages, each with a type	sender if full; receiver if empty	provided by the kernel
Shared memory	nothing travels: same bytes in both	nobody	not provided
Semaphore set	counts only	a caller whose operations cannot be done yet	is the tool
Signal	just "event n happened" (one bit)	nobody waits for delivery	n/a

Pipes

A pipe is a fixed-size **circular buffer** kept by the kernel and used producer/consumer style: one process writes bytes in, another reads them out in the same order, and positions wrap around to slot 0. The kernel lets only one process at a time work on the pipe.

- **Write:** if there is room for the whole write it proceeds at once; otherwise the writer blocks until there is. (Real systems promise this only for writes up to PIPE_BUF bytes, 4,096 on Linux. A Linux pipe holds 64 KiB by default.)
- **Read (classic rule):** if the pipe holds at least as many bytes as requested, the read proceeds at once; otherwise the reader blocks. A real POSIX read() returns whatever bytes are there and blocks only on an empty pipe.
- **End of file:** when the pipe is empty and every write end is closed, read() returns 0.
- **SIGPIPE:** writing when no read end is open sends the writer SIGPIPE, which terminates it by default.

Worked example (16-byte pipe, `ls | wc -l`): `ls` writes "app.c↵" (6), "db.c↵" (5), "io.h↵" (5): 16/16, full. Writing "main.c↵" (7) blocks `ls`. `wc` reads 4: 12 used, 4 free, `ls` still blocked. `wc` reads 4 more: 8 used, 8 free, so `ls`'s 7 bytes go in: 15 used.

Unnamed and named pipes

`pipe(fd)` creates an unnamed pipe and returns two file descriptors: `fd[0]` reads, `fd[1]` writes. It has no name, so other processes normally reach it by **inheriting** the descriptors through `fork`: unnamed pipes join **related** processes. (Passing a descriptor over a UNIX-domain socket is a rare advanced exception.) For `ls | wc -l` the shell calls `pipe`, forks a child that points its standard output (fd 1) at the write end with `dup2`, closes its spare descriptors and execs `ls`; forks a second child that points standard input (fd 0) at the read end and execs `wc`; then closes both of its own ends and waits. Each child follows exec with `_exit(127)`, which runs only if exec failed. Every unused write end must be closed, or the reader never sees end of file.

A **named pipe (FIFO)**, made with `mkfifo`, has a name in the file system, so **unrelated** processes can open it by name. It buffers and blocks exactly like an unnamed pipe.

Message queues (System V)

A message is a block of bytes plus a **type** (a positive number chosen by the sender). Each queue works like a mailbox.

1. `msgsnd` adds a message at the back; a sender to a full queue (its limit is a total number of bytes) blocks.
2. `msgrcv` for any type takes the oldest message; a receiver of an empty queue blocks.
3. `msgrcv` for type k takes the oldest message of type k , skipping other types.
4. Classic rule: if no message of the requested type is present, the receiver is not suspended: the call fails and returns. (Real `msgrcv` does this only when given `IPC_NOWAIT`, which also makes it fail on an empty queue; `msgsnd` takes the same flag. Without it the caller waits. A negative type $-k$ takes the lowest type $\leq k$.)

Example: queue A(2), B(1), C(2), D(3), oldest first. Requests type 3, type 2, any, any return D, A, B, C.

Shared memory

The fastest form of IPC: one block of memory is mapped into several address spaces (possibly at different virtual addresses) and used with ordinary load and store instructions: no copying and no system calls after setup (System V: `shmget`, `shmat`; POSIX: `shm_open` and `mmap`). Each process attaches it read-only or read-write (as far as the permission bits given at creation allow); a store through a read-only mapping causes `SIGSEGV`. Sending 100 MiB through a pipe instead copies 200 MiB (into and out of the kernel) and needs at least 1,600 reads of 64 KiB.

The facility gives no mutual exclusion. If two processes each run `counter = counter + 1` (load, add, store) and interleave as load, load, add, store, add, store, a counter at 7 ends at 8 instead of 9. Wrapping the update in a semaphore (`semop -1` before, `+1` after) gives 9.

Semaphores (System V)

They generalize `semWait/semSignal`: `sem_op` may be larger than 1, and one call can operate on several semaphores. Semaphores are created in **sets**. `semctl` sets one value or all values at once, reads the fields or removes the set. `semop` takes a list of operations and the kernel performs **all of them atomically, or none**, suspending the caller if any one cannot proceed (with the `IPC_NOWAIT` flag the call fails at once instead). Each semaphore holds:

- **semval**: current value (never negative)
- **sempid**: process ID of the last process to operate on it
- **semncnt**: number of processes waiting for the value to rise above its current value
- **semzcnt**: number of processes waiting for the value to become zero

sem_op	Effect
> 0	Add it to the value; wake every process waiting for the value to increase.
= 0	If the value is 0, continue; otherwise suspend until it becomes 0.
< 0, sem_op ≤ value	Add it (the value drops); if it reaches 0, wake processes waiting for zero.
< 0, sem_op > value	Suspend until the value increases enough: the caller re-checks at every rise and goes on once sem_op ≤ value.

Example: scanner 1, printers 2, jobs 0. P1: [scanner −1, printers −1, jobs +1] → 0, 1, 1. P2 asks the same: scanner cannot drop, so nothing is done (printers stays 1) and P2 waits (scanner semncnt 1). P3: [jobs 0] waits (semzcnt 1). P1: [scanner +1, printers +1, jobs −1] → 1, 2, 0; P3's wait for zero completes, then P2's whole list → 0, 1, 1. Asking for everything in one call means a process never holds part of what it needs while waiting, which avoids hold-and-wait.

Signals

A signal informs a process that an asynchronous event occurred. It is like a software interrupt but without priorities: all signals are treated equally, and signals that arrive together are handled one at a time in no particular order. The kernel or another process (for example with kill) can send one. Sending sets **one bit per signal type** in the target's process table entry, so several signals of one type cannot be queued: they collapse into one. (Real systems keep this for the classic signals; the real-time signals POSIX added later do queue, and Linux, for example, does pick an order: SIGKILL first, then faults such as SIGSEGV, then the lowest number.) The signal is acted on when the process next wakes to run or is about to return from a system call; sending one also wakes a process sleeping in an interruptible wait, such as for keyboard input. The process takes the **default action** (usually terminate, sometimes with a core dump; SIGCHLD's default is to ignore it), runs a **signal handler** it registered, or **ignores** the signal. SIGKILL cannot be caught or ignored.

Signal	Sys V / Linux x86	Meaning
SIGHUP, SIGINT, SIGQUIT	1, 2, 3 (same)	terminal hung up; Ctrl-C; quit key
SIGILL, SIGTRAP, SIGFPE	4, 5, 8 (same)	illegal instruction; trace trap; arithmetic error
SIGKILL	9 (same)	kill, cannot be caught or ignored
SIGBUS, SIGSEGV, SIGSYS	10/7, 11/11, 12/31	bus error; segmentation violation; bad system call argument
SIGPIPE, SIGALRM, SIGTERM	13, 14, 15 (same)	write to a pipe nobody reads; alarm clock; polite termination request
SIGUSR1, SIGUSR2	16/10, 17/12	user defined
SIGCHLD, SIGPWR	18/17, 19/30	child stopped or ended; power failure

Because many numbers differ between UNIX variants, programs always use the names.

Key terms

Interprocess communication (IPC)

Any way for separate processes to exchange data or coordinate with each other, such as pipes, message queues, shared memory, semaphores and signals.

Pipe

A one-way channel kept by the kernel: a fixed-size circular buffer that one process writes bytes into and another reads them out of, first in, first out.

Named pipe (FIFO)

A pipe that has a name in the file system, so any process with permission can open it by that name, even one unrelated to the process that created it.

File descriptor

A small whole number a process uses to refer to something it has open, such as a file or one end of a pipe. By convention 0 is standard input and 1 is standard output.

Circular buffer

A fixed-size array used as a queue whose positions wrap around: after the last slot, writing and reading continue at slot 0.

End of file (EOF)

What a read reports when no more data exists and none can ever arrive. For a pipe: it is empty and every write end has been closed.

Message queue

A list of messages held by the kernel that works like a mailbox: senders add messages with `msgsnd` and receivers remove them with `msgrcv`.

Message type

A positive whole number the sender attaches to each message. A receiver can ask for the oldest message of one type instead of simply the oldest message.

`msgsnd`

The System V call that adds a message (a type plus a block of bytes) to a message queue. The sender blocks while the queue is full.

`msgrcv`

The System V call that removes a message from a message queue: either the oldest message of any type, or the oldest message of a chosen type.

Shared memory

A block of memory mapped into the address spaces of several processes at once, so all of them read and write the same bytes with ordinary instructions.

Semaphore set

A group of System V semaphores created together under one identifier. One `semop` call can operate on several members of the set as a single indivisible unit.

`semop`

The System V call that performs a list of semaphore operations. The kernel applies the whole list at once or, if any part cannot proceed, applies none of it and suspends the caller.

semctl

The System V call for managing a semaphore set: setting values (one at a time or all at once), reading its bookkeeping fields, or removing the set.

Signal

A software notice telling a process that some event happened, such as Ctrl-C being pressed or a child process ending. It arrives asynchronously, much like a hardware interrupt.

Pending signal

A signal that has been sent but not yet acted on. The kernel records it as one bit per signal type in the target process's process table entry.

Signal handler

A function a process registers to run when a particular signal arrives, in place of the default action.

Default action

What happens when a process has neither caught nor ignored a signal. For most signals it terminates the process; for a few, such as SIGCHLD, it does nothing.

SIGKILL

Signal number 9, which always terminates the target process. It cannot be caught by a handler and cannot be ignored.

Core dump

A file holding a snapshot of a process's memory, written when certain signals end the process, so a programmer can later examine what went wrong.

Review questions with answers

1. A UNIX pipe can hold 16 bytes and currently holds 11. A process asks to write 7 bytes into it. What happens?

- A. The writer blocks until there is room for all 7 bytes
- B. The 5 bytes that fit are written and the other 2 are thrown away
- C. The 2 oldest bytes are overwritten to make room
- D. The write fails at once with an error and the writer carries on

Answer: A. The writer blocks until there is room for all 7 bytes

Only $16 - 11 = 5$ bytes are free. A write that does not fit blocks the writer until the reader has removed enough bytes; then the write goes ahead. (Real systems guarantee this all-or-nothing behaviour for writes up to PIPE_BUF bytes, and 7 bytes is far below that.)

2. A program started in one terminal window can open, by name, an unnamed pipe that a program in another terminal window created, just as it would open a file.

Answer: False

An unnamed pipe has no name in the file system, so there is nothing to open. Other processes normally reach it by inheriting its file descriptors through fork, which is why it joins related processes such as a parent and its children. Unrelated programs use a named pipe (FIFO), which can be opened by name.

3. In the classic description of UNIX message queues, a process that asks `msgrcv` for a message of a type that is not in the queue is suspended until such a message arrives.

Answer: False

Under the classic rules only an empty queue suspends a receiver that asks for any message. A request for a missing type is not suspended: the call fails and returns, so the process can do other work. (Real `msgrcv` behaves this way when the `IPC_NOWAIT` flag is given; without it the caller waits.)

4. A message queue holds four messages, oldest first: A (type 2), B (type 1), C (type 2), D (type 3). The receiver calls `msgrcv` for type 3, then for type 2, then for any type, then for any type. Put the messages in the order the receiver gets them.

Correct order:

1. D (type 3)
2. A (type 2)
3. B (type 1)
4. C (type 2)

Type 3 picks D, skipping the three older messages. Type 2 picks the oldest type-2 message, A. Then “any” takes the oldest message left, B, and finally C.

5. Which statements about UNIX shared memory are true?

- A. It is the fastest way for processes to exchange data
- B. Each process can be given read-only or read-write access
- C. The kernel automatically makes accesses to it mutually exclusive
- D. Every read or write of the shared block is a system call
- E. Two processes may see the same block at different virtual addresses

Answer: A, B, E

Once attached, the block is used with ordinary instructions (no copying and no system calls), which makes it the fastest tool, and each process sees it at its own address with its own permission. Mutual exclusion is not provided; the processes add it themselves, for example with semaphores.

6. A semaphore set has two members: A with value 2 and B with value 0. A process calls `semop` with the list `[A -1, B -1]`. Right after the call, what is the value of A?

Answer: 2

B cannot drop below 0, so the whole list must wait and none of it is done: A stays at 2 and the caller is suspended until B rises.

7. A process calls `semop` with a single operation and no special flags. Sort each case by what happens to the caller.

Continues at once	Is suspended
value 3, <code>sem_op -2</code> value 0, <code>sem_op 0</code> value 0, <code>sem_op +3</code>	value 1, <code>sem_op -2</code> value 2, <code>sem_op 0</code> value 0, <code>sem_op -1</code>

A negative `sem_op` goes ahead only if its size is at most the value. `sem_op = 0` goes ahead only when the value is already 0. A positive `sem_op` always goes ahead.

8. Before a process next runs, SIGUSR1 is sent to it three times. The process has registered a handler for SIGUSR1. How many times does the handler run when the process next runs?

Answer: 1

Delivery sets one pending bit per signal type. The second and third SIGUSR1 find the bit already set and collapse into the first, so the handler runs once.

9. Match each signal to the event that usually causes it.

SIGINT	Ctrl-C typed at the terminal
SIGCHLD	A child process stopped or ended
SIGPIPE	A write to a pipe that nobody reads
SIGSEGV	An access to a memory address the process may not use
SIGALRM	A timer the process set has expired
SIGFPE	An arithmetic error such as dividing by zero

Most signals report one specific event: the terminal (SIGINT), the process family (SIGCHLD), pipes (SIGPIPE), memory faults (SIGSEGV), timers (SIGALRM) and arithmetic errors (SIGFPE).

10. A process asks to ignore SIGTERM and also asks to ignore SIGKILL. Later it is sent both signals. What happens?

- A. Both are ignored and the process keeps running
- B. Both terminate it, because no signal may be ignored
- C. SIGTERM is ignored, but SIGKILL still terminates the process
- D. SIGKILL waits in a queue until the ignore setting is removed

Answer: C. SIGTERM is ignored, but SIGKILL still terminates the process

Any signal except SIGKILL (and SIGSTOP) can be caught or ignored. SIGKILL always takes its default action, so a process can always be stopped.

11. Choose the UNIX mechanism that fits each job best.

Pipe	Message queue	Shared memory	Semaphore	Signal
Stream the output of sort straight into a pager	Let a print server pick urgent jobs before routine ones	Let two image filters work on one 200 MB frame without copying it	Allow at most 3 processes to use a software licence at once Stop two processes from updating a shared counter at the same moment	Nudge a running server to re-read its settings file (no data needs to travel)

Pipes stream bytes from one program to the next; message types let a receiver choose; shared memory avoids copying; semaphores count and exclude; a signal announces that an event happened.

12. Why do UNIX programs refer to signals by name (such as SIGUSR1) rather than by number?
- A. Signal numbers are reassigned every time the system boots, so a stored number goes stale
 - B. The kernel rejects a bare number, so kill(pid, 10) fails where kill(pid, SIGUSR1) works
 - C. A name is resolved faster than a number, so the signal reaches its target process sooner
 - D. Many numbers differ between UNIX variants: SIGUSR1 is 16 on System V but 10 on Linux x86

Answer: D. Many numbers differ between UNIX variants: SIGUSR1 is 16 on System V but 10 on Linux x86

A few numbers (1 SIGHUP, 2 SIGINT, 3 SIGQUIT, 9 SIGKILL, 15 SIGTERM) are the same almost everywhere, but many others differ. Using the names keeps a program correct on every system.

6.8 Linux Kernel Concurrency Mechanisms

You should be able to

- Contrast real-time signals with standard signals: queuing, delivery order and the value they carry.
- Use atomic integer and bitmap operations, and explain why atomic_t is a separate type.
- Decide between spinning and sleeping, and pick the right spinlock variant when interrupt handlers share the data.
- Use kernel semaphores, mutexes and reader-writer locks, including down_interruptible and down_trylock.
- Explain how memory barriers and read-copy-update (RCU) keep data consistent without stopping readers.

6.8 Linux Kernel Concurrency Mechanisms

Linux gives programs every UNIX tool of 6.7 plus **real-time signals**. Inside the kernel, code runs on several CPUs at once and can be interrupted between any two instructions, so the kernel has its own toolbox: **atomic operations, spinlocks, semaphores and mutexes, memory barriers and RCU**.

Real-time signals vs standard signals

	Standard signal (SIGINT, SIGUSR1, ...)	Real-time signal (SIGRTMIN to SIGRTMAX)
Repeated copies	One pending bit per signal: copies sent while it is pending are merged	Every copy is queued and delivered
Order	Unspecified by POSIX (Linux: standard before real-time)	Lowest number first; copies of one signal in the order sent
Data	None with <code>kill</code> ; <code>sigqueue</code> can attach one, but merged copies keep only the first	Its own value per copy (int or pointer): <code>sigqueue(pid, sig, value)</code> , read by an <code>SA_SIGINFO</code> handler

About 30 real-time signals exist; numbers vary by C library, so programs write SIGRTMIN+n. SIGUSR1 sent four times while blocked runs its handler **once**; four real-time signals run it four times.

Atomic operations

An atomic operation runs with no interruption and no interference: no other CPU sees it half done. Two kinds: **integer operations** on the type `atomic_t`, and **bitmap operations** on one bit of any ordinary word. `atomic_t` is a separate type so that (1) only `atomic_` functions accept it (`count++` on it will not compile) and (2) the compiler cannot keep it in a register or optimize an access away. Typical use: counters and **reference counts**.

`count++` is three steps (load, add, store). If `count = 5` and two CPUs both load before either stores, both store 6: a lost update (it should be 7). `atomic_inc` is one indivisible step (on x86, an instruction with a `lock` prefix), so any order gives 7.

Integer operation	Effect
<code>atomic_t v = ATOMIC_INIT(i);</code>	declare v with starting value i
<code>atomic_read(&v)</code> , <code>atomic_set(&v, i)</code>	return v; set v to i
<code>atomic_add(i, &v)</code> , <code>atomic_sub(i, &v)</code> , <code>atomic_inc(&v)</code> , <code>atomic_dec(&v)</code>	add or subtract i (amount first) or 1
<code>atomic_sub_and_test(i, &v)</code>	subtract i; return true if the result is 0
<code>atomic_add_negative(i, &v)</code>	add i; return true if the result is negative
<code>atomic_dec_and_test(&v)</code> / <code>atomic_inc_and_test(&v)</code>	subtract / add 1; return true if the result is 0

Example: `v = 10`; add 5 → 15; sub 3 → 12; dec → 11. With `v = 5`, `atomic_add_negative(-7, &v)` leaves -2 and returns true. For a reference count, the one caller whose `atomic_dec_and_test` returns true frees the object.

Bitmap operations take a bit number `nr` and the address of a word: `set_bit` , `clear_bit` , `change_bit` (flip), `test_and_set_bit` , `test_and_clear_bit` , `test_and_change_bit` (each returns the bit's old value) and `test_bit` (read only). Example: `flags = 0x25 (00100101)`; `test_and_set_bit(3, &flags)` returns 0 and leaves 0x2d; a second call returns 1. That makes a tiny try-lock.

Spinlocks: spin or sleep?

The most common kernel lock. One CPU holds it; others **spin** (busy-wait) until it is free. API:

`spin_lock_init` , `spin_lock` , `spin_unlock` , `spin_trylock` (returns 0 at once, never spinning, if the lock is held) and `spin_is_locked` . A spinning waiter wastes the whole hold time `H`; a sleeping waiter pays two context switches, `2C`. Spin only when **H is shorter than two context switches**. With `C = 3 μs` (break-even `6 μs`): `H = 4 μs` → spin (4 wasted, not 6); `H = 14 μs` → sleep (6 wasted, not 14). Never sleep holding a spinlock. On a uniprocessor kernel `spin_lock` only switches off kernel preemption (nothing if it is not preemptible).

Spinlocks and interrupts

Process context (a system call) may sleep; **interrupt context** (a handler) must not, so handlers use spinlocks. If a system call holds a lock with plain `spin_lock` and an interrupt on the **same CPU** runs a

handler that wants that lock, the handler spins forever: the holder cannot run until the handler returns. The fix is to switch local interrupts off while holding the lock; the interrupt then waits as pending.

Variant	What it adds · when to use it
<code>spin_lock</code>	Nothing. For locks no handler takes, or when interrupts are already off.
<code>spin_lock_irq</code>	Local interrupts off; <code>spin_unlock_irq</code> always turns them on. Only when you know they were on.
<code>spin_lock_irqsave(&l, flags)</code>	Saves the interrupt state in flags, then off; <code>spin_unlock_irqrestore</code> restores it. When you do not know.
<code>spin_lock_bh</code>	Holds off bottom halves (softirqs, work deferred by handlers) on this CPU.

Reader-writer spinlocks (`rwlock_t`: `read_lock`, `read_unlock`, `write_lock`, `write_unlock`) admit many readers or one writer. In the classic design readers are favoured: a new reader enters even while a writer waits, so overlapping readers can starve the writer. (Since Linux 3.16, x86 and later most other processors use a queued `rwlock_t` in which only readers in interrupt context can still cut ahead of a waiting writer.)

Scope. Ordinary kernels. Under PREEMPT_RT (real-time), `spinlock_t` and `rwlock_t` become sleeping rt-mutexes: taking one does not disable preemption, `_irq/_irqsave` do not change the interrupt state, and only `raw_spinlock_t` still spins.

Kernel semaphores and mutexes

Kernel-only, invisible to programs, and cheaper than user-visible semaphores. A waiter sleeps. Three kinds: binary, counting and reader-writer. Counting API: `sema_init(&sem, count)`; `down` (`semWait`; sleeps and ignores signals); `down_interruptible` (sleeps, but a signal wakes it and it returns `-EINTR` without the semaphore; the usual choice); `down_trylock` (never sleeps; returns nonzero if it could not take it); `up` (`semSignal`; hands the unit to the longest sleeper, or adds it back to count). Linux keeps `count ≥ 0` with a separate wait queue; the classic model's negative value is `count` minus the number of sleepers. For plain mutual exclusion Linux prefers the **mutex** (`mutex_lock` / `mutex_unlock`): one holder, only it may unlock, never used in interrupt context. **Reader-writer semaphores:** `init_rwsem`, `down_read` / `up_read`, `down_write` / `up_write`; many readers or one writer, sleeping uninterruptibly.

Memory barriers

Compilers and processors may reorder loads and stores for speed. One thread never notices; another CPU can. `rmb()` keeps loads from crossing it, `wmb()` stores, `mb()` both; `barrier()` stops only the compiler. `smp_rmb()`, `smp_wmb()` and `smp_mb()` are real barriers on a multiprocessor kernel and only compiler barriers on a uniprocessor kernel. Pattern: CPU 0 runs `data = 42; wmb(); ready = 1;` and CPU 1 runs `if (ready) { rmb(); use(data); }`. Counting the possible orders of the four accesses: no barriers, 6 of 24 let CPU 1 see `ready = 1` with `data = 0`; `wmb` alone or `rmb` alone, 1 of 12; both, 0 of 6. (x86 keeps stores in order with stores and loads with loads, so there `smp_wmb` and `smp_rmb` only restrain the compiler; ARM and POWER need real fences.)

Read-copy-update (RCU)

For data read constantly and changed rarely (a routing table). Readers call `rcu_read_lock()`, fetch the pointer with `rcu_dereference()`, and finish with `rcu_read_unlock()`; this marks a read-side critical section and takes no lock. A writer: (1) copies the current version, (2) changes the copy, (3) publishes it with `rcu_assign_pointer`, one pointer store, (4) waits a **grace period** until every reader that might hold the old version has finished, either blocking in `synchronize_rcu()` or registering a callback with `call_rcu()`, and (5) frees the old version. Readers see the whole old or the whole new version, never a mix. Freeing before the grace period ends is a use-after-free. Writers still coordinate among themselves, usually with a spinlock.

Choosing a tool

One counter or bit: atomic operation. Short critical section: spinlock (`_irqsave` if a handler shares it). Long wait or a holder that may sleep: mutex or semaphore. Read-mostly data: RCU. Data handed over through a flag: `wmb/rmb`.

Key terms

Real-time signal (RT signal)

A Linux signal numbered from `SIGRTMIN` to `SIGRTMAX`. Each copy that is successfully sent is queued separately (the queue is finite, so `sigqueue` can fail with `EAGAIN` when it is full), pending ones are delivered lowest number first, and each queued copy keeps its own small value (sent with `sigqueue`).

Standard signal

One of the classic UNIX signals such as `SIGINT`, `SIGTERM` or `SIGUSR1`. It does not queue: copies sent while one is pending are merged into it, and only the first copy's information is kept. Sent with `kill` it carries no value; on Linux, `sigqueue` can attach one.

atomic_t

A Linux kernel integer type that can be used only through the `atomic_` functions, so the compiler cannot apply ordinary arithmetic to it by mistake or optimize its accesses away.

Atomic integer operation

A kernel function such as `atomic_inc` or `atomic_dec_and_test` that reads, changes and writes an `atomic_t` as one indivisible step.

Atomic bitmap operation

A kernel function such as `set_bit` or `test_and_set_bit` that changes or tests one bit of an ordinary memory word as one indivisible step.

Reference count

A counter of how many users currently hold an object. Each user adds one when it starts and subtracts one when it is done; the user that brings it to 0 frees the object.

Process context

Kernel code running on behalf of a process, for example inside a system call. It is allowed to sleep.

Interrupt context

Kernel code running because a device interrupted, such as an interrupt handler. It runs on no process's behalf and must never sleep.

Bottom half (softirq)

Work that an interrupt handler postpones so it can run a little later with interrupts enabled. Also called deferred work or software interrupts.

Reader-writer spinlock (rwlock_t)

A spinlock that lets many readers hold it together, or one writer alone (on ordinary kernels; under PREEMPT_RT it becomes a sleeping lock). The classic design favours readers, so a steady stream of readers can starve a writer.

Kernel semaphore

A semaphore used only by kernel code and invisible to user programs. down is its semWait and up is its semSignal; a waiter sleeps instead of spinning.

down_interruptible

The usual way kernel code waits on a semaphore: it sleeps, but a signal wakes it early, and it then returns -EINTR without taking the semaphore.

Linux mutex

A sleeping lock that one task holds at a time and only the holder may release. Linux now prefers it to a binary semaphore for plain mutual exclusion.

Reader-writer semaphore (rw_semaphore)

A sleeping lock that admits many readers at once or a single writer, used with down_read/up_read and down_write/up_write.

Memory reordering

The compiler or the processor carrying out loads and stores in a different order from the program text, to run faster. A single thread cannot tell, but another processor can.

Memory barrier (mb, rmb, wmb)

An instruction that forbids reordering across it: rmb() orders loads, wmb() orders stores, and mb() orders both.

Compiler barrier (barrier())

A marker that stops the compiler from moving memory accesses across it. It emits no instruction, so the processor itself may still reorder.

Read-copy-update (RCU)

A kernel technique for data that is read far more than written: readers take no lock, a writer updates a copy and publishes it by switching one pointer, and the old copy is freed only after a grace period.

Grace period

In RCU, a wait long enough that every reader that might still be using the old version has finished. After it, the old version can be freed safely.

Read-side critical section

In RCU, the code between rcu_read_lock() and rcu_read_unlock(). A reader may use RCU-protected data only inside it.

Review questions with answers

1. A process has SIGUSR1 blocked. Another process sends it SIGUSR1 four times. The receiver then unblocks the signal. How many times does its SIGUSR1 handler run?

Answer: 1

SIGUSR1 is a standard signal: while it is pending, the kernel keeps just one pending bit for it, so the extra copies are merged. One handler call. Four real-time signals would have produced four calls.

2. Which statements about Linux real-time signals are true? Select all that apply.

- A. Every copy sent is queued, so none is merged away
- B. Only the kernel can send them; another process cannot
- C. When several are pending, the lowest-numbered one is delivered first
- D. Copies of the same real-time signal arrive in random order
- E. sigqueue can attach an integer or a pointer value to one

Answer: A, C, E

Real-time signals queue every successfully sent copy (the queue is finite, so a send can fail when it is full), are delivered lowest number first (copies of one signal in the order sent), and can carry a value given to sigqueue. Ordinary processes send them, for example with sigqueue.

3. Why does Linux give atomic integers their own type, `atomic_t`, instead of using a plain `int` ?

- A. Because `atomic_t` is wider than an `int`, so even a very busy counter can never overflow
- B. Because declaring an `atomic_t` switches interrupts off for the rest of the function
- C. So only `atomic_` functions can use it, and the compiler cannot optimize its accesses away
- D. So user programs can share the counter directly with the kernel, without a system call

Answer: C. So only `atomic_` functions can use it, and the compiler cannot optimize its accesses away

A separate type means ordinary arithmetic such as `count++` will not even compile on it, so every change goes through an atomic function, and the compiler must really perform each access.

4. Kernel code declares `atomic_t v = ATOMIC_INIT(10);` and then runs `atomic_add(5, &v);` `atomic_sub(3, &v);` `atomic_dec(&v);`. What does `atomic_read(&v)` return now?

Answer: 11

$10 + 5 = 15$, then $15 - 3 = 12$, then $12 - 1 = 11$. Note the argument order: the amount comes first, the pointer second.

5. A shared counter holds 7. Two CPUs each run `count++` once, as a load, an add and a store, and both CPUs load before either one stores. What does the counter hold when both have finished?

Answer: 8

Both load 7, both compute 8, both store 8. Two increments ran but the counter rose by only one: a lost update. `atomic_inc` would give 9 in any order.

6. On a multiprocessor, one context switch costs 4 μ s. A sleeping waiter pays for two switches; a spinning waiter wastes the whole time the lock is held. Up to what hold time does spinning waste no more CPU time than sleeping?

Answer: 8 μ s

Sleeping costs $2 \times 4 = 8 \mu$ s of switching. Spinning costs the hold time. So spinning is the better deal for hold times up to 8 μ s, which is why spinlocks suit only very short critical sections.

7. On an ordinary (non-`PREEMPT_RT`) kernel, a system call holds a lock taken with plain `spin_lock()`. A network interrupt arrives on the same CPU, and its handler calls `spin_lock()` on the same lock. What happens?

- A. The handler spins for a few microseconds until the system call finishes and releases the lock
- B. That CPU deadlocks: the handler spins forever, because the holder cannot run until the handler returns
- C. The kernel notices the conflict and hands the lock to the handler, then back to the system call
- D. The CPU holds the interrupt pending until the system call releases the lock, then runs it

Answer: B. That CPU deadlocks: the handler spins forever, because the holder cannot run until the handler returns

The fix is to take the lock with `spin_lock_irqsave` (or `spin_lock_irq`) in the system call, so the interrupt is held pending until the lock is released.

8. Kernel code must take a lock that an interrupt handler also uses, and it cannot know whether interrupts are currently enabled. Which call should it use?

- A. `spin_lock_irq` (interrupts off)
- B. `spin_lock_bh` (bottom halves off)
- C. `spin_trylock` (never spins)
- D. `spin_lock_irqsave` (interrupts off)

Answer: D. `spin_lock_irqsave` (interrupts off)

`spin_lock_irqsave` saves the current interrupt state in flags, switches interrupts off, and `spin_unlock_irqrestore` puts back exactly the saved state.

9. Match each kernel semaphore call with what it does.

<code>down</code>	Sleeps until it gets a unit; signals cannot wake it
<code>down_interruptible</code>	Sleeps, but a signal wakes it and it returns <code>-EINTR</code>
<code>down_trylock</code>	Never sleeps; returns nonzero if no unit is free
<code>up</code>	Gives a unit back or wakes a sleeper (<code>semSignal</code>)
<code>sema_init</code>	Sets the starting count

`down` and `down_interruptible` are both `semWait`; they differ only in whether a signal can end the sleep. `down_trylock` is the no-wait try. `up` is `semSignal`, and `sema_init` sets the initial count.

10. On an ordinary (non-`PREEMPT_RT`) kernel, when the lock is not available, does the waiting task spin or sleep?

Spins (busy-waits)	Sleeps
spin_lock on a held spinlock write_lock on an <code>rwlock_t</code> that readers hold spin_lock_irqsave on a held spinlock	down on a semaphore whose count is 0 mutex_lock on a held mutex down_write on a reader-writer semaphore that a reader holds

Spinlocks, including reader-writer spinlocks and the interrupt-safe variants, make the waiter loop on its CPU. Semaphores, mutexes and reader-writer semaphores put the waiter to sleep so the CPU can run something else (a modern mutex may first spin for a moment while the holder is still running, but a long wait always ends in sleep). On a `PREEMPT_RT` kernel `spinlock_t` and `rwlock_t` sleep too.

11. Put the steps of an RCU update in order.

Correct order:

1. Allocate memory and copy the current version into it
2. Change the copy
3. Publish the copy with `rcu_assign_pointer`
4. Wait for a grace period, for example with `synchronize_rcu`
5. Free the old version

Copy, change, publish with one pointer store, wait until every reader that might hold the old version has finished, then free it. Freeing before the grace period could pull memory out from under a reader.

12. `barrier()` stops the processor itself from reordering memory accesses.

Answer: False

`barrier()` only stops the compiler from moving accesses across it; it emits no instruction. Stopping the processor needs `rmb()`, `wmb()` or `mb()` (or the `smp_` versions on a multiprocessor kernel).

6.9 Solaris Thread Synchronization Primitives

You should be able to

- Name the four Solaris thread synchronization primitives and describe the shared synchronization object each one creates.
- Trace `mutex_enter`, `mutex_exit` and `mutex_tryenter`, and explain spinning, sleeping on a turnstile and the adaptive mutex.
- Predict the effect of `sema_p`, `sema_v` and `sema_tryv`, and of the readers/writer operations, including `rw_downgrade` and `rw_tryupgrade`.
- Explain how `cv_wait` releases and reacquires its mutex, and why the waiting thread must re-test its condition in a while loop.
- Choose the right primitive for a given sharing problem.

Solaris thread synchronization primitives

Solaris keeps the classic UNIX mechanisms (pipes, messages, shared memory, semaphores, signals) and adds four **thread synchronization primitives**: mutual exclusion (mutex) locks, semaphores, readers/writer

locks and condition variables. Each exists in the kernel (for kernel threads) and in the threads library (for user threads). Initializing one (`mutex_init` , `sema_init` , `rw_init` , `cv_init`) creates a **synchronization object**: a small data structure, shared by every thread that uses it, that each later call reads and changes. (These are the kernel's names; the library uses its own, such as `mutex_lock` .)

Mutex locks

- One thread at a time. The thread that acquired the lock is its **owner**, and only the owner may release it. The Solaris kernel **panics** (halts with an error report) on a release by any other thread, or on a recursive `mutex_enter` by the owner, which would wait for itself forever.
- `mutex_enter()` acquires, waiting if the lock is held. `mutex_exit()` releases, so a waiter can take the lock. `mutex_tryenter()` acquires only if the lock is free, otherwise returns 0 at once: a **try operation**, which never waits.

Spin, sleep or adapt

Basic design: a waiter **spins** by default (busy-waits on its processor, testing the lock). Optionally, an interrupt-based blocking mechanism lets it **sleep** on a **turnstile**, the lock's priority-ordered kernel queue of blocked threads, until the lock is released.

Spinning burns processor time for the whole wait but takes the lock the instant it is free. Sleeping costs two thread switches (to sleep and back) but frees the processor. Short waits favor spinning; long ones favor sleeping.

Worked example (illustrative: one switch = 3 μ s). Lock free in 4 μ s: spinning uses 4 μ s and gets it at 4 μ s; sleeping uses $2 \times 3 = 6$ μ s and gets it at 7 μ s. Free in 30 μ s: spinning 30 μ s, sleeping 6 μ s. Owner preempted (about 2,000 μ s): spinning burns about 2,004 μ s, sleeping still 6 μ s.

Adaptive mutexes. Solaris kernel mutexes choose each time: owner running on another processor $\rightarrow$ spin (it will probably release soon); owner not running (preempted or asleep) $\rightarrow$ sleep on the turnstile. Not knowing the hold time, it bets on short holds, so it may spin when sleeping would have been cheaper. Code at a high interrupt level may not sleep, so it uses pure spin mutexes.

Semaphores

- `sema_p()`: count above 0 $\rightarrow$ subtract 1 and go on; count 0 $\rightarrow$ sleep until a unit is available.
- `sema_v()`: add 1 and wake one sleeper, if any, which takes that unit when it runs.
- `sema_tryv()`: subtract 1 only if no sleep is needed; otherwise return 0 at once.
- The count never drops below 0; sleepers wait in a separate list (Chapter 5's `semWait` let the value go negative instead). There is **no owner**: any thread may call `sema_v`, so a stray release cannot be caught.

Worked trace. Start at 2. Three `sema_p` calls: $2 \rightarrow 1 \rightarrow 0$, and the third caller sleeps (count stays 0). `sema_tryv` fails. `sema_v` makes it 1 and wakes the sleeper; once that thread takes the unit, the count is 0 again.

Readers/writer locks

- Many **readers** together, or one **writer** alone, never both.
- `rw_enter(lock, RW_READER or RW_WRITER)` acquires in that mode, waiting if needed; `rw_exit()` releases either mode; `rw_tryenter()` is the try version.

- **Writers first.** A new reader waits if a writer holds the lock *or is waiting for it*, so readers cannot keep a writer out forever; a writer waits until nobody holds the lock. Classic Solaris rule, used in this guide: on release a waiting writer goes first; if none waits, all waiting readers enter together. Newer kernels (OpenSolaris, illumos) also weigh thread priority when handing the lock on.
- **rw_downgrade():** the writer turns its lock into a read lock without letting go, so no other writer can change the data in between. Classic rule: waiting writers keep waiting; if none waits, the waiting readers are admitted at once.
- **rw_tryupgrade():** a reader tries to become the writer. It succeeds only if it is the only reader and no writer is waiting; otherwise it returns 0 at once and the caller keeps its read lock. There is no waiting version: two readers each waiting for the other to leave would deadlock. After a failure: `rw_exit`, `rw_enter(RW_WRITER)`, then **re-check the data**, since another writer may have changed it.

Examples. (1) T1 writes, T2 and T3 wait to read, T1 downgrades: all three read. (2) T1 writes, writer T2 and reader T3 wait, T1 downgrades: T1 reads alone; after T1 exits T2 writes, then T3 reads. (3) Readers T1 and T2 both try to upgrade: both fail; once T2 exits, T1's next try succeeds.

Condition variables

A condition variable lets a thread sleep until a condition on shared data holds. It is always used with the mutex that protects that data; together they behave like a monitor.

- **cv_wait(cv, m):** in one atomic step, release mutex m and go to sleep on cv. When woken, reacquire m before returning, so the caller holds m again exactly as before the call.
- **cv_signal(cv)** wakes one sleeping thread; **cv_broadcast(cv)** wakes all of them. A condition variable keeps no count, so a signal sent when nobody sleeps is simply lost.

```
mutex_enter(&m);           /* lock the shared data          */
while (!condition)         /* test; re-test after every wake */
    cv_wait(&cv, &m);      /* release m and sleep; relock m  */
/* use the shared data: m is held and the condition is true */
mutex_exit(&m);            /* unlock                          */
```

Why atomic? If `cv_wait` released m and then slept as two steps, another thread could slip into the gap, change the data and signal while nobody slept. The signal would be lost and the waiter could sleep forever although its condition is true.

Why while, not if? A wake-up only means the condition *may* hold (the Mesa-style monitor behavior of Chapter 5): before the woken thread gets m back, another thread can take m and change the data. Example: C1 sleeps on an empty queue; P adds an item, signals, unlocks; C2 locks m first and takes the item; then C1 gets m back. With `if`, C1 takes from an empty queue (count -1). With `while`, C1 re-tests, finds it empty and sleeps again. The loop also covers **spurious wakeups** and `cv_broadcast`, which wakes every sleeper even if only one can proceed.

Choosing a primitive

Situation	Best fit
A short critical section; a stray release must be caught	Mutex lock (ownership)
Counting identical resources	Semaphore set to the number of units
A signal that may arrive before anyone waits	Semaphore starting at 0 (it remembers the signal)
Data read very often and written rarely	Readers/writer lock
Waiting for any condition on shared data	Condition variable + mutex (cv_broadcast wakes all)

Rule of thumb: start with a mutex lock; use a readers/writer lock only when reads clearly dominate, since its bookkeeping costs more.

Key terms

Synchronization object

The small data structure in memory that is created when a Solaris lock, semaphore or condition variable is initialized. Every thread that uses the primitive works on this one shared object.

Mutex lock

The Solaris lock that lets only one thread at a time hold it. The thread that acquired it is its owner, and only the owner may release it.

mutex_enter

Acquire a mutex lock. If another thread holds it, the caller waits (by spinning or sleeping) until it is released.

mutex_exit

Release a mutex lock you own. If threads are waiting for it, one of them can now acquire it.

Try operation

The non-blocking version of an acquire call, such as `mutex_tryenter`, `sema_tryp`, `rw_tryenter` or `rw_tryupgrade`. It succeeds only if it can do so at once; otherwise it reports failure immediately and the caller keeps running.

Turnstile

A Solaris kernel queue on which threads that are blocked on a mutex lock or a readers/writer lock sleep, ordered by priority, until the lock is released.

Adaptive mutex

A Solaris kernel mutex that chooses how to wait each time: it spins while the owner is running on another processor, and sleeps on a turnstile when the owner is not running.

sema_p

Solaris semaphore wait. If the count is above 0, subtract 1 and continue; if it is 0, sleep until a `sema_v` makes a unit available.

sema_v

Solaris semaphore signal. Add 1 to the count; if threads are sleeping on the semaphore, wake one, which takes that unit when it runs (if another `sema_p` took it first, it sleeps again).

Readers/writer lock

A lock with two modes: any number of threads may hold it together as readers, or exactly one thread may hold it as a writer, never both at once.

rw_enter

Acquire a readers/writer lock as a reader (RW_READER) or as a writer (RW_WRITER), waiting if that mode is not available right now.

rw_exit

Release a readers/writer lock held in either mode. When the writer or the last reader leaves, waiting threads can be admitted.

rw_downgrade

Turn the write lock you hold into a read lock without ever letting go of the lock. Under the classic Solaris rule, waiting writers keep waiting, and only if no writer is waiting are the waiting readers admitted as well.

rw_tryupgrade

Try to turn the read lock you hold into a write lock. It succeeds only if you are the only reader and no writer is waiting; otherwise it fails at once and you still hold your read lock.

cv_wait

Condition-variable wait: in one atomic step, release the given mutex and go to sleep on the condition variable; when woken, reacquire the mutex before returning.

cv_signal

Wake one thread that is sleeping on a condition variable. If no thread is waiting, nothing happens and nothing is remembered.

cv_broadcast

Wake every thread that is sleeping on a condition variable. Each one reacquires the mutex in turn and re-tests its own condition.

Spurious wakeup

A return from a wait that finds the condition being waited for not true: no thread made it true, or another thread got to the data first and made it false again. Re-testing the condition in a while loop handles it.

Review questions with answers

1. Which of these is **not** one of the four thread synchronization primitives that Solaris adds on top of the classic UNIX mechanisms?

- A. Readers/writer lock
- B. Condition variable
- C. Semaphore
- D. Message queue

Answer: D. Message queue

The four are mutex locks, semaphores, readers/writer locks and condition variables, provided in the kernel for kernel threads and in the threads library for user threads. Message queues belong to the classic UNIX tools that Solaris also keeps.

2. Any thread may call `mutex_exit` on a Solaris mutex lock, as long as some thread currently holds the lock.

Answer: False

Only the owner, the thread that acquired the lock, may release it. A release by any other thread is a bug, and the Solaris kernel detects it and panics (halts the whole system with an error report).

3. Thread T calls `mutex_tryenter` on a mutex lock that another thread holds. What happens?

- A. The call fails at once and T keeps running
- B. T sleeps on the turnstile until the lock is released
- C. T spins until the lock is released
- D. T takes the lock away from its owner

Answer: A. The call fails at once and T keeps running

A try operation acquires only if it can do so immediately. Otherwise it returns failure (0) at once, and the caller can do other work and try again later.

4. A Solaris semaphore starts with a count of 2. Three threads each call `sema_p`, one after another. Next, a fourth thread calls `sema_try`. Finally, a thread calls `sema_v`, and the thread it wakes finishes its `sema_p`. What is the count now?

Answer: 0

The first two `sema_p` calls take both units ($2 \rightarrow 1 \rightarrow 0$). The third finds 0 and sleeps; the count stays 0, because a Solaris count never goes negative. `sema_try` finds 0 and fails without changing anything. `sema_v` adds 1 (count 1) and wakes the sleeper; when it runs, it takes that unit to finish its `sema_p`, so the count is back to 0.

5. Which statements about the Solaris adaptive mutex are true?

- A. A waiter spins while the owner is running on another processor
- B. A waiter sleeps on a turnstile when the owner is not running
- C. A waiter always sleeps, because spinning wastes processor time
- D. It may spin even when sleeping would have used less processor time, because it cannot know how long the owner will keep the lock

Answer: A, B, D

The adaptive mutex looks at the owner. Running elsewhere means the lock will probably be free soon, so it spins; not running means the wait could be long, so it sleeps. It bets on short hold times, the common case in the kernel, so it can lose that bet on an unusually long wait.

6. Put the steps of a correct condition-variable wait in order.

Correct order:

1. `mutex_enter(&m)` locks the shared data
2. The thread tests its condition and finds it false
3. `cv_wait` releases m and puts the thread to sleep, in one atomic step
4. Another thread changes the data and calls `cv_signal`
5. The sleeper wakes and reacquires m before `cv_wait` returns
6. The thread re-tests the condition in its while loop and finds it true
7. The thread uses the shared data, then calls `mutex_exit(&m)`

`cv_wait` is called with the mutex held, gives it up while sleeping, and returns with it held again. The re-test after waking is what makes the pattern safe.

7. Match each Solaris call to its effect.

sema_v	Adds 1 to a count and may wake one sleeper
rw_downgrade	Turns a write lock into a read lock without letting go
rw_tryupgrade	Turns a read lock into a write lock, only if that is possible at once
cv_broadcast	Wakes every thread sleeping on a condition variable
mutex_enter	Acquires a lock, waiting if another thread holds it
cv_wait	Releases a mutex and sleeps, then reacquires the mutex before returning

Acquire calls may wait, try calls never do, downgrade and upgrade change the mode of a readers/writer lock you already hold, and the cv_ calls sleep and wake around a condition.

8. Can each call make its caller wait?

May wait	Never waits
mutex_enter sema_p rw_enter cv_wait	mutex_tryenter rw_tryupgrade cv_signal rw_downgrade

Calls that acquire something (mutex_enter, sema_p, rw_enter) may wait, and cv_wait always sleeps. The try versions, the wake-up calls and rw_downgrade never wait: downgrading only gives up rights, so there is nothing to wait for.

9. A thread holds a Solaris readers/writer lock as a writer. Writer W2 is waiting for the lock, and so is reader R1. The holder calls rw_downgrade. Under the classic Solaris writers-first rules, what happens?

- A. W2 gets the write lock immediately
- B. R1 joins the holder as a reader immediately
- C. The holder becomes a reader, and W2 and R1 both keep waiting
- D. The holder releases the lock entirely, and W2 and R1 compete for it

Answer: C. The holder becomes a reader, and W2 and R1 both keep waiting

rw_downgrade converts the write lock to a read lock in place. Waiting writers keep waiting, and waiting readers are woken only if no writer is waiting. (Newer kernels such as illumos also weigh thread priority, so there a reader whose priority is at least the writer's may be admitted.)

10. Why must a thread re-test its condition in a while loop after `cv_wait` returns?

- A. Because `cv_wait` returns without holding the mutex, so the shared data must be read twice to be safe
- B. Because before it gets the mutex back, another thread may change the data, so the condition may be false again
- C. Because `cv_signal` always wakes every thread that is waiting on that condition variable, never just one
- D. Because the compiler may skip a plain if statement after `cv_wait`, but it must always keep a while loop

Answer: B. Because before it gets the mutex back, another thread may change the data, so the condition may be false again

A wake-up is only a hint that the condition may now hold, the Mesa-style behavior from Chapter 5. Another thread can get the mutex first and change the data, so the woken thread must check again and wait again if needed.

11. A Solaris condition variable remembers a `cv_signal` sent while no thread is waiting, so the next `cv_wait` returns at once.

Answer: False

A condition variable keeps no count. A signal with no sleeper is simply lost. That is why the condition itself lives in shared data protected by the mutex, and why a semaphore is the tool for a signal that must be remembered.

12. A thread finds a mutex lock held. The owner is running on another processor and will release the lock in 20 μ s. Sleeping would cost the waiter two thread switches of 4 μ s each. How many μ s of processor time would sleeping save compared with spinning?

Answer: 12 μ s

Spinning burns the whole wait: 20 μ s. Sleeping costs $2 \times 4 = 8$ μ s. Sleeping saves $20 - 8 = 12$ μ s. An adaptive mutex would still spin here, because the owner is running and it cannot know the wait will be that long.

6.10 Windows Concurrency Mechanisms

You should be able to

- Explain how a wait function puts a thread to sleep until a dispatcher object is signaled, and use `WaitForSingleObject` and `WaitForMultipleObjects` in wait-any and wait-all form.
- State, for each of the eight kinds of dispatcher object, when it becomes signaled and how many waiting threads it releases.
- Compare critical sections with mutexes and explain why a critical section usually never enters the kernel.
- Use slim reader-writer locks and condition variables correctly, re-checking the condition in a loop.
- Update shared variables without a lock using interlocked operations, including `InterlockedCompareExchange`.

The Windows toolbox

Windows builds synchronization into its object architecture. **Dispatcher objects** with **wait functions** are kernel objects: any process holding a handle can use them, and every wait and release is a system call.

Critical sections, **SRW locks** and **condition variables** live in one process's memory, serve only its threads, and enter the kernel only when a thread must sleep. **Interlocked operations** need no lock and never enter the kernel.

Wait functions

`WaitForSingleObject(h, ms)` waits on one object; `WaitForMultipleObjects` on several. The call checks the condition: if it holds, it returns at once; if not, the thread enters the **Waiting** state and uses **no processor time** until released (no busy waiting). Time limit: `INFINITE` waits forever, `0` only tests. Returns: `WAIT_OBJECT_0` (signaled), `WAIT_TIMEOUT` (time ran out; do not touch the data), `WAIT_ABANDONED` (a mutex owner died), `WAIT_FAILED` (error). Example: limit 2000 ms, event set at 700 ms: the wait returns `WAIT_OBJECT_0` after 700 ms, having used 0 ms of processor time.

Dispatcher objects

Always **signaled** or **nonsignaled**; a waiting thread is released when the object becomes signaled.

Object	What it is	Signaled when	Waiters released
Notification event	announcement that an event happened	a thread sets it	all
Synchronization event	the same, but resets itself	a thread sets it	one
Mutex	mutual exclusion, owned by a thread	the owner releases it	one
Semaphore	counter limiting users of a resource	count above zero	one per unit
Waitable timer	counter for the passage of time	set time arrives / interval ends	all
File	open file or I/O device	an I/O operation completes	all
Process	running program and its resources	its last thread terminates	all
Thread	executable entity in a process	the thread terminates	all

- The first five exist to synchronize; files, processes and threads can also be waited on (e.g. wait for a child process to exit).
- Notification event = manual-reset: stays signaled until `ResetEvent`, so later waiters pass.
Synchronization event = auto-reset: with nobody waiting it stays signaled until one thread waits. Events do not count: a second `SetEvent` before the first is used is lost.
- Mutex: only the owner releases it; the owner may re-acquire it (counted); it can be named and shared by processes. If the owner ends without releasing it, it is **abandoned** and the next owner's wait returns `WAIT_ABANDONED`.
- Semaphore: `ReleaseSemaphore(h, n, ...)` adds n; each released waiter takes one unit; a release above the maximum fails. Count 0, three waiters, release 2: two wake, count stays 0.
- A synchronization timer releases only one waiter. Windows does not promise first-in, first-out order among waiters.

WaitForMultipleObjects

Up to 64 handles plus the flag `bWaitAll`. **FALSE (any)**: returns when one object is signaled; return value – `WAIT_OBJECT_0` = its index, the **lowest index** if several are signaled (`WAIT_ABANDONED_0` + index if it is an abandoned mutex); only that object is claimed. **TRUE (all)**: returns only when all are signaled at once and claims them in one atomic step. Until then it claims nothing, so another thread may take a free mutex; the waiter never holds one object while sleeping on another (no hold and wait).

Critical sections

`CRITICAL_SECTION`, for the threads of one process: `InitializeCriticalSectionAndSpinCount`, `EnterCriticalSection`, `LeaveCriticalSection`. (1) Free: one interlocked instruction in user mode, no system call. (2) Busy on a multiprocessor: spin up to the **spin count** (ignored on one processor, where the holder cannot run while you spin). (3) Still busy: sleep on a kernel object until `Leave` wakes it. Unlike a mutex: no use across processes, no time limit or multi-object wait, no abandonment warning. Illustrative cost (interlocked 1, system call 30, sleep + wake 300): 10% contention, holder done after 100, spin count 4000, 4 processors: mutex $0.9 \times 60 + 0.1 \times 360 = 90$ per lock/unlock, critical section $0.9 \times 2 + 0.1 \times 102 = 12$. Spinning pays only if the holder lets go sooner than a sleep would cost.

SRW locks and condition variables

An SRW lock is pointer-sized (since Windows Vista), for one process. **Shared** mode (`AcquireSRWLockShared` / `ReleaseSRWLockShared`): many readers at once. **Exclusive** mode (`AcquireSRWLockExclusive` / `ReleaseSRWLockExclusive`): one writer alone. Not recursive: it records no owner, so a holder that asks again waits on itself forever. No promise of waiter order.

A condition variable works with a critical section or SRW lock.

`SleepConditionVariableCS` / `SleepConditionVariableSRW` release the lock and sleep atomically, then re-acquire it before returning. `WakeConditionVariable` wakes one; `WakeAllConditionVariable` wakes all. Re-test in a **while** loop: another thread may take the data first, and **spurious wakeups** happen.

```
while (q.count == 0)    // re-test after every wake
    SleepConditionVariableSRW(&cv, &lk, INFINITE, 0);
```

Interlocked operations

Function	Does, atomically	Returns
<code>InterlockedIncrement(&x)</code>	$x = x + 1$	new value
<code>InterlockedDecrement(&x)</code>	$x = x - 1$	new value
<code>InterlockedExchange(&x, v)</code>	$x = v$	old value
<code>InterlockedCompareExchange(&x, v, c)</code>	$x = v$ only if $x == c$	old value, always

`CompareExchange` is chapter 5's `compare_and_swap`. $x = 7$: `InterlockedCompareExchange(&x, 9, 4)` stores nothing and returns 7. A plain `hits++` (load, add, store) can lose an update: two threads load 5 and

both store 6. A loop that reads old, computes old + 1 and stores only if hits still equals old (else retries) never loses one and never blocks: **lock-free synchronization**.

Choosing a tool

Across processes: named **mutex**. N identical resources: **semaphore**. Tell everyone once: **notification event**. Wait for a program to exit: its **process**. Short locking in one process, possibly re-entered: **critical section**. Read-mostly data: **SRW lock**. Sleep under a lock until data changes: **condition variable**. One counter: **interlocked operation**.

Key terms

Wait function

A Windows call, such as `WaitForSingleObject` or `WaitForMultipleObjects`, with which a thread blocks itself until the objects it names are signaled or a time limit runs out. While it waits, the thread uses no processor time.

Dispatcher object

A Windows kernel object that a thread can wait on with a wait function. It is always in one of two states, signaled or nonsignaled, and waiting threads are released when it becomes signaled.

Signaled state

The "go" state of a dispatcher object: a thread waiting on it may continue. Its opposite, the nonsignaled state, keeps waiting threads asleep.

Notification event (manual-reset event)

An event object that, once set, stays signaled and releases every waiting thread, and any thread that waits later, until some thread resets it.

Synchronization event (auto-reset event)

An event object that, when set, releases exactly one waiting thread and then returns to nonsignaled by itself. If nobody is waiting, it stays signaled until one thread waits.

Abandoned mutex

A Windows mutex whose owning thread ended without releasing it. The next thread to get it receives ownership plus a warning (`WAIT_ABANDONED`), because the data it protects may be half-updated.

Waitable timer

A dispatcher object that becomes signaled when a chosen time arrives or a repeating interval expires, so threads can sleep until then.

Critical section object

A Windows lock (type `CRITICAL_SECTION`) for the threads of one process only. Taking it when it is free costs one interlocked instruction in user mode; the kernel is involved only when a thread must sleep.

Contention

A thread asks for a lock that another thread is holding at that moment. A lock that is free when it is requested is uncontended.

Spin count

How many times a thread that finds a critical section busy re-checks it in a tight loop before it gives up and sleeps in the kernel. Single-processor machines ignore it.

Slim reader-writer lock (SRW lock)

A pointer-sized Windows lock for the threads of one process, with a shared mode that many readers may hold together and an exclusive mode that one writer holds alone. It cannot be acquired recursively.

Shared mode

The way readers hold an SRW lock: any number of threads may hold it in shared mode at once, as long as nobody holds it in exclusive mode.

Exclusive mode

The way a writer holds an SRW lock: one thread alone, with no readers and no other writer.

Spurious wakeup

A return from a condition-variable sleep that no wake call caused. Windows allows them, so a woken thread must re-test its condition, just as it must when another thread got to the data first.

Interlocked operation

A Windows function, such as `InterlockedIncrement`, that reads, changes and writes one shared variable as a single atomic hardware operation, so no lock is needed.

InterlockedCompareExchange

An interlocked operation that stores a new value in a variable only if the variable still holds an expected value, and always returns the value it found there. It is the Windows form of `compare_and_swap`.

Lock-free synchronization

Keeping shared data correct without any lock: threads update it with atomic instructions, and a thread whose update loses a race simply retries, so no thread ever blocks.

Review questions with answers

1. A thread calls `WaitForSingleObject` with a time limit of `INFINITE` on an event that is nonsignaled. What does the thread do until the event is set?

- A. It loops inside the call, re-checking the event as fast as it can
- B. It enters the Waiting state and uses no processor time until the kernel releases it
- C. The call fails at once with `WAIT_FAILED`, because the event is not set
- D. It keeps running the code after the call and is interrupted when the event is set

Answer: B. It enters the Waiting state and uses no processor time until the kernel releases it

When the wait condition is not met, the thread enters the Waiting state and the dispatcher runs other threads. The kernel makes the waiter ready again when the object becomes signaled (or when a finite time limit runs out).

2. Match each Windows dispatcher object with the moment it becomes signaled.

Notification event	A thread sets it
Mutex	Its owning thread releases it
Semaphore	Its count is above zero
Waitable timer	Its set time arrives
Process	Its last thread terminates
File	An I/O operation on it completes

Every dispatcher object is either signaled or nonsignaled, and each type has its own rule for becoming signaled. A waiting thread is released when that happens.

3. When each of these Windows objects becomes signaled, how many of its waiting threads are released?

All waiting threads	Just one waiting thread
Notification event Waitable timer (notification kind) Process Thread	Synchronization event Mutex, when its owner releases it

Objects that announce that something happened (notification events and timers, files, processes, threads) release every waiter. A synchronization event releases one and resets itself; a released mutex goes to one new owner.

4. A Windows semaphore has a count of 0 and a maximum count of 10, and 5 threads are waiting on it. Another thread calls `ReleaseSemaphore` with a release count of 3. How many of the waiting threads are released?

Answer: 3 threads

Each released waiter takes one unit of the count. Three units release three waiters, the count drops back to 0, and the other two threads keep waiting.

5. Why is entering a free critical section much cheaper than waiting on a free mutex?

- A. The critical section disables interrupts for as long as it is held, so no other thread can cut in
- B. A mutex always puts the waiting thread to sleep at least once, even when the mutex is already free
- C. A free critical section takes one interlocked instruction in user mode; every mutex wait is a system call
- D. A critical section skips mutual exclusion altogether whenever the machine has only one processor

Answer: C. A free critical section takes one interlocked instruction in user mode; every mutex wait is a system call

An uncontended critical section never leaves user mode. A mutex is a dispatcher object, so each wait and each release is a system call, which costs far more than one interlocked instruction.

6. Which statements about Windows slim reader-writer (SRW) locks are true?

- A. Many threads can hold one in shared mode at the same time
- B. A thread holding one in exclusive mode has it to itself
- C. The holder may acquire it again recursively
- D. It is about the size of one pointer
- E. It can be used together with a condition variable
- F. Waiting threads are guaranteed to get it in first-in, first-out order

Answer: A, B, D, E

SRW locks are small and fast: readers share, a writer is alone, and `SleepConditionVariableSRW` works with them. They cannot be acquired recursively, and they make no promise about the order in which waiters get the lock.

7. Put the steps of a consumer that waits on a condition variable with an SRW lock in order.

Correct order:

1. Acquire the SRW lock
2. Test the condition and find it false
3. `SleepConditionVariableSRW` releases the lock and sleeps, as one atomic step
4. Another thread changes the data and calls `WakeConditionVariable`
5. The sleeper re-acquires the lock before its sleep call returns
6. Test the condition again at the top of the while loop

Lock, test, then sleep (which unlocks atomically). After a wake, the sleep call re-acquires the lock before returning, and the while loop re-tests the condition, because it may be false again.

8. When `SleepConditionVariableSRW` returns, the condition the thread was waiting for is guaranteed to be true.

Answer: False

Another thread may have changed the data first (a stolen wake-up), and Windows also allows spurious wakeups that nobody sent. That is why the condition is re-tested in a while loop.

9. A shared variable `x` holds 7. A thread calls `InterlockedCompareExchange(&x, 9, 4)`, with new value 9 and comparand 4. What value does the call return?

Answer: 7

`x` (7) does not equal the comparand (4), so nothing is stored and `x` stays 7. The function always returns the value it found, 7, which tells the caller its swap did not happen.

10. A thread calls `WaitForMultipleObjects` on a mutex and a semaphore with `bWaitAll = TRUE`. The mutex becomes free, but the semaphore's count is still 0. What happens to the mutex?

- A. The thread takes ownership now and keeps it while it waits for the semaphore
- B. Windows reserves the mutex for this thread so that no other thread can take it
- C. The call returns at once, and its return value reports the mutex's index
- D. Nothing yet: the thread does not take it, so another thread may acquire it meanwhile

Answer: D. Nothing yet: the thread does not take it, so another thread may acquire it meanwhile

In wait-all mode Windows claims nothing until every object is signaled at the same moment, then claims them all in one step. The waiting thread never holds one object while sleeping on another.

11. A thread that owns a Windows mutex terminates without releasing it while another thread waits for that mutex. What happens?

- A. The waiter gets ownership, and its wait returns `WAIT_ABANDONED` as a warning that the data may be inconsistent
- B. The mutex stays owned by the dead thread forever, so the waiter is blocked for good
- C. The waiter gets ownership with `WAIT_OBJECT_0`, exactly as if the mutex had been released normally
- D. Windows terminates the waiting thread as well, since it depended on the dead one

Answer: A. The waiter gets ownership, and its wait returns `WAIT_ABANDONED` as a warning that the data may be inconsistent

The mutex becomes abandoned. The next thread to acquire it owns it, but its wait returns `WAIT_ABANDONED` so it can check or repair the data the dead owner may have left half-changed.

12. A thread calls `WaitForMultipleObjects` on 4 handles (indexes 0 to 3) with `bWaitAll = FALSE`. When the wait is satisfied, the objects at index 1 and index 3 are both signaled. What is the return value minus `WAIT_OBJECT_0`?

Answer: 1

In wait-any mode the return value identifies the object that satisfied the wait. When several are signaled, it is the one with the lowest index, here 1.

6.11 Android Interprocess Communication

You should be able to

- Explain why sandboxed Android apps must communicate through the kernel, and why Android adds Binder to the Linux IPC tools.
- Describe a remote procedure call and trace one Binder transaction through proxy, parcel, driver, Binder thread, stub and service, two-way and one-way.
- Marshal a method call into a parcel, unmarshal it again, and explain why proxy and stub must come from the same AIDL interface.
- Predict how a server's Binder thread pool serves several callers at once, including when every thread is busy.
- Explain what the Binder driver adds (one copy, caller identity, counted references, a service registry) and compare Binder with pipes and shared memory.

Why Android needs interprocess communication

Every Android app normally runs in its **own process** under its **own Linux user ID (UID)**. This is the application sandbox: the kernel gives each process a separate address space and checks every file access against the UID, so one app cannot read another app's memory or files. So any request from one process to another must travel **through the kernel**.

Linux already offers IPC tools: **pipes** (one-way byte streams), **shared memory** (one region mapped into several processes), **sockets** (two-way connections, even across machines) and **signals** (tiny event notices that carry no message). Android adds **Binder**: a lightweight **remote procedure call** mechanism that is

efficient in memory and processing time, which suits the limited resources of a phone. **All interaction between app components in different processes goes through Binder.**

Remote procedure call (RPC)

A remote procedure call is written exactly like an ordinary call, but the procedure runs in **another process**; the arguments and the result travel between the two as messages. A local call takes nanoseconds and never involves the kernel; a Binder call crosses the kernel both ways, typically tens of microseconds, so apps avoid many tiny remote calls.

The cast of characters

Part	Where	Job
Proxy	client	Stand-in with the remote object's methods; marshals, sends, waits, unmarshals the reply.
Parcel	travels	Flat container of plain values (no pointers).
Binder driver	kernel	Carries transactions, copies once, stamps UID and PID, tracks references.
Binder thread pool	server	Threads waiting in the driver; each call goes to one free thread.
Stub	server	Unmarshals the arguments, calls the real method, marshals the result.
Service object	server	Does the real work; never sees a parcel.
Service manager	own process	Registry of named system services, reached through handle 0.
AIDL file	build time	Declares the interface; proxy and stub are generated from it.

One Binder transaction, step by step

1. (Once) The client finds the service (by name in the service manager for a system service, with `bindService()` for an app's) and gets a **handle**, wrapped in a proxy.
2. The client calls a method on the **proxy**, as if it were local.
3. The proxy **marshals** the arguments into a **parcel**: an interface token first, then each argument.
4. The proxy calls `transact(method code, parcel, flags)` and enters the **Binder driver**; for a two-way call the calling thread now **blocks**.
5. The driver turns the handle into the target object, records the sender's UID and PID, and copies the parcel **once** into the server's mapped receive buffer.
6. A thread from the server's **Binder thread pool** wakes up with the transaction; the **stub** checks the token and **unmarshals** the arguments in the same order.
7. The stub calls the real method on the service object.
8. The stub marshals a reply (status 0 = no exception, then the result); the driver copies it to the client and wakes the caller; the proxy unmarshals it and returns.

One-way calls (declared `oneway` in AIDL, sent with `FLAG_ONEWAY`): the caller continues as soon as the driver accepts the transaction. There is no reply, so the caller never learns whether the method worked.

One-way calls to the same object are delivered one at a time, in order.

Worked example. `getTemp("Tampa")`: token "IWeather" = 24 bytes, string "Tampa" = 16 bytes, request = 40 bytes; reply = status 4 bytes + int 4 bytes = 8 bytes.

Marshalling and unmarshalling

- **int**: 4 bytes, least significant byte first (little-endian). 32 → 20 00 00 00.
- **boolean**: stored as a whole 4-byte int, 1 or 0.
- **String**: a 4-byte length (in characters), 2 bytes per UTF-16 character, a 2-byte zero terminator, padded to a multiple of 4. Bytes = 4 + round-up-to-4(2 × (n + 1)). Example: "Denver" (n = 6): 4 + 14 → 4 + 16 = **20 bytes**.

A parcel stores values by **position, not by name**. The stub must read exactly the types, in exactly the order, the proxy wrote. If one side is rebuilt from an AIDL file whose parameters were reordered and the other is not, values are silently misread or a read fails. Generating both halves from one AIDL file prevents this.

The Binder thread pool

Incoming calls run on the server's Binder threads, not its main thread. When all are busy the driver asks the process to start another, by default up to 15 more beside the first (16 in all for an app); beyond that, calls wait in the driver's queue, first in, first out, and their callers stay blocked. Example: 2 threads, 4 calls arriving together, 30 ms each: two run 0–30 ms, two run 30–60 ms, so the last reply leaves at 60 ms. A blocked main thread freezes the screen (after about 5 s without responding to input, Android offers to close the app), so slow calls belong on background threads.

What the Binder driver adds

- **One copy.** The driver copies a parcel straight into a receive buffer the receiver has mapped. It holds 1 MB minus 8 KB, shared by all calls in progress (a one-way call may use only half); bigger transactions fail (`TransactionTooLargeException`), so large data goes in shared memory and only its file descriptor travels through Binder.
- **Caller identity.** The driver records the sender's UID (and PID, for two-way calls); the service reads them with `Binder.getCallingUid()` and `getCallingPid()` to check permissions. The caller cannot forge them; anything inside a parcel was written by the caller and proves nothing.
- **Counted references.** Parcels can carry Binder object references. The driver gives each process its own handle number for an object (like a file descriptor) and counts references; at zero the owner is told it may free the object. If the owner dies, holders that asked (`linkToDeath`) get a **death notification**; calls through a dead handle fail with `DeadObjectException`.
- **Service manager.** Registry reached through handle 0, where system services are found by name; an app's service is reached with `bindService()`.

Binder compared with a pipe and shared memory

	Pipe	Shared memory	Binder
Kernel copies	2 (in and out)	0	1
What crosses	byte stream, one way	nothing: same memory	typed method call and reply
Knows the sender	no	no	yes: UID and PID
Waiting	reader blocks for data	none: add semaphores	caller blocks for reply (unless one-way)
Finding the other side	inherited or named pipe	map the same region	by name, or by binding
Object references	no	no	counted, with death notices

Example: 300 KB through a pipe = 600 KB copied; through Binder, 300 KB; through shared memory, none.

Built on Binder

- **Intents:** start an activity or service, or announce an event (a broadcast such as "battery low").
- **Content providers:** share a set of data (such as contacts) through queries with permission checks.
- **Bound services:** bind once, then call the service's Binder interface (usually AIDL) directly, again and again.

Key terms

Binder

Android's main interprocess communication mechanism: a kernel driver plus user-space libraries that let a thread in one process call a method on an object in another process as if the object were local.

Binder driver

The part of Binder that lives inside the Linux kernel (opened as `/dev/binder`). It carries every transaction between processes, copies the data, records who sent it and keeps track of object references.

Remote procedure call (RPC)

A call that looks like an ordinary procedure call in the caller's code, but the procedure really runs in another process (or on another machine); the arguments and the result travel between them as messages.

Proxy

The client-side stand-in for a remote object. It offers the same methods as the real object, but each method only packs its arguments into a parcel, sends it through Binder and unpacks the reply.

Stub

The server-side partner of a proxy. It receives an incoming transaction, unpacks the arguments, calls the real method on the service object, and packs the result into a reply.

Parcel

A flat container of data (numbers, strings, object references, file descriptors) laid out byte by byte so that it can be copied from one process into another. Binder requests and replies travel as parcels.

Marshalling

Packing a call's arguments (or its result) into a flat, agreed format such as a parcel, so that the values can cross a process boundary.

Unmarshalling

The reverse of marshalling: reading the fields back out of a parcel, in the agreed order, to rebuild the original values.

Binder transaction

One message carried by the Binder driver: the target object, a method code, flags such as one-way, the parcel of data, and the sender's identity, which the driver adds.

Binder thread pool

A set of threads in a server process that wait inside the Binder driver for incoming calls. Each incoming transaction is handed to one free thread of the pool.

One-way call

A Binder call (declared oneway) whose caller does not wait for a reply: it carries on as soon as the driver has accepted the transaction.

Service manager

A special Android process that acts as the registry of named system services. They register under a name; clients look the name up and receive a reference (an ordinary app's service is reached through the system with `bindService()` instead). Every process reaches it through handle 0.

Binder handle

A small per-process number that names a remote Binder object, much as a file descriptor names an open file. The driver turns handles into the real objects and counts the references to each object.

Calling UID

The Linux user ID (and, for a two-way call, the process ID) of whoever sent the Binder call now being served. The driver records it, so the caller cannot forge it and the service can trust it for permission checks.

AIDL (Android Interface Definition Language)

A small language for declaring the methods of a Binder interface. Build tools turn one AIDL file into matching proxy and stub code, so both sides pack and unpack parcels in exactly the same way.

Intent

A message object that asks Android to start an activity or a service, or announces an event to every app that registered for it. Intents reach the system through Binder.

Content provider

An app component that shares a set of data, such as the contacts list, through a standard query interface. Other apps reach it through Binder, and it can check their permissions.

Bound service

A service that other components connect (bind) to and then call directly through a Binder interface for as long as they stay connected.

Pipe

A one-way channel of bytes kept by the kernel: one process writes bytes in at one end and another reads them out, in the same order, at the other.

Review questions with answers

1. Why must two Android apps communicate through the kernel instead of reading each other's data directly?

- A. Apps are written in Java or Kotlin, and those languages have no pointers that could reach another app's memory.
- B. Android forbids apps from opening any files at all, so every piece of data must be sent through the kernel instead.
- C. Each app runs in its own process, with its own address space and Linux user ID, so neither can reach the other's memory.
- D. All apps share one large address space, and the kernel must check every access so that they do not collide.

Answer: C. Each app runs in its own process, with its own address space and Linux user ID, so neither can reach the other's memory.

The application sandbox gives every app its own process and UID. Only the kernel can see into every process, so any data that crosses from one to another must pass through it; on Android that is normally Binder.

2. Put the stages of a two-way Binder call in order.

Correct order:

- 1. The client calls a method on the proxy
- 2. The proxy marshals the arguments into a parcel
- 3. The proxy hands the transaction to the Binder driver and the calling thread blocks
- 4. The driver copies the data into the server and records the caller's UID and PID
- 5. A Binder thread in the server receives it and the stub unmarshals the arguments
- 6. The stub calls the real method on the service
- 7. The reply travels back the same way and the proxy returns the result

Pack on the client side, cross the kernel once, unpack on the server side, run the method, and retrace the path with the reply while the caller waits.

3. Match each part of Binder to its job.

Proxy	Client-side stand-in that packs the arguments
Stub	Server-side code that unpacks the arguments and calls the real method
Binder driver	Kernel code that carries transactions and stamps the sender
Service manager	Registry where services are looked up by name
AIDL	Interface language from which proxy and stub are generated

Proxy and stub are the two generated halves of one interface (from AIDL); the driver in the kernel joins them; the service manager is how a client finds a service in the first place.

4. In a one-way (oneway) Binder call, the calling thread blocks until the server has finished running the method.

Answer: False

A one-way call returns as soon as the driver has accepted the transaction. The caller never waits for, or receives, a reply.

5. A proxy writes the string "Denver" into a parcel: a 4-byte length, then 2 bytes per character, then a 2-byte terminator, with the total padded up to a multiple of 4 bytes. How many bytes does the string take?

Answer: 20 bytes

$4 (\text{length}) + 6 \times 2 (\text{characters}) + 2 (\text{terminator}) = 18 \text{ bytes}$, padded up to the next multiple of 4: 20 bytes.

6. An app sends a 300 KB message to another process through a pipe. In total, how many KB of data does the kernel copy?

Answer: 600 KB

A pipe copies twice: `write()` copies 300 KB into the kernel's buffer and `read()` copies 300 KB out again, 600 KB in all. Binder would copy the 300 KB once.

7. A service has 2 Binder threads. Four calls arrive at the same moment; each needs 30 ms of work, and waiting calls are served first come, first served. How many milliseconds after they arrive is the last reply sent?

Answer: 60 ms

Two calls run at once from 0 to 30 ms; the other two wait in the driver and then run from 30 to 60 ms. The last reply goes out at 60 ms.

8. Which of these does the Binder driver do? Select all that apply.

- A. Copies a parcel once, straight into a buffer mapped into the receiving process
- B. Stamps each transaction with the caller's UID, which the caller cannot forge
- C. Counts references to Binder objects that are shared between processes
- D. Encrypts every parcel
- E. Decides the order in which arguments are written into the parcel

Answer: A, B, C

The driver copies once, stamps the sender's identity, and tracks object references (and death notices). It does not encrypt, and the argument order is fixed by the interface, in the generated proxy and stub code.

9. A location service must decide whether the app calling it may see the user's position. What should it trust to identify the caller?

- A. A "my UID" field that the caller wrote into the parcel
- B. `Binder.getCallingUid()`, which the driver filled in
- C. The interface token at the start of the parcel
- D. The method code of the transaction

Answer: B. Binder.getCallingUid(), which the driver filled in

Only the kernel-written stamp is trustworthy, because the caller cannot change it. Android permission checks are built on the calling UID and PID.

10. Sort each property into the mechanism it describes.

Pipe	Shared memory	Binder
A one-way stream of bytes, copied into and out of the kernel The reader is never told who wrote the bytes	No copying at all, but you must add your own synchronization Best for large data such as video frames	Typed method calls with a reply The caller's identity is stamped by the kernel

Pipes move raw byte streams with two copies and no sender identity; shared memory skips copying but gives no calls, no waiting and no identity; Binder adds typed calls, replies, one copy and a trustworthy identity.

11. One side of a Binder interface was rebuilt from an AIDL file whose parameters were put in a different order; the other side was not. What happens when a call is made?

- A. The driver notices the mismatch and reorders the fields before delivering the parcel.
- B. The stub matches fields by name, so nothing goes wrong and every value arrives correctly.
- C. The interface token check always rejects the call, so the stub never reads any fields.
- D. Values are misread, or a read fails, because a parcel stores fields by position, not by name.

Answer: D. Values are misread, or a read fails, because a parcel stores fields by position, not by name.

Proxy and stub must be generated from the same interface definition. Change the order on one side only and every later field is read as the wrong thing, sometimes silently.

12. Match each request to the Binder-based tool it normally uses.

Open a web page in whatever browser the user prefers	Intent
Read the phone's list of contacts	Content provider
Keep calling play() and pause() on a music app's playback service	Bound service

An intent asks the system to start something or announces an event; a content provider shares a set of data with permission checks; a bound service is called directly, again and again, through its Binder interface.

Chapter 7 · Memory Management

Many programs, one physical memory. The OS decides who lives where, and keeps them from trampling each other.

Every running process needs part of main memory, and there is never enough of it. This chapter sets out the five requirements any memory manager must meet (relocation, protection, sharing, logical organization and physical organization), then follows the history of solutions: carving memory into fixed or variable **partitions**, the **buddy system**, and finally cutting programs into small pieces with **paging** and **segmentation**. Along the way it shows how a logical address used by a program becomes a physical address in memory. An appendix explains how a program is loaded into memory and linked with the code it calls.

Learning objectives

- Describe the five requirements of memory management.
- Compare fixed partitioning, dynamic partitioning and the buddy system, including internal and external fragmentation.
- Simulate the first-fit, best-fit and next-fit placement algorithms.
- Translate a logical address to a physical address under paging and under segmentation.
- Explain loading and linking: absolute, relocatable and dynamic run-time loading, and static versus dynamic linking.

How Chapter 7 fits together

Processes share one main memory and come and go. Section 7.1 lists five jobs of a memory manager:

1. **Relocation:** a process may be loaded, or swapped back in, at any address, so its addresses must be translated to wherever it lands.
2. **Protection:** no process may touch another's memory, or the OS's, without permission; hardware checks every address as it is used.
3. **Sharing:** processes running the same code share one copy, under control.
4. **Logical organization:** programs are modules with different needs (read-only code, writable data).
5. **Physical organization:** memory is fast but small and volatile, the disk large and slow; moving data between them is the system's job, not the programmer's (the old answer was overlays).

Partitioning (7.2)

Fixed partitioning cuts memory into set partitions; each process takes a whole one, so the unused end is lost to **internal fragmentation**. **Dynamic partitioning** gives each process exactly what it asks for, but departures leave holes, and free memory split into pieces too small to use is **external fragmentation**; compaction merges the holes at a cost. Placement chooses a hole: first-fit, best-fit or next-fit. The **buddy system** splits power-of-two blocks in half and merges free buddies. Base and bounds registers relocate and protect each process.

Paging and segmentation (7.3, 7.4)

Paging cuts memory into equal frames and processes into pages of the same size; a page table maps each page to any free frame, and an address is (page number, offset). There is no external fragmentation, only a little internal fragmentation in each process's last page. **Segmentation** cuts a process into variable-length segments that match its logical parts; an address is (segment number, offset), checked against the segment's length. It suits protection and sharing but suffers external fragmentation.

A worked comparison

In 48 KB of memory, A (7 KB), B (11), C (9) and D (10) arrive, B leaves, then E (4) and F (12) arrive. With four 12 KB fixed partitions, F waits while $5 + 8 + 3 + 2 = 18$ KB sits unused inside partitions. With dynamic partitions and first fit, F waits because the 18 KB that is free lies in gaps of 7 KB and 11 KB. With 4 KB pages, F fits in three scattered frames, and the only waste is $1 + 3 + 2 = 6$ KB at the ends of last pages.

Loading and linking (7A)

A **linker** joins object modules into one load module; a **loader** puts it in memory using absolute, relocatable or dynamic run-time loading. Linking can be static, or dynamic at load time or run time with shared libraries.

Why it matters next

Chapter 8 leaves some pieces on disk entirely: virtual memory.

7.1 Memory Management Requirements

You should be able to

- Explain what memory management is, why multiprogramming needs it, and why it decides how busy the processor can be.
- Describe relocation and trace how a relative address is translated to the right physical address after a process moves.
- Explain why every memory reference must be checked at run time, and why the processor hardware, not the OS, must do the checking.
- Explain how controlled sharing saves memory without breaking protection, and why memory should match a program built from modules.
- Describe the two levels of storage and why moving information between them is the OS's job rather than the programmer's (overlays).

Why memory management exists

With **uniprogramming**, main memory has two parts: one for the operating system and one for the single program running now. With **multiprogramming**, the user part must be divided among many processes, and the division keeps changing as processes start, finish, block and are swapped out. Doing that dividing dynamically is **memory management**, and it is the OS's job.

It matters for performance: a process waiting for I/O cannot use the processor, so the processor stays busy only if memory holds enough ready processes. A rough model: if each of n processes independently waits

for I/O a fraction w of the time, the processor is idle only when all n wait at once, so it is busy $1 - w^n$ of the time. Example: $w = 0.8$ gives 20% busy with one process, $1 - 0.8^4 \approx 59\%$ with four, and about 74% with six.

The five requirements

Any memory-management scheme must provide, in this order: (1) relocation, (2) protection, (3) sharing, (4) logical organization, (5) physical organization.

1. Relocation

The programmer cannot know where a program will sit in memory when it runs, and a process that is swapped out may come back to a different place. So a program's memory references (its entry point, branch targets, data references and stack) are written as **relative addresses**, counted from the start of its process image, and each one must be translated to the right **physical address** at run time.

Example. Process P's image is 2,000 bytes: process control information (relative 0–199), program (200–1,199, entry point 200), data (1,200–1,599) and stack (1,600–1,999). A branch at relative 400 says *jump 920*. Loaded at 2,000, the branch must reach $2,000 + 920 = 2,920$. P is swapped out, a new process S takes 2,000–3,999, and P returns at 6,000. With run-time translation the code is unchanged and the hardware adds the new base: $6,000 + 920 = 6,920$, still inside P. If the addresses had instead been fixed once at the first load, the code would still say *jump 2,920*, which now lies inside S: P would run S's instructions. Re-patching every address on each swap-in is impractical because addresses also live in registers and on the stack.

2. Protection

A process must not reference memory belonging to another process, or to the OS, without permission. Addresses cannot be checked once at compile time, for two reasons: because of relocation the physical addresses are unknown until the program is loaded and change when it moves, and many addresses are computed while the program runs (array indexes, pointers). So **every memory reference must be checked as it happens**, and the check must be done by the **processor hardware**, not by OS software: the OS cannot anticipate every reference, and running OS code for each one would be far too slow.

Example (base and bounds, named here, detailed in 7.2). P occupies 6,000–7,999, so its base register holds 6,000 and its bounds register 8,000. The statement $x = a[i]$ with a at relative 1,300 and 4-byte elements uses relative address $1,300 + 4i$. For $i = 3$: $1,312 \rightarrow 7,312$, allowed. For $i = 60$: $1,540 \rightarrow 7,540$, allowed, because it is still inside P (a bug in P, but it only hurts P). For $i = 200$: $2,100 \rightarrow 8,100$, outside the region: the hardware traps to the OS. Protection keeps a process out of other memory; it does not stop it from damaging its own data.

Cost of software checking (rough model). Each instruction makes about 1.5 memory references (its own fetch plus a data access about half the time). If each check cost C OS instructions, every instruction would cost about $1 + 1.5C$ instruction times: with $C = 40$, about 61 times slower. And the OS is not even running while the process runs: the processor executes the process's instructions directly, and the OS gets control back only after an interrupt or a trap, so a software check would mean stopping the process before every single access.

3. Sharing

Protection must still allow **controlled sharing**. Several processes running the same program should share one copy of its code rather than each holding a copy, and cooperating processes may need to share a data structure such as a buffer. Shared code must not be modified while in use; each process keeps its own data, registers and stack. The mechanisms that support relocation also support sharing: each process's addresses are translated anyway, so two translations can lead to the same physical region, while everything not granted stays protected.

Example. Three editors, code 30 MB, each document 6 MB, a shared 4 MB clipboard. Private copies: $3 \times (30 + 6) + 4 = 112$ MB. One shared code copy: $30 + 3 \times 6 + 4 = 52$ MB, saving 60 MB = $(3 - 1) \times 30$ MB. A write into the shared code traps (it is execute-only); reading another editor's document traps (it is not in the reader's map).

4. Logical organization

Main memory is a flat, one-dimensional sequence of bytes or words, but programs are built from **modules** (main program, library routines, data tables). If memory management understands modules: modules can be written and compiled independently, with references between them resolved at run time; each module can get its own access rights (read-only, execute-only, read-write); and modules can be shared between processes. The tool that matches this best is **segmentation** (section 7.4).

Example. Modules main (1,200 bytes), sort (400), prices (600), work (800), stack (400) laid end to end: main's references read prices at 1,680 and write work at 2,400. If sort grows to 700 bytes, everything after it shifts by 300: 1,680 now falls inside sort and 2,400 inside prices, so every module must be re-linked. With module-relative references such as (prices, 80), nothing else changes.

5. Physical organization

	Main memory	Secondary memory
Speed	fast	slow
Cost per byte	high	low
Without power	contents lost (volatile)	contents kept (permanent)
Size	smaller	much larger

Information must move between the two levels, and that movement cannot be left to the programmer: (1) a program plus its data may not fit in the memory available, and the old fix, **overlaying** (modules that are never needed together take turns in one region, loaded by the program itself; later tools could insert the loads, but the programmer still had to plan which modules share the region), is tedious and error-prone; (2) under multiprogramming the programmer does not know how much space will be available. So the OS must manage the movement (virtual memory, chapter 8, is how it does so).

Example. main (20 KB, always resident) plus modules L (14 KB), C (30 KB) and P (16 KB) need 80 KB; with 64 KB free, the overlay area is 44 KB. L and C fit together (44 KB), C and P do not (46 KB). The calls L, C, P, C, P need at least 5 disk loads, and any forgotten load makes main jump into the wrong code. On a busy day with only 48 KB free, the overlay area is 28 KB and C (30 KB) cannot fit at all: the hand-made plan breaks.

Key terms

Memory management

The operating system's job of dividing the user part of main memory among processes as they arrive and leave, and of keeping track of which piece belongs to which process.

Relocation

Making a program run correctly wherever it is placed in main memory, including a different place after it is swapped out and back in, by translating its memory references to the right physical addresses.

Relative address

A logical address written as a distance from a known point, usually the first byte of the program, so it stays the same wherever the program is placed in memory.

Address translation

Turning an address used by a program into the physical address that is sent to memory. With run-time translation the processor hardware does it on every memory reference.

Memory reference

Any access a running program makes to memory: fetching an instruction, jumping to a branch target, reading or writing data, or pushing and popping the stack.

Entry point

The address of the first instruction a program runs when it starts.

Branch instruction (jump)

An instruction that tells the processor to continue at another address, the branch target, instead of the next instruction in line.

Memory protection (protection)

The requirement that a process cannot read or write memory belonging to another process or to the OS unless it has been given permission.

Bounds register

A processor register that holds the address where the running process's memory region ends. The hardware compares each translated address with it and traps to the OS if the address is at or beyond it. (Some processors store the region's length instead, in a limit register.)

Controlled sharing (sharing)

Letting several processes use the same region of main memory on purpose, such as one copy of a program's code or a data area they cooperate through, while protection still blocks every access that was not granted.

Shared code

One copy of a program's instructions in main memory that several processes run at the same time. It is never changed while in use, and each process keeps its own data separately.

Access rights

The permissions attached to a piece of memory, such as read-only, read-write or execute-only, which the hardware enforces on every access.

Logical organization

The requirement that memory management fit the way programs are really built: as separate modules that can be compiled on their own, given their own access rights and shared.

Segmentation

Dividing a program and its data into variable-length pieces called segments, usually one per logical part (code, a library, a data table, the stack); an address names a segment and an offset inside it.

Physical organization

The requirement that the OS, not the programmer, manage the two levels of storage: moving programs and data between small, fast, volatile main memory and large, slow, permanent secondary memory.

Overlaying (overlay)

An old, programmer-managed way to run a program larger than the memory available: modules that are never needed at the same moment take turns in the same region of memory, and the program itself must load each one before calling it. (Later tools could insert those loads, but the programmer still had to plan which modules share the region.)

Review questions with answers

1. In a multiprogramming system, why does good memory management help keep the processor busy?
- A. It makes each process's instructions execute faster, because its code and data are kept closer to the processor.
 - B. It lets one processor execute instructions from several processes at the same instant, one per memory region.
 - C. More ready processes fit in memory, so one is likely to be able to run whenever the others wait for I/O.
 - D. It removes the need for processes to wait for I/O, because the data they need is already in main memory.

Answer: C. More ready processes fit in memory, so one is likely to be able to run whenever the others wait for I/O.

The processor idles only when every process in memory is waiting. Fitting more ready processes into memory makes that moment rarer.

2. A process image was compiled with every address counted from its own first byte. It is now loaded starting at physical address 14,000. One of its branch instructions targets relative address 2,350. Which physical address must the branch actually reach?

Answer: 16350

With run-time relocation the hardware adds the start of the process's current region to the relative address: $14,000 + 2,350 = 16,350$.

3. In a multiprogramming system, a programmer can know in advance the physical address at which a program will run.

Answer: False

Where a program lands depends on what else is in memory when it is loaded, and swapping may later bring it back somewhere else. That is why relocation is needed.

4. Why can memory references not be checked for protection once and for all when the program is compiled?

- A. Compilers are not allowed to read the addresses inside a program, since only the OS may decide which addresses a process may use.
- B. Many addresses are computed while the program runs (array indexes, pointers), and its location is unknown until it is loaded and may change.
- C. Checking at compile time would make the program file too large to load, since a stored check would have to sit beside every address.
- D. The OS's own memory moves every time a process is created, so a check made at compile time would test against the wrong region.

Answer: B. Many addresses are computed while the program runs (array indexes, pointers), and its location is unknown until it is loaded and may change.

Relocation means the physical addresses are unknown until load time, and computed addresses such as `a[i]` exist only during execution. So each reference is checked as it happens.

5. Who must check each memory reference for protection while a program runs, and why?

- A. The OS, because only the OS knows which memory each process owns, so it can inspect each reference before the access is made.
- B. The compiler, because it generates every memory reference and can therefore prove that each one stays inside the program's region.
- C. The program itself, because it knows which addresses it means to use and can test each of its pointers before following it.
- D. The processor hardware, because the OS cannot foresee every computed address and a software check on each reference would be too slow.

Answer: D. The processor hardware, because the OS cannot foresee every computed address and a software check on each reference would be too slow.

Every instruction fetch and data access is a memory reference. Only hardware can check that many references at full speed, alongside the access itself.

6. A base-and-bounds hardware check stops a process from overwriting its own variables through an out-of-range array index.

Answer: False

The check only asks whether the address lies inside the process's own region. An index that strays into the same process's other data is still allowed: a bug in that process, but other processes and the OS stay safe.

7. Eight users run the same web browser at once. Its code takes 120 MB and each user's private data takes 40 MB. How many megabytes are saved by keeping one shared copy of the code instead of eight private copies?

Answer: 840 MB

Private copies need $8 \times 120 = 960$ MB of code; one shared copy needs 120 MB. The saving is $960 - 120 = 840$ MB (7×120). The 40 MB of private data per user is needed either way.

8. Which statements about sharing memory are correct?

- A. Several processes running the same program can share one copy of its code.
- B. Cooperating processes may share a data structure, such as a buffer.
- C. Once two processes share some memory, protection no longer applies between them.
- D. The translation mechanisms that support relocation also make sharing easy.
- E. Each process should modify the shared code to suit its own data.

Answer: A, B, D

Sharing is controlled: only the granted region is shared and everything else stays protected. Shared code must never be modified while in use, so each process keeps its own data elsewhere.

9. Match each memory-management requirement to the situation that illustrates it.

Relocation	A swapped-out process returns to a different region and must still run correctly
Protection	A stray pointer must not reach another process's memory
Sharing	Twenty users run one editor from a single copy of its code
Logical organization	A library module is compiled on its own and marked execute-only
Physical organization	A program bigger than the free memory must have parts moved between disk and memory

Moving means relocation; another process's memory means protection; one copy for many means sharing; modules mean logical organization; memory versus disk means physical organization.

10. Sort each property into the level of storage it describes.

Main memory	Secondary memory
Fast to access Loses its contents when the power is off Higher cost per byte	Slow to access Keeps its contents without power Lower cost per byte

Main memory is fast, expensive and volatile; secondary memory (disks, solid-state drives) is slower, cheaper and permanent. That trade-off is why both levels exist.

11. Why should the OS, rather than the programmer, move information between main memory and secondary memory?

- A. A program and its data may not fit in the memory available, and overlays are tedious and error-prone to write.
- B. Under multiprogramming, the programmer cannot know how much memory will be free when the program runs.
- C. Secondary memory is volatile, so only the OS may use it.
- D. Programs are never allowed to read files from disk.

Answer: A, B

These are the two classic reasons. Secondary memory is permanent, not volatile, and programs read files all the time through the OS; the point is who manages main memory's contents.

12. Which technique matches the logical organization requirement most closely?

- A. Swapping
- B. Overlaying
- C. Segmentation
- D. Uniprogramming

Answer: C. Segmentation

Segmentation divides a program into variable-length segments that match its modules, each with its own size and access rights, and each shareable on its own.

7.2 Memory Partitioning

You should be able to

- Explain fixed partitioning with equal and unequal partitions, and identify internal fragmentation and the need for overlays.
- Compare one queue per partition with a single queue, and choose which process to swap out when every partition is full.
- Run dynamic partitioning by hand, recognise external fragmentation, and explain what compaction fixes and what it costs.
- Apply the first-fit, best-fit and next-fit placement algorithms and describe how each tends to behave.
- Allocate and free blocks in a buddy system, and translate a relative address with base and bounds registers.

The problem

The oldest way to share main memory is to give each process one contiguous slice, a **partition**. Two kinds of waste result: **internal fragmentation** (unused space inside an allocated block, because the block is bigger than what was loaded) and **external fragmentation** (free memory broken into holes between allocated blocks, so a request can fail although the holes add up to enough).

Fixed partitioning

Memory after the OS is cut into partitions at start-up; their number and sizes never change.

- **Equal sizes:** any process no larger than a partition fits in any free one; if all are full, the OS can swap one out.
- A program bigger than a partition needs **overlays** (modules that take turns in the same region), and every process, however small, occupies a whole partition: internal fragmentation.
- **Unequal sizes** ease both problems.

Example (64 MB, OS 4 MB; jobs 2, 9, 4, 13, 6, 3, 24 MB in order). Six 10 MB partitions: 13 and 24 fit nowhere; the other five waste $8 + 1 + 6 + 4 + 7 = 26$ MB. Partitions of 3, 5, 7, 10, 15, 20 MB (smallest free one that fits): six jobs load, wasting $1 + 1 + 1 + 2 + 1 + 17 = 23$ MB.

Disadvantages: the number of partitions caps the number of active processes, and small jobs waste space. Used by early mainframe systems.

Placement with unequal partitions

- **One queue per partition:** each process waits for the smallest partition that can hold it. Least waste inside partitions, but a large partition may sit idle while a small one has a long queue.
- **Single queue:** a process takes the smallest *available* partition that holds it. More processes run, with more waste.

Example (partitions 3 to 20 MB; jobs 2, 1, 3, 2, 6, 2, 1 MB): per-partition queues run 2 jobs, 5 wait, 2 MB wasted; a single queue runs 6, 1 waits, 44 MB wasted.

When all partitions are full, swap out a process from the smallest partition that will hold the newcomer, preferring a **blocked** process over a ready one and weighing priority.

Dynamic partitioning

Partitions are created as needed, each **exactly** the size of its process: no internal fragmentation. As processes end, holes appear between partitions: **external fragmentation**. Example (64 MB, OS 4): P1 18 MB at 4, P2 12 at 22, P3 20 at 34, P4 8 at 54; end P2 and P4: 22 MB free in holes of 12 and 10, yet a 14 MB process does not fit.

Compaction: the OS shifts processes together so all free memory forms one block (here P3 moves from 34 to 22, copying 20 MB). It costs processor time and needs **dynamic relocation**, since moved processes run at new addresses.

Replacement: if every process in memory is blocked and there is no room even after compaction, the OS swaps a process out; which one is a replacement decision.

Placement algorithms

- **Best-fit:** the free block closest in size to the request.
- **First-fit:** scan from the beginning; the first block big enough.
- **Next-fit:** scan from where the last placement ended; the next block big enough.

Example: holes 5, 13, 9, 3, 6, 20 MB in order, last placement just before the 3 MB hole, request 8 MB. First-fit takes 13 (examines 2 holes), best-fit takes 9 (examines all 6, leaves 1), next-fit takes 20 (leaves 12). A 24 MB request fails under all three though 56 MB is free.

Typical behaviour (the standard verdict of classic simulation studies): **first-fit** is usually the simplest, the fastest and often the best, but it litters the front of memory with small blocks that every search must step past. **Next-fit** tends to break up the large free block at the end of memory, so compaction is needed more often than with first-fit. **Best-fit** is usually the worst: each placement leaves the smallest leftover possible, so memory fills with **slivers** too small for any request.

Exact rankings depend on the workload. In a race on one request sequence, best-fit may even need fewer compactions than the others, because it keeps the big end block whole and each compaction sweeps its slivers away; counting the slivers each algorithm leaves shows the effect behind its usual last place. What holds on every placement: best-fit must examine every hole (unless it finds an exact fit) and leaves the smallest leftover.

Buddy system

Block sizes are 2^K , $L \leq K \leq U$ (2^L the smallest block, 2^U usually all of memory). Start with one block of 2^U . For a request s : if $2^{U-1} < s \leq 2^U$, allocate the whole block; otherwise split it into two **buddies** of 2^{U-1} and repeat on one of them, until reaching the smallest block of at least s . A list of free blocks (holes) is kept for each size. When a block is freed and its buddy is free, they merge into one block of the next size, and merging continues upward.

Example (1024 KB, smallest 64 KB): A 90 KB splits $1024 \rightarrow 512 \rightarrow 256 \rightarrow 128$ and gets 128 KB at 0. B 200 KB gets 256 at 256; C 40 KB gets 64 at 128; D 120 KB gets 128 at 512. Later, freeing C merges $64 + 64$ into 128 at 128 and, if A has gone, $128 + 128$ into 256 at 0.

Internal fragmentation (100 KB takes a 128 KB block) but free space easily re-forms large blocks. Variants are used in real kernels, for example Linux's page allocator.

Relocation: base and bounds

With fixed partitions and one queue per partition, a process always returns to the same partition, so absolute addresses could be set at load time. With a single queue, swapping or compaction its location changes, so programs use:

- **Logical address:** a reference independent of where the data currently sits in memory.
- **Relative address:** a logical address relative to a known point, usually the program's start.
- **Physical (absolute) address:** an actual location in main memory.

When a process becomes Running, the **base register** gets its start address and the **bounds register** its end (the first address past its last byte). Each relative address is added to the base; the result is compared with the bounds; inside, the access proceeds; outside, an interrupt goes to the OS. This allows a process to be swapped back in anywhere (relocation) and keeps it out of others' memory (protection). Example: base 40,000, bounds 52,000: relative 1,200 $\rightarrow$ 41,200, allowed; 13,500 $\rightarrow$ 53,500, interrupt. Swapped back in at 76,000, relative 1,200 $\rightarrow$ 77,200.

Summary

Scheme	Waste	Strength	Weakness
Fixed	internal	simple, little OS overhead	caps active processes; small jobs waste space
Dynamic	external	no internal fragmentation	holes accumulate; compaction costs time
Buddy	internal (some external)	fast split and merge	a block can be nearly half wasted

Key terms

Partition

One contiguous region of main memory set aside to hold a single process.

Fixed partitioning

A scheme in which memory outside the OS is cut into partitions when the system starts; their number and sizes never change while it runs.

Overlay

A programming technique for a program too big for its memory space: the programmer splits it into modules that take turns occupying the same region, loading each when it is needed.

Internal fragmentation

Wasted memory inside an allocated partition or block, because the block handed out is bigger than what was loaded into it.

Dynamic partitioning

A scheme in which partitions are created as processes arrive, each one exactly as large as the process that needs it.

Hole

A stretch of free memory lying between allocated regions.

External fragmentation

Free memory broken into many separate holes between allocated regions, so that a request can fail even though the holes add up to enough space.

Compaction

The OS slides the processes in memory together so that they sit side by side and all the free memory forms one single block.

Dynamic relocation

Turning a program's addresses into physical addresses while it runs, on every reference, so the process can be moved to a new place in memory at any time.

Placement algorithm

The rule the OS uses to choose which free block of memory a new process is put into.

First-fit

A placement algorithm that scans memory from the beginning and takes the first free block big enough for the request.

Best-fit

A placement algorithm that takes the free block closest in size to the request, so the leftover is as small as possible.

Next-fit

A placement algorithm that starts scanning where the previous placement ended and takes the next free block big enough for the request.

Buddy system

An allocator whose blocks are always a power of two in size: a block is halved into two buddies until it just fits a request, and two free buddies are merged back into one block.

Buddies

The two equal halves produced by splitting one block of size 2^k ; only these two partners may later merge back together.

Relative address

A logical address expressed as a distance from a known point, usually the start of the program.

Base register

A processor register loaded with the starting physical address of the running process; it is added to every relative address.

Bounds register

A processor register loaded with the physical address where the running process's partition ends (the first address past its last byte); any translated address at or beyond it causes an interrupt to the OS.

Review questions with answers

1. Is each example internal or external fragmentation?

Internal fragmentation	External fragmentation
A 5 MB process in an 8 MB fixed partition A 100 KB request served with a 128 KB buddy block The unused tail of an allocated partition	Three separate 4 MB holes while a 10 MB process waits Free space that compaction could gather into one block Gaps left between dynamic partitions after processes end

Internal fragmentation is waste inside a block that has been handed out. External fragmentation is free memory between allocated blocks, scattered in pieces too small to use.

2. A system uses unequal fixed partitions and keeps a separate queue for each partition (each job waits for the smallest partition that can hold it). What is the main drawback?

- A. Each job wastes more space inside its partition than it would with a single queue.
- B. Jobs must be relocated every time they are loaded.
- C. A large partition can sit idle while jobs wait in the queue of a smaller partition.
- D. The OS cannot tell which partition a job should wait for.

Answer: C. A large partition can sit idle while jobs wait in the queue of a smaller partition.

Per-partition queues minimize internal fragmentation, but if many small jobs arrive they all queue for the small partitions while bigger partitions stay empty. A single queue would place them in the smallest free partition that fits.

3. A system uses unequal fixed partitions with a single queue. Every partition is occupied, all residents have the same priority, and a 6 MB process must be brought in. Which resident is the best one to swap out?

- A. A ready process in a 7 MB partition
- B. A blocked process in a 4 MB partition
- C. A blocked process in a 16 MB partition
- D. A blocked process in a 7 MB partition

Answer: D. A blocked process in a 7 MB partition

Only partitions big enough for the newcomer are candidates. Among them, prefer the smallest, and prefer a process that is blocked (it cannot use the processor now anyway), also weighing priority.

4. Which statements about compaction are true?

- A. It moves processes so that all free memory forms one block.
- B. It costs processor time, because the processes must be copied.
- C. It only works if processes can be relocated while they are in memory.
- D. It removes the internal fragmentation inside fixed partitions.
- E. It is done by the compiler when the program is built.

Answer: A, B, C

Compaction slides processes together so the holes join into one free block. Copying takes processor time, and each moved process now runs at a new address, so the hardware must translate addresses at run time. It fixes external fragmentation in dynamic partitioning, not waste inside fixed partitions, and it is done by the OS while the system runs.

5. Free holes, in address order, are 14 MB, 6 MB, 9 MB and 20 MB. The most recent placement ended between the 9 MB hole and the 20 MB hole. A process needs 8 MB. Match each placement algorithm to the hole it chooses.

First-fit	The 14 MB hole
Best-fit	The 9 MB hole
Next-fit	The 20 MB hole

First-fit scans from the start and the first hole of at least 8 MB is the 14 MB one. Best-fit picks the smallest hole that is big enough: 9 MB. Next-fit resumes after the 9 MB hole, so the first big-enough hole it meets is the 20 MB one.

6. Classic simulation studies of dynamic partitioning usually rank best-fit as the worst of the placement algorithms. Why?

- A. It always takes the largest hole, so the big free blocks are used up first.
- B. Each placement leaves the smallest leftover possible, so memory fills with slivers too small for any request.
- C. It gives each process a block bigger than it needs, which causes internal fragmentation.
- D. It resumes where the last placement ended, so it carves up the big free block at the end of memory.

Answer: B. Each placement leaves the smallest leftover possible, so memory fills with slivers too small for any request.

Best-fit wastes the least on each single placement, but those tiny leftovers add up until memory is littered with slivers no request can use. Classic studies therefore usually put first-fit (simplest, fastest, often best) ahead, next-fit a little behind, and best-fit last; exact rankings depend on the workload.

7. Compared with first-fit, next-fit tends to break up the large free block at the end of memory, so compaction is needed more often.

Answer: True

Next-fit keeps moving forward from its last placement, so it soon reaches the big block at the end and keeps slicing it. Large requests then find no room, and the OS must compact more often.

8. A buddy system manages 1024 KB with a smallest block size of 32 KB. How big is the block given to a request for 130 KB?

Answer: 256 KB

The block must be the smallest power of two that holds the request: $128 < 130 \leq 256$, so it gets a 256 KB block, and 126 KB of it is wasted inside.

9. A buddy system starts with one free 1024 KB block; its smallest block size is 64 KB. Put the steps that serve a 100 KB request in order.

Correct order:

1. Split the 1024 KB block into two 512 KB buddies
2. Split one 512 KB buddy into two 256 KB buddies
3. Split one 256 KB buddy into two 128 KB buddies
4. Hand one 128 KB block to the request (28 KB of it is wasted)

$64 < 100 \leq 128$, so the request needs a 128 KB block. The allocator halves blocks one level at a time until it reaches that size; the other halves go on the free lists for their sizes.

10. A running process has 30,000 in its base register and 42,000 in its bounds register. It uses relative address 9,500. Which physical address is accessed?

Answer: 39500

Physical address = base + relative address = $30,000 + 9,500 = 39,500$. That is below the bounds 42,000, so the access goes ahead.

11. A running process has 30,000 in its base register and 42,000 in its bounds register. It uses relative address 13,000. What happens?

- A. The result 43,000 is not below the bounds, so the hardware raises an interrupt and the OS takes over.
- B. The access goes to physical address 13,000, since 13,000 is below the bounds and needs no translation.
- C. The access goes ahead at 43,000, because the bounds are checked only once, when the program is loaded.
- D. The hardware wraps the address round to 31,000, so the access stays inside the process's own partition.

Answer: A. The result 43,000 is not below the bounds, so the hardware raises an interrupt and the OS takes over.

$30,000 + 13,000 = 43,000$, which lies beyond the end of the process's partition. The hardware detects this on the spot and interrupts, so the process cannot touch memory that is not its own.

12. Match each kind of address to its description.

Logical address	A reference that does not depend on where the data currently sits in memory
Relative address	A distance from a known point, usually the start of the program
Physical address	An actual location in main memory

Programs are written with logical addresses, most often relative ones (counted from the start of the program). The base register turns a relative address into a physical one each time it is used.

7.3 Paging

You should be able to

- Explain how paging divides main memory into frames and each process into pages of the same size, and why that removes external fragmentation while leaving only a little internal fragmentation.
- Load and swap out processes using a free-frame list and one page table per process, and read off where every page lives.
- Split a logical address into page number and offset and translate it into a physical address, both in binary and in decimal.
- Explain why the page size is always a power of two, and spot an address whose page number lies beyond the end of the page table.
- Compare paging with fixed and dynamic partitioning, and run paging against dynamic partitioning on the same workload.

The idea of paging

Dynamic partitioning gives each process one unbroken (contiguous) block of exactly its size. As processes come and go, free memory ends up in scattered holes, and a process can be refused even though the total free space is enough. **Paging** avoids this. Main memory is divided into many small, equal, fixed-size chunks called **frames** (page frames). Each process is divided into chunks of exactly the same size called **pages**. Any page can be loaded into any free frame, so the pages of one process may be scattered anywhere in memory. In simple paging all pages of a process are in memory while it runs.

Example: 12 frames of 1 KB with 5 KB free in holes of 2, 1 and 2 KB. A 5 KB process cannot get one contiguous block, but under paging its 5 pages simply take the 5 free frames.

Pages, frames and the only fragmentation

- The **page size** (= frame size) is set by the processor's design and is always a power of two; simple paging uses one size for the whole machine. 4 KB (4,096 bytes) is the most common today, some systems use 16 KB, and modern processors can also map a few much larger pages.
- A process of S bytes needs $\lceil S \div \text{page size} \rceil$ pages. Every page is full except, usually, the last one.
- **Internal fragmentation** = pages $\times$ page size – S : only the unused end of the last page: under one page per process, about half a page on average.
- **No external fragmentation**: a free frame is never too small, because every page needs exactly one frame. So paging never needs **compaction**.

Example: $S = 5,300$ bytes, 1,024-byte pages: $\lceil 5.18 \rceil = 6$ pages, $6 \times 1,024 = 6,144$ bytes allocated, 844 bytes wasted at the end of page 5. Smaller pages waste less but give each process more pages and a longer page table.

The OS's records: page tables and the free-frame list

The OS keeps one **page table** per process: entry i holds the number of the frame holding page i , so the table has one entry per page. It holds frame **numbers**, not the pages themselves. The OS also keeps one **free-frame list** for the whole machine. Loading a process takes frames from the list and fills in its page

table; swapping it out returns its frames to the list and discards its table (when it returns it will usually get different frames and a new table).

Example with 16 frames of 1 KB: A (3,900 B, 4 pages) gets frames 0–3; B (2,600 B, 3 pages) gets 4–6; C (4,500 B, 5 pages) gets 7–11. D (5,900 B, 6 pages) must wait: only 4 frames are free. Swapping B out frees 4, 5, 6; now 7 frames are free and D gets frames 4, 5, 6, 12, 13, 14, not side by side. Internal fragmentation: $A\ 196 + C\ 620 + D\ 244 = 1,060$ bytes; external fragmentation: none. The only question when loading is how many frames are free, never where they are.

Logical addresses and translation

Programs use **relative addresses** counted from their own byte 0. Under paging the processor reads each as a pair (**page number, offset**): which page, and how far into it. **Address translation** turns (page, offset) into (frame, offset): the frame number replaces the page number, and the offset stays the same because a page is copied into its frame whole. The lookup is done by the processor's memory management unit on every access; at each process switch the OS loads the address and length of the new process's page table into processor registers (many modern processors keep no length and mark unused entries invalid instead).

Why the page size is a power of two

With n -bit addresses and pages of 2^m bytes, the rightmost **m bits are the offset** and the leftmost **$n - m$ bits are the page number** (so a process can have up to 2^{n-m} pages). The physical address is the frame number's bits written in front of the offset bits, which equals **$\text{frame} \times 2^m + \text{offset}$** . Dividing by the page size costs nothing in binary: quotient and remainder are already the two groups of bits, so translation is pure wiring, with no divider, multiplier or even an adder.

Worked example. 16-bit addresses, 1,024-byte pages ($m = 10$): 6 page bits, 10 offset bits, up to 64 pages. Relative address 3,000 = 000010 1110111000, so page 2, offset 952 ($3,000 = 2 \times 1,024 + 952$). If page 2 is in frame 9 (001001), the physical address is 001001 1110111000 = 10,168 = $9 \times 1,024 + 952$. With 1,000-byte pages, finding page 3, offset 0 for address 3,000 would need a real division on every access: a power of two is to binary hardware what 1,000 is to us in decimal. A common mistake is to add the frame number to the address; it replaces the page number.

Protection: the page-table length check

Before looking anything up, the hardware compares the page number with the length of the page table. A page number that is too big (equal to or larger than the number of entries) makes the hardware raise an interrupt (a trap) instead of touching memory, and the OS normally ends the process. Example: 6 entries (pages 0–5), 512-byte pages, address 3,100 = $6 \times 512 + 28$, so page 6: trap. The check is on whole pages only: a byte in the unused end of the last page passes it.

Translating by hand (decimal recipe)

1. page = address $\div$ page size (whole part); offset = the remainder.
2. If page $\geq$ the number of page-table entries: trap.
3. physical = frame[page] $\times$ page size + offset.

Example: page size 512, page table 0→7, 1→19, 2→4, 3→26, 4→11, address 1,300. $1,300 \div 512 = 2$ remainder 276, so page 2, offset 276; entry 2 holds frame 4; physical = $4 \times 512 + 276 = 2,324$.

Paging versus partitioning

Same workload in 16 KB: A, B, C arrive, A is swapped out, D (5,900 B) arrives, then E (1,800 B). **Dynamic partitioning (first-fit)**: after A leaves, 9,284 bytes are free but in holes of 3,900 and 5,384 bytes, so D does not fit; the OS must compact, copying 7,100 bytes (B and C), before D loads. **Paging with 1 KB pages**: D takes 6 scattered frames at once, nothing is ever copied, and at the end 1,584 bytes are wasted inside last pages. With 2 KB pages the waste inside last pages grows to 3,384 bytes and E (needing 1 frame) has to wait: larger pages waste more.

	Fixed partitioning	Dynamic partitioning	Paging
Unit handed out	One whole partition, fixed at start-up	One block of exactly the process's size	Many equal frames
Process contiguous?	Yes	Yes	No
Internal fragmentation	Yes, rest of the partition	None	Small: part of the last page
External fragmentation	None	Yes, holes multiply	None
Compaction	Never	From time to time	Never
Hardware per access	Base register + bounds	Base register + bounds	Page-table lookup + length check
OS bookkeeping	Partition table	List of holes	Page table per process + free-frame list

Three consequences: paging is **transparent** (programmers and compilers see one continuous range of relative addresses; only the OS and hardware know about frames); translation is **simple to do in hardware**; and paging is like **fixed partitioning with very small, equal partitions**, where one process may occupy several partitions that need not be next to each other.

Looking ahead

Reading the page table on every access would double the memory traffic, so processors cache recent translations, and large programs need page tables organised in other ways. Once a page-table entry can say "this page is not in memory", a process can run with only some of its pages loaded: virtual memory (Chapter 8).

Key terms

Paging

A memory-management scheme that cuts main memory into many small, equal frames and each process into pages of the same size, so any page can be loaded into any free frame and a process no

longer needs one unbroken stretch of memory. In simple paging every page of a process is in memory while it runs.

Frame (page frame)

One of the many equal, fixed-size chunks main memory is divided into. Each frame holds exactly one page.

Page

One of the equal, fixed-size chunks a process is divided into. A page is exactly the size of a frame, so it fits in any frame.

Page size

The number of bytes in one page (and so in one frame). It is always a power of two, such as 1,024 or 4,096 bytes.

Page table

A table the OS keeps for each process: entry *i* holds the number of the frame that currently holds page *i* of that process. The processor uses it to translate every address the process issues.

Free-frame list

The OS's list of frames that hold no page right now. Loading a process takes frames from it; removing a process gives its frames back.

Page number

The left part of a logical address under paging: which page of the process the byte lies on. It is the index into the page table.

Offset

The right part of a logical address under paging: how many bytes past the start of its page the byte sits. Translation keeps it unchanged, because a page lands whole in a frame.

Relative address

An address counted from the start of the program, so the program's first byte is address 0. With power-of-two pages its bits are exactly the page number followed by the offset.

Address translation

Turning the logical address a program uses into the physical address sent to the memory chips. Under paging the hardware swaps the page number for a frame number and keeps the offset.

Internal fragmentation

Wasted space inside an allocated piece of memory, because the piece is bigger than what it holds. Under paging it is only the unused tail of each process's last page.

External fragmentation

Free memory split into separate holes between allocated regions, each too small for the next request, even though together they would be big enough.

Compaction

Moving processes in memory so they sit next to each other and all the free space joins into one block. It cures external fragmentation but costs processor time spent copying.

Contiguous allocation

Placing a whole process in one unbroken run of addresses in main memory, as fixed and dynamic partitioning both do. Paging drops this requirement.

Dynamic partitioning

Giving each process one contiguous region of exactly the size it needs, created when the process is loaded. The holes left behind as processes leave cause external fragmentation.

Fixed partitioning

Dividing memory, once at system start, into partitions whose boundaries never change; each process takes one whole partition, wasting whatever it does not fill.

Review questions with answers

1. Under paging, what does entry i of a process's page table hold?

- A. A copy of the bytes stored in page i of the process
- B. The number of bytes the process actually uses in page i
- C. The number of the next free frame after page i
- D. The number of the frame that currently holds page i

Answer: D. The number of the frame that currently holds page i

A page table is a map from page numbers to frame numbers, one entry per page. The hardware indexes it with the page number to find the frame.

2. Pages are 1,024 bytes. A program uses relative address 5,000. What is the offset part of this address?

Answer: 904 bytes

$5,000 \div 1,024 = 4$ remainder 904: the byte is on page 4, 904 bytes from the start of that page. In binary, the offset is simply the lowest 10 bits.

3. Pages are 1,024 bytes, and page 4 of a process is held in frame 11. Which physical address does the process's relative address 5,000 map to?

Answer: 12168

5,000 is page 4, offset 904. The frame number replaces the page number and the offset is kept: $11 \times 1,024 + 904 = 12,168$.

4. A machine uses 20-bit logical addresses and 4,096-byte pages. How many bits of each address form the page number?

Answer: 8 bits

$4,096 = 2^{12}$, so 12 bits are the offset and the remaining $20 - 12 = 8$ bits are the page number, allowing up to $2^8 = 256$ pages per process.

5. A 9,000-byte process is loaded under paging with 2,048-byte pages. How many bytes of its frames are wasted (internal fragmentation)?

Answer: 1240 bytes

It needs $\lceil 9,000 \div 2,048 \rceil = 5$ pages, so it is given $5 \times 2,048 = 10,240$ bytes. $10,240 - 9,000 = 1,240$ bytes sit unused at the end of the last page.

6. A programmer writing a program for a paged system must know the page size and split the program into pages by hand.

Answer: False

Paging is transparent: the program sees one continuous range of relative addresses starting at 0. The OS splits the process into pages and the hardware translates every address.

7. A process has a page table with 6 entries and the page size is 512 bytes. The process issues relative address 3,100. What happens?

- A. The access goes to frame 6, offset 28, because the page number is used directly as the frame number
- B. The hardware raises an interrupt (a trap), because the address lies on page 6 and the table has no entry 6
- C. The access goes to the frame in entry 5, offset 540, since entry 5 is the last page the table covers
- D. The address is quietly cut down to 3,071, the last valid byte, so the access stays inside the process

Answer: B. The hardware raises an interrupt (a trap), because the address lies on page 6 and the table has no entry 6

$3,100 \div 512 = 6$ remainder 28, so the byte is on page 6. A table of 6 entries covers pages 0 to 5, so the hardware traps before touching memory, which also stops the process from reaching anyone else's frames.

8. Why is the page size always a power of two?

- A. So every process gets a power-of-two number of pages, which lets the OS size each page table exactly
- B. So a page can be split evenly into smaller frames, letting its pieces sit in different places in memory
- C. So the page number and offset are just the high and low bits of the address, making translation simple bit wiring
- D. So that no process ever has internal fragmentation, since every program size divides evenly into pages

Answer: C. So the page number and offset are just the high and low bits of the address, making translation simple bit wiring

With 2^m -byte pages, dividing by the page size is free: the quotient is the high bits and the remainder the low m bits. The physical address is the frame bits written in front of the offset bits.

9. Which statements about simple paging are true?

- A. The OS keeps one page table for each process in memory.
- B. The OS keeps a list of free frames.
- C. The only fragmentation is the unused end of each process's last page.
- D. The pages of one process may sit in frames that are far apart.
- E. A process must fit into one free hole of memory.
- F. Each page may be a different size, chosen to fit the process.

Answer: A, B, C, D

Paging uses equal pages, tracks free frames in a list and maps each process's pages through its own table, so pages can be scattered and only the tail of the last page is wasted. Needing one big hole is the problem of contiguous allocation.

10. Put the steps the hardware follows to translate a logical address under paging in order.

Correct order:

1. Split the logical address into page number and offset
2. Compare the page number with the page-table length
3. Read the frame number from the page-table entry
4. Put the frame number in front of the offset
5. Send the resulting physical address to memory

Split, check, look up, assemble, access. If the check fails, the hardware traps instead of continuing.

11. Match each paging term to its meaning.

Frame	A fixed-size chunk of main memory
Page	A same-size chunk of a process
Page table	Maps one process's page numbers to frame numbers
Free-frame list	The frames ready to receive pages
Offset	Where a byte sits inside its page

Frames are slots in memory, pages are pieces of a process, the page table connects the two, and the free-frame list tells the OS where it can put the next page.

12. Sort each statement under the scheme it describes.

Dynamic partitioning	Paging
Needs compaction from time to time Each process occupies one contiguous block Free memory can be useless because it is split into small holes	Wastes memory only at the end of each process's last page A process can be spread over scattered frames Keeps a table per process mapping pieces to locations

Dynamic partitioning gives each process one exact-size block, so holes appear between blocks and compaction is needed. Paging hands out equal frames anywhere, so nothing is ever too small, at the cost of a page table per process.

7.4 Segmentation

You should be able to

- Explain what a segment is, why segments follow a program's logical parts, and why segmentation is visible to the programmer.
- Split a logical address into a segment number and an offset, and state the maximum segment length for a given bit split.
- Translate a logical address with a segment table: look up base and length, check the offset, then add or trap.
- Describe how segments are placed in memory holes, and why segmentation has external but no internal fragmentation.
- Use per-segment permissions and shared segments to explain protection and sharing, and compare the four memory techniques of this chapter.

Segmentation: cutting a program along its logical parts

Paging cuts a program into equal pieces wherever the boundaries happen to fall. **Segmentation** divides a program and its data along its **logical parts** instead: the main code, each library routine, each data table, the stack. Each part becomes a **segment**, a piece of whatever length it needs, up to a **maximum segment**

length. Example used throughout: a music player P with four segments: main code 2,600 bytes, a sort routine 900 bytes, a song table 3,000 bytes and a stack 1,100 bytes (7,600 bytes in all).

Every segment has its own addresses starting at offset 0. If the main code grows, the song table's addresses do not move. In one flat block they would: the song table would start at byte 3,500, so its byte 500 would be address 4,000, and 100 more bytes of code would push it to 4,100.

Segmentation is usually visible to the programmer. The programmer or compiler decides which code and data go in which segment, and modules can be compiled separately. Paging, by contrast, is invisible. The one inconvenience: no segment may exceed the maximum length, so a large table may have to be split across segments.

Two-part logical addresses

A segmented logical address is (**segment number, offset**). In binary, the leftmost n bits are the segment number and the rightmost m bits are the offset. A process can have up to 2^n segments, each at most 2^m bytes long. The split is fixed by the processor design.

Split of 16 bits ($n + m$)	Most segments	Maximum segment length
2 + 14	4	16,384 bytes
4 + 12 (the running example)	16	4,096 bytes
6 + 10	64	1,024 bytes

With a 4 + 12 split, each hex digit is exactly 4 bits, so the first hex digit is the segment number and the last three are the offset. Example: 0x21F4 = 0010 000111110100 is segment 2, offset 0x1F4 = 500. Likewise 0x3A10 is segment 3, offset 0xA10 = 2,576.

Placing segments in memory

Because segments have different sizes, placing them resembles **dynamic partitioning**: each segment needs a hole (free block) at least its size. The difference is that one program now occupies **several** partitions, and they need not be next to each other. The OS keeps a **free-block list** of holes and uses a placement rule such as first-fit. In *simple* segmentation every segment of a process must be in memory before it runs; if one segment cannot be placed, the load fails (the OS might compact memory or wait).

- **No internal fragmentation:** each segment gets exactly its size.
- **External fragmentation:** holes build up between segments. It is less severe than with dynamic partitioning, because smaller pieces fit into smaller holes.

Worked example: 32 KB of memory holds the OS and four processes, leaving holes of 5, 3, 6, 2 and 4 KB (20 KB free). An 11 KB program loaded as one block fails, since the largest hole is 6 KB. Loaded as segments of 4, 2, 3 and 2 KB with first-fit, it fits (4 KB into the 5 KB hole, 2 KB into the 3 KB hole, 3 KB and then 2 KB into the 6 KB hole), leaving 9 KB free in five holes. Four 4 KB segments (16 KB) fail at the fourth: 8 KB is free, but no hole is 4 KB.

Translating an address with a segment table

Each process has a **segment table**. Entry i holds segment i 's **base** (its starting physical address), its **length**, and usually its protection bits. The processor keeps the location of the running process's table in a register, reloaded at every process switch. There is no simple bit splice as in paging, because a segment can start at any address. The hardware does this on every reference:

1. Take the segment number from the leftmost n bits and the offset from the rightmost m bits.
2. Use the segment number to index the segment table. If there is no such entry, trap.
3. Compare the offset with the length. If $\text{offset} \geq \text{length}$, the address is invalid: trap.
4. Otherwise, physical address = base + offset.

Seg (P)	Holds	Base	Length
0	main code	21,000	2,600
1	sort routine	9,400	900
2	song table	30,500	3,000
3	stack	14,200	1,100

- (2, 500): $500 < 3,000$, so physical = $30,500 + 500 = \mathbf{31,000}$.
- (1, 1,200): $1,200 \geq 900$, trap.
- (3, 1,100): offsets run 0 to 1,099, so 1,100 is outside: trap. (The test is $\text{offset} < \text{length}$, not $\leq$.)
- (6, 100): P has only segments 0 to 3, trap.

A failed check is a **segmentation fault**. On UNIX-like systems the OS normally ends the process with the signal SIGSEGV ("segmentation violation"); today the OS sends the same signal for bad references that paging hardware catches.

Protection and sharing

Each segment table entry can carry **protection bits** such as read (R), write (W) and execute (X), checked by the hardware on every reference along with the length. Because a segment is one whole logical unit, one setting fits all of it: code R-X (a stray write cannot change instructions), a dictionary R-- (read-only), data and stack RW- (not executable, which blocks running injected data as code).

A **shared segment** is simply an entry that appears in several processes' tables, pointing at one physical copy. Two editors sharing 3,800 bytes of code and a 2,500-byte dictionary need 13,100 bytes instead of 19,400, saving 6,300. A process can reach only what its own table points at, so another process's private segments are unreachable.

Paging versus segmentation (program P, 7,600 bytes)

	Simple paging, 1,024-byte pages	Simple segmentation
Pieces	8 equal pages	4 segments of 2,600, 900, 3,000 and 1,100 bytes
Internal fragmentation	$8,192 - 7,600 = 592$ bytes in the last page	none
External fragmentation	none: any free frame will do	yes: each segment needs a big enough hole
Programmer's view	one flat address range (invisible)	segments (visible)
Translation	frame number placed in front of the offset bits	look up base and length, check, add

Pages can also mix parts: with 1,024-byte pages, 3 of P's 8 pages hold bytes from two parts, so no single protection setting fits those pages.

The four techniques so far

Technique	How it works	Strengths	Weaknesses
Fixed partitioning	Partitions of set sizes are made at start-up; each process takes one at least its size.	Simple; little OS overhead.	Internal fragmentation; the number of partitions caps the number of active processes.
Dynamic partitioning	Partitions are made on demand, exactly the size of each process.	No internal fragmentation; better use of memory.	External fragmentation; compaction costs processor time.
Simple paging	Equal frames and pages; all pages loaded, each into any free frame.	No external fragmentation.	A small amount of internal fragmentation (last page).
Simple segmentation	Variable-size segments; all loaded into partitions that need not be contiguous.	No internal fragmentation; better utilization and lower overhead than dynamic partitioning.	External fragmentation.

"Simple" means the whole process must be in memory. Chapter 8 removes that rule (virtual memory) and shows how paging and segmentation can be combined. Most current systems rely mainly on paging; on 64-bit x86 processors segmentation is almost switched off.

Key terms

Segmentation

A memory-management scheme that divides a program and its data into segments of different lengths, each loaded into its own block of memory; the blocks need not be next to each other.

Segment

A variable-length piece of a program that matches one logical part of it, such as the main code, a library routine, a data table or the stack. Each segment has its own addresses starting at 0.

Segment number

The part of a segmented logical address that says which segment is meant: the leftmost n bits. It is an index into the process's segment table.

Offset (within a segment)

How many bytes past the start of its segment a location lies: the rightmost m bits of the logical address. Valid offsets run from 0 to the segment's length minus 1.

Maximum segment length

The largest size any one segment may have. With an m -bit offset field it is 2^m bytes; a larger object has to be split across segments.

Segment table

A per-process table, kept by the OS and used by the processor, with one entry per segment holding the segment's base and length (and usually its protection bits).

Segment base (base)

The physical address where a segment starts in main memory; it is stored in the segment's table entry.

Segment length (limit)

The size of a segment in bytes, stored in its table entry so the hardware can reject any offset that falls outside the segment.

Segmentation fault (segfault)

The trap to the OS that happens when a logical address fails the hardware check: no such segment, an offset at or beyond the segment's length, or an access its protection bits forbid.

Free-block list

The OS's list of the holes (free blocks) in main memory, with their start addresses and sizes, consulted whenever segments must be placed.

Hole

A free block of main memory lying between allocated areas.

External fragmentation

Free memory that is split into many holes, each too small for the next request, even though their total would be enough.

Internal fragmentation

Space wasted inside an allocated piece because the piece is bigger than what it holds.

Dynamic partitioning

Placing each whole process in one block of memory created at exactly the process's size.

Paging

Dividing memory into equal frames and each process into pages of the same size, so any page can go in any free frame.

Protection bits (access rights)

Flags in a segment table entry such as read, write and execute that say which kinds of access to that segment are allowed; the hardware checks them on every reference.

Shared segment

A segment whose entry appears in the segment tables of several processes, so one physical copy serves all of them.

Review questions with answers

1. Under segmentation, a logical address consists of:

- A. A segment number and an offset within that segment
- B. A page number and an offset within that page
- C. A base address and a length
- D. A frame number and a segment number

Answer: A. A segment number and an offset within that segment

A segmented address names the segment (an index into the process's segment table) and how many bytes into that segment the location lies.

2. A machine uses 16-bit logical addresses: a 4-bit segment number followed by a 12-bit offset. What is the largest a segment can be, in bytes?

Answer: 4096 bytes

A 12-bit offset counts from 0 to 4,095, so a segment holds at most $2^{12} = 4,096$ bytes. (The 4-bit segment number allows up to 16 segments.)

3. Entry 3 of a process's segment table holds base 7,200 and length 2,000. What physical address does the logical address (segment 3, offset 1,350) map to?

Answer: 8550

First the check: $1,350 < 2,000$, so the offset is inside the segment. Then the addition: $7,200 + 1,350 = 8,550$.

4. Segment 1 of a process has length 600. The process uses the logical address (segment 1, offset 600). What happens?

- A. It maps to base + 600, which is the segment's last byte
- B. It wraps around and reaches offset 0 of the same segment
- C. The hardware traps: offset 600 is outside the segment
- D. The OS quietly grows the segment by one byte to fit it

Answer: C. The hardware traps: offset 600 is outside the segment

Valid offsets run from 0 to length – 1. The test is offset < length, and $600 < 600$ is false, so the reference traps to the OS.

5. Put the steps of translating a segmented logical address in order.

Correct order:

1. Split the address: the leftmost bits give the segment number, the rest the offset
2. Use the segment number to index the process's segment table
3. Read that segment's base and length from the entry
4. Compare the offset with the length, trapping if the offset is too large
5. Add the base to the offset to form the physical address

Split, look up, read the entry, check, add. The check must come before the addition so that an out-of-range reference never reaches memory.

6. Does each property describe simple paging, simple segmentation, or both?

Paging	Segmentation	Both
Every piece has the same size A little internal fragmentation	Usually visible to the programmer External fragmentation Translation adds a base and checks a length Pieces follow the program's logical parts	A process's pieces need not sit next to each other A per-process table maps each piece to memory

Both schemes scatter a process through memory using a per-process table. Paging uses equal pieces (invisible, small internal waste); segmentation uses logical, variable-size pieces (visible, external fragmentation, base-plus-offset translation).

7. Simple segmentation suffers from internal fragmentation, because each segment is rounded up to a fixed size.

Answer: False

Segments are not rounded up: each gets exactly the space it needs, so nothing is wasted inside it. The waste segmentation does suffer is external: holes left between segments.

8. Which statements about protection and sharing under segmentation are true?

- A. Each segment table entry can carry its own access rights, such as read-only for code
- B. A segment is shared by listing it in the segment tables of several processes
- C. To share a segment, each process must keep its own copy of it in memory
- D. A process cannot reach memory that none of its segment table entries covers
- E. The OS checks the protection bits in software on each memory reference

Answer: A, B, D

Rights are stored per entry and checked by the hardware on every reference, together with the length. Sharing means two tables point at one physical copy, and a process can only reach memory through its own table.

9. Match each memory-management technique to its main weakness.

Fixed partitioning	Internal fragmentation and a fixed cap on active processes
Dynamic partitioning	External fragmentation that calls for costly compaction
Simple paging	A little internal fragmentation in each process's last page
Simple segmentation	External fragmentation, though less than with whole-process partitions

Fixed partitions waste space inside each partition; dynamic partitions leave holes between processes; paging wastes only the end of the last page; segmentation leaves holes too, but its smaller pieces fit them more easily.

10. Why does simple segmentation usually suffer less external fragmentation than dynamic partitioning?

- A. It never leaves holes in memory
- B. Its pieces are smaller, so they fit into smaller holes
- C. The OS compacts memory after every load
- D. Every segment is the same size, so any hole can be reused

Answer: B. Its pieces are smaller, so they fit into smaller holes

A whole process needs one large hole; its segments need several smaller ones, which are far easier to find among scattered holes.

11. A system uses a 4-bit segment number and a 12-bit offset. What is the offset, in decimal, in the logical address 0x3A10?

Answer: 2576

The first hex digit, 3, is the segment number. The other three, 0xA10, are the offset: $10 \times 256 + 1 \times 16 + 0 = 2,576$.

12. Segmentation is usually visible to the programmer, who (or whose compiler) decides what goes in each segment.

Answer: True

Unlike paging, which is invisible, segments match a program's modules and data structures. Programmers and compilers organize code and data into them, and must keep each one under the maximum segment size.

7A Loading and Linking

You should be able to

- Trace a program from source files through object modules, the linker, the load module and the loader to a process image in main memory.
- Compare absolute loading, relocatable loading (with a relocation dictionary) and dynamic run-time loading, and say which survives being swapped back in at a new address.
- Name the four times at which addresses can be bound and who does the conversion at each.
- Compute where each module starts in a load module and rewrite internal and external references.
- Contrast static linking with load-time and run-time dynamic linking, including shared libraries.

From source files to a running process

Creating a process starts with putting a program into main memory and building its process image. Real programs are made of several separately compiled or assembled modules, so they must also be **linked**: references between the modules, and references to library routines, must be turned into addresses. Library routines are either copied into the program or left as shared code that the OS supplies when the program loads or runs.

The pipeline: source files → compiler or assembler → **object modules** → **linker** → **load module** → **loader** → process image in main memory. A shared (dynamic) library can join at load time or at run time.

- **Object module:** the translation of one source file. Its code and data are numbered from its own word 0; names it needs from elsewhere (external references) are left as symbols, listed with the names it

defines.

- **Load module:** one file made by the linker from all the object modules, laid end to end.
- **Loader:** the part of the OS that copies the load module into memory and makes its addresses work there.

Example: main.o (60 words) calls average (in stats.o, 40 words) and print (in a library). The linker puts main.o at 0–59 and stats.o at 60–99, so `CALL average` becomes `CALL 60`. Loaded at 3000, that call reaches 3060.

Absolute loading

Every address in the load module is already a real memory address, so the loader only copies it, always to the same place. The binding happens either at **programming time** (the programmer writes real addresses, e.g. `LOAD 1006`) or at **compile or assembly time** (the programmer writes symbolic addresses such as `LOAD count` and the assembler converts them, given the start address).

Drawbacks: the module runs at one place only, and any change that shifts code moves every later address. With hand-written addresses, inserting one instruction at word 3 of a 9-word loop left 4 of its 5 address fields pointing at the wrong word; assembling from names recomputes them all, but the module is still tied to one address.

Relocatable loading

The assembler or compiler produces **relative addresses** (counted from the module's start, word 0) plus a **relocation dictionary**: a list of every word that contains an address. When the loader places the module at address L, it adds L to each listed word and copies all other words unchanged. The dictionary is needed because a loader cannot tell an address from a plain number by looking at it (word 0 holding `LOAD 6` and word 8 holding the number 6 look alike).

Example: loaded at L = 2400, `LOAD 6` becomes `LOAD 2406` and `JNEG 0` becomes `JNEG 2400`; the data 0, 2, 6 stay as they are.

Drawback: once patched, the image in memory holds real addresses for L. If the process is swapped out, it must be brought back to the same place.

Dynamic run-time loading

The load module keeps its relative addresses even after it is in memory. The real address is computed only when an instruction actually runs: the processor hardware adds the **base register** (which the OS sets to where the process currently sits) to each address. Example: base register 2400, `LOAD 6` reaches 2406. The process can be swapped out and brought back **anywhere**; the OS just sets the base register to the new start. The cost is an addition on every memory reference, done by hardware so it is fast.

Loaded at 1000, swapped out, back at 2400	First run at 1000	Run after coming back at 2400
Absolute (built for 1000)	works	breaks: still uses 1000–1008
Relocatable	works	breaks: words were patched for 1000
Dynamic run-time	works	works: base register = 2400

Address binding times

Binding time	What happens	Can the program move later?
Programming time	The programmer writes actual physical addresses.	No
Compile or assembly time	The compiler or assembler converts symbolic references into physical addresses.	No
Load time	The loader converts relative addresses to physical ones while loading.	No, once loaded
Run time	Relative addresses are kept; the processor hardware converts them on each reference.	Yes

Linking

The linker takes a set of object modules and produces one load module that is their **contiguous concatenation**. Inside an object module, references to its own code and data are relative to that module's start; references to other modules (**external references**) appear only as symbols. If modules A, B and C have lengths L, M and N, then in the load module A starts at 0, **B at L** and **C at L + M**. A reference to relative address x inside B becomes **L + x**, and a call to B becomes a jump to **L**.

Worked example: A = 300, B = 250, C = 120 words, in the order A, B, C. B starts at 300, C at 300 + 250 = 550, and the load module is 670 words. A call to C becomes `CALL 550`; `LOAD 210` inside B becomes `LOAD 510`; `JUMP 15` inside C becomes `JUMP 565`. In the order B, C, A, C would start at 250 and A at 370.

Static and dynamic linking

A **linkage editor** performs **static linking**: it produces a single relocatable load module containing everything, library routines included. **Dynamic linking** defers the linking of some external modules until after the load module is created, so the load module still holds unresolved references to them.

Load-time dynamic linking: the load module is read into memory, and every reference to an external (target) module makes the loader find that module, load it and link it. Benefits: (1) when a target module, such as an OS utility, is updated, programs use the new version automatically without being relinked; (2) one copy of the target code, kept in a dynamic link library, is shared by many programs; (3) independent developers can extend the system with new modules. A program that is already running keeps the version it linked; it picks up an update at its next load.

Example: three apps of 300, 250 and 200 KB all use a 600 KB library. Statically linked, the three running apps need $750 + 3 \times 600 = 2,550$ KB of memory; dynamically linked to one shared copy, $750 + 600 = 1,350$ KB.

Run-time dynamic linking: some linking is postponed until execution. When the program calls a module that is not yet present, the OS finds it, loads it and links it, and then the call proceeds. Benefit: memory is used only for modules the program actually calls; the price is a pause on the first call to each one.

Example: an 800 KB program with optional modules of 300, 200, 500 and 150 KB that only ever prints holds $800 + 200 = 1,000$ KB with run-time linking, against 1,950 KB with load-time linking.

Today's names: shared libraries are .so files on Linux and other UNIX-like systems and dynamic link libraries (DLLs, .dll files) on Windows. Programs can also request a library explicitly while running (for example dlopen on Linux, LoadLibrary on Windows), and many systems postpone looking up each library function until its first call.

Do not mix up the look-alike names. *Dynamic run-time loading* is about addresses: relative addresses stay in memory and the base register is added on every reference. *Run-time dynamic linking* is about modules: a module that is not yet present is found, loaded and linked on its first call.

Key terms

Object module

The output of compiling or assembling one source file: machine code and data numbered from 0, plus a table of the names it defines and the outside names it still needs.

Load module

The single file a linker produces from a set of object modules, laid end to end; it is what the loader copies into memory.

Linker

A program that combines object modules into one load module, working out where each module starts and replacing references between modules with addresses.

Loader

The part of the operating system that copies a load module into main memory, fixes up its addresses as needed, and so builds the process image.

Symbolic address

A name such as count or average that a programmer uses in place of a number; a translator or linker later replaces it with an address.

Relative address

An address counted from the start of a module or program (its first word is 0), so it says how far into the program something is, not where it sits in memory.

Address binding

The moment an address in a program is fixed to a real location in main memory. It can happen when the code is written, when it is compiled, when it is loaded, or each time it runs.

Absolute loading

Loading in which every address in the load module is already a real memory address, so the module must always be placed at the one location it was built for.

Relocatable loading

Loading in which the load module holds relative addresses and the loader adds the load address to each of them as it copies the program into memory.

Relocation dictionary

A list, made by the assembler or compiler and carried in the module, of every word that contains an address, so the loader knows exactly which words to adjust.

Dynamic run-time loading

Loading in which the program keeps its relative addresses even in memory; the processor hardware adds the base register to each address as the instruction runs.

External reference

A use, in one module, of a name defined in a different module, such as a call to a routine in another file. It stays a symbol until the modules are linked.

Linkage editor

A linker that does all the linking in advance, producing one relocatable load module that already contains every module it needs.

Static linking

Linking everything, including library routines, into the load module before it runs, so the program carries its own copy of each routine.

Dynamic linking

Deferring the linking of some external modules until after the load module is made, so the load module keeps some unresolved references to be filled in at load time or run time.

Load-time dynamic linking

Dynamic linking done while the program is being loaded: for each reference to an outside module, the loader finds that module, loads it and links it.

Run-time dynamic linking

Dynamic linking postponed until the program actually calls a missing module: only then does the OS find it, load it and link it.

Shared library

A file of routines that the OS links into programs dynamically; one copy in memory can serve many processes. Called a .so file on Linux and a dynamic link library (DLL, .dll) on Windows.

Dynamic link library (DLL)

Windows name for a shared library: a file of routines linked into programs at load time or run time, with one copy in memory shared by many processes.

Review questions with answers

1. Put the stages that turn a program into a running process in order.

Correct order:

1. A programmer writes the source files
2. A compiler or assembler turns each file into an object module
3. A linker combines the object modules into one load module
4. A loader copies the load module into main memory
5. The process image is in memory, ready to run

Each source file is translated separately into an object module; the linker joins the modules into a load module; the loader places that module in memory, which builds the process image.

2. Every address in a load module is a real main-memory address that is correct only if the module starts at address 4000. Which kind of loading does this module need?

- A. Relocatable loading
- B. Absolute loading
- C. Dynamic run-time loading
- D. Run-time dynamic linking

Answer: B. Absolute loading

A module whose addresses are already real can only be copied to the one place it was built for, which is absolute loading.

3. A relocatable load module is placed in main memory starting at address 7300. One word listed in its relocation dictionary holds the relative address 148. What value does the loader store in that word?

Answer: 7448

The loader adds the load address to every word on the relocation list: $148 + 7300 = 7448$.

4. A relocation dictionary lists every word of a module, and the loader adds the load address to all of them.

Answer: False

It lists only the words that contain addresses. Adding the load address to a plain number, such as a loop limit, would silently change it.

5. A process was loaded with relocatable loading at address 2000 and has since been swapped out. Only the region starting at 5000 is free now. What happens if its saved image is copied there unchanged?

- A. The loader automatically patches the listed words again for 5000, so every address follows the move.
- B. The processor adds 5000 to each address as the program runs, so the process works at its new place.
- C. It works, because the image still holds relative addresses, and those do not depend on location.
- D. Its address fields still point into the region near 2000, so it reads and writes the wrong memory.

Answer: D. Its address fields still point into the region near 2000, so it reads and writes the wrong memory.

Relocatable loading binds addresses at load time. Once patched, the image is tied to that place, so a swapped-out process must come back to the same location.

6. Under dynamic run-time loading, a process's base register holds 12000. An instruction in memory refers to relative address 356. Which physical address does the processor access?

Answer: 12356

The hardware adds the base register to the relative address on each reference: $12000 + 356 = 12356$. Move the process and only the base register changes.

7. Match each address binding time to who turns addresses into real ones.

Programming time	The programmer, by writing real addresses into the code
Compile or assembly time	The compiler or assembler, converting symbolic addresses
Load time	The loader, adding the load address to relative addresses
Run time	The processor hardware, on every memory reference

The later the binding, the more freedom the OS has to place and move the program, and the more work is left for the loader or the hardware.

8. A linker joins three object modules in the order A, B, C. Module A is 1,200 words long, B is 800 words and C is 500 words. The load module is numbered from address 0. At which address does C start?

Answer: 2000

Each module starts where the ones before it end: C starts after A and B, at $1200 + 800 = 2000$.

9. A linker joins object modules A (1,200 words), B (800 words) and C (500 words) in that order. Inside B, an instruction refers to B's own relative address 75. The load module is numbered from 0. Which address does that instruction hold there?

Answer: 1275

B starts at 1200, the length of A, so relative address 75 inside B becomes $1200 + 75 = 1275$.

10. In an object module, how does a call to a routine that is defined in a different module appear before linking?

- A. As a symbol, the routine's name, still to be resolved
- B. As the routine's real main-memory address
- C. As an address relative to the start of the calling module
- D. As a relocation dictionary entry of the other module

Answer: A. As a symbol, the routine's name, still to be resolved

This is an external reference. The compiler cannot know where another module will be placed, so it leaves the name for the linker (or, with dynamic linking, the loader or OS) to resolve.

11. Sort each statement under the kind of linking it describes.

Static linking	Dynamic linking
Done in advance by a linkage editor	The load module still contains unresolved references
Each program file carries its own copy of the library code	An updated library is picked up without relinking the program
The program still runs if the library file is later deleted	One copy of a library in memory serves many programs

Static linking copies library code into each program ahead of time. Dynamic linking defers that work, so programs share one copy and pick up updates, but need the library present.

12. Which statements about run-time dynamic linking are true?

- A. A module is found, loaded and linked the first time the program calls it.
- B. Memory is spent only on the modules the program actually calls.
- C. Every referenced module is loaded before the program starts.
- D. The first call to a module can take longer than later calls.
- E. The program must be relinked whenever a library changes.

Answer: A, B, D

Run-time dynamic linking waits for the first call, so unused modules never take memory, at the cost of a pause on that first call. Loading everything before the start is load-time dynamic linking; relinking after a library change is a static-linking chore.

Chapter 8 · Virtual Memory

A program can run with only part of itself in memory. The trick is guessing which part, and paying as little as possible when the guess is wrong.

Paging and segmentation make it possible to keep only the pieces of a process that are actually in use in main memory and leave the rest on disk. This is virtual memory: each process sees a large private address space, and the hardware and the operating system bring pieces in on demand. The chapter first covers the hardware: page tables, multilevel and inverted tables, the translation lookaside buffer, page size and segmentation with paging. It then covers the operating system's decisions: when to fetch a page, where to put it, which page to replace (OPT, LRU, FIFO, clock), how many frames each process gets, when to write changed pages back, and how to avoid thrashing. It ends with how UNIX, Solaris, Linux, Windows and Android manage memory.

Learning objectives

- Explain why virtual memory works, using the principle of locality, and what thrashing is.
- Translate addresses through single-level, two-level and inverted page tables and a TLB.
- Simulate the OPT, LRU, FIFO and clock replacement policies and count page faults.
- Describe resident set management, the working set idea, cleaning policy and load control.
- Compare the memory managers of UNIX, Solaris, Linux, Windows and Android.

How Chapter 8 fits together

Virtual memory lets a process run with only some of its pieces in memory. The pieces that are present form the process's **resident set**; touching any other piece causes a **page fault**: the process is blocked while the OS reads the page from disk, often evicting another page. In the overview, 12 pages share 4 frames under first-in-first-out replacement: clustered references fault mainly when the program moves to a new cluster, random ones most of the time.

This works because of the **principle of locality**: over a short period, a program uses only a few of its pages (a loop, the function it is in, the data it walks through), so the recent past predicts the near future. When that fails, or each process has too few frames for the pages it is using, the system spends most of its time moving pages instead of running programs. That is **thrashing**.

The hardware (8.1)

Each page table entry holds a frame number plus a present bit and a modified bit. Tables too big to keep whole are split into two or more levels, or replaced by an inverted table with one entry per frame. A translation lookaside buffer (TLB) caches recent translations so most references skip the table. Page size is a trade-off between internal fragmentation and table size, and segmentation can be combined with paging.

The operating system's six decisions (8.2)

- **Fetch policy**: when a page comes in: demand paging (only on a fault) or prepaging (neighbours too).
- **Placement policy**: where it goes; important for segmentation, trivial for plain paging.

- **Replacement policy:** which page leaves: OPT (the ideal, needs the future), LRU, FIFO and clock; locked frames are never chosen.
- **Resident set management:** how many frames each process gets (fixed or variable) and whether a victim may come from another process (local or global scope); the working set guides the size.
- **Cleaning policy:** when changed pages are written back: on demand or ahead of time (precleaning).
- **Load control:** how many processes are in memory at once; suspending some is the cure for thrashing caused by overcrowding.

Real systems (8.3 to 8.6)

UNIX SVR4 and Solaris keep four paging tables, use copy on write, a two-handed clock and a lazy buddy kernel allocator (8.3). Linux describes its multilevel page tables the same way on every processor, with a buddy allocator, active and inactive lists and the slab allocator (8.4). Windows lets a process reserve and commit regions and grows and trims working sets (8.5). Android adds ashmem, ION, zram compression, trim requests to apps and a low-memory killer (8.6).

Why it matters next

Chapter 9 turns to the processor: which ready process runs next.

8.1 Hardware and Control Structures

You should be able to

- Explain why run-time address translation and scattered pieces mean a process can run with only its resident set in main memory, and trace what happens on a page fault.
- Use the principle of locality to explain why virtual memory works, and describe thrashing.
- Read a page table entry (frame number, present bit, modified bit, protection bits) and translate addresses through single-level, two-level and inverted page tables.
- Trace a reference through the TLB and page table, and compute the effective access time from the TLB hit ratio.
- Weigh the page size trade-offs, and describe segmentation in virtual memory, combined segmentation and paging, and ring protection.

Why not all of a process need be in memory

Two properties of paging and segmentation make virtual memory possible: (1) every reference is a **logical address translated at run time**, so a swapped-out process can come back into different frames; (2) a process is broken into **pieces that need not be contiguous**. So **not all pieces need be in main memory while it runs**.

The OS loads a few pieces; the part in main memory is the **resident set**. When the process references a piece that is not resident:

1. The hardware raises an interrupt: a memory access fault (**page fault**).
2. The OS blocks the process and issues a disk read (first freeing a frame if none is free).
3. The OS dispatches another process while the disk works.
4. The disk's I/O interrupt arrives; the OS updates the table and makes the process Ready.

5. When dispatched again, it re-executes the faulting instruction.

Payoffs: more processes fit, so the processor is busier (if each waits for I/O 80% of the time, it idles 0.8ⁿ of the time: 64% for 2 processes, 33% for 5), and a process can be **larger than main memory**. **Real memory** = main memory; **virtual memory** = the larger memory a process perceives, backed by disk.

Both kinds of paging use equal-size frames, invisible pages, internal fragmentation, and a page table per process plus a free-frame list; both kinds of segmentation use no fixed division, visible segments, external fragmentation, and a segment table per process plus a list of holes. The virtual-memory versions differ: not every piece is in memory while running; pieces are read in when needed; reading one in may force another out; a process may exceed main memory.

Locality and thrashing

A piece thrown out just before it is needed must be fetched straight back; when this happens constantly, the system spends most of its time moving pieces and little executing: **thrashing**. The **principle of locality** says that over a short period references cluster in a few pages (a loop, a function, an array), so recent behaviour predicts the near future and good guesses are possible.

Requirements: **hardware** support for paging and/or segmentation, and **OS software** to move pieces.

Page table entries and translation

Each process has its own page table (indexed by page number, so the page number and owner are not stored). A **page table entry (PTE)** holds the **frame number** plus control bits: **P** (present: the page is in main memory), **M** (modified since loaded; if $M = 0$ at replacement, the page need not be written out), and bits for protection and sharing (often also a use bit and a lock bit). Translation: the **page table pointer register** holds the start of the running process's table; the page number indexes it; frame number + unchanged offset = real address.

Example (1 KB pages, table at 30,720, 4-byte entries): virtual address 3,500 = page 3, offset 428. The entry is at $30,720 + 4 \times 3 = 30,732$. If page 3 is in frame 25, the real address is $25 \times 1,024 + 428 = 26,028$. A write sets $M = 1$; writing a read-only page is a protection fault.

Two-level page tables

A page table needs an entry per virtual page. With 32-bit addresses and 4 KB pages there are 2^{20} pages; with 4-byte entries the user page table is **4 MB**, which fills $2^{10} = 1,024$ pages. So page tables are kept in virtual memory too. A **root page table** (page directory) with 2^{10} entries (exactly 4 KB) points to those pages and always stays in memory. The address splits **10 | 10 | 12**: root index, index inside that page of the user table, offset. Example: 0x00403A7C → root entry 1, table entry 3, offset 2,684; if that page is in frame 5,021 the real address is $5,021 \times 4,096 + 2,684 = 20,568,700$. A fault can occur at either level: if the root entry's $P = 0$, that page of the page table must be read in first. Without a TLB a reference costs 3 memory accesses. 64-bit machines use more levels (x86-64: 4, or 5).

Inverted page tables

An **inverted page table** has one entry per **frame** of real memory, so its size is a fixed fraction of real memory whatever the number or size of processes. The page number is **hashed** to find where to start; entries with the same hash are linked by a **chain pointer**. Each entry holds the **page number, process ID, control bits** (valid, referenced, modified, protection, locking) and the **chain pointer** (next entry, or empty). A match on page number *and* process ID gives the frame (the entry's index); the end of the chain means a page fault. Used on PowerPC, UltraSPARC and IA-64.

Translation lookaside buffer (TLB)

Without help, each reference needs two memory accesses (PTE, then data). The **TLB** is a small, fast cache of recently used PTEs, searched by **associative lookup** (all entries compared with the page number at once). **Hit**: the frame number gives the real address. **Miss**: read the PTE; if $P = 1$, form the address and load the PTE into the TLB; if $P = 0$, page fault: the OS reads the page in (freeing a frame if needed), updates the table, and the instruction is retried. The real address then goes to the memory cache and main memory. Ignoring faults, with hit ratio h , TLB time t and memory time m : **EAT** = $h(t + m) + (1 - h)(t + 2m)$. Example: $h = 0.95$, $t = 2$, $m = 100$ ns: $0.95 \times 102 + 0.05 \times 202 = 107$ ns (no TLB: 200). On a process switch the TLB must be **flushed**, or its entries tagged with an **address-space identifier (ASID)**.

Page size

- **Smaller pages**: less internal fragmentation, but more pages, so larger page tables, parts of which may be paged out (one reference can then fault twice).
- **Larger pages**: disks transfer large blocks more efficiently, and the TLB covers more memory.
- **Fault rate vs page size** (fixed memory): low for very small pages; rising as each page holds more data far from recent references; falling again as a page nears the process size (zero if one page holds it all).
- **Fault rate vs frames**: falls as more frames are allocated.

4 KB is most common. **Large pages** (x86-64: 2 MB, 1 GB) extend **TLB reach** = entries $\times$ page size: 64 entries $\times$ 4 KB = 256 KB, but 64×2 MB = 128 MB. Sizes range from 512 bytes (VAX) to 4 KB–16 MB choices (MIPS R4000). Scattered objects and many threads reduce locality.

Segmentation in virtual memory

Segments can **grow** (extended or moved by the OS), modules can be **recompiled independently**, and **sharing and protection** are natural. A segment table entry holds the segment's **base, length, P** and **M** bits and protection/sharing bits. Real address = base + offset, after checking offset < length.

Combined segmentation and paging

The programmer sees segments, each cut into pages. Logical address = segment number + offset, and the offset is read as page number + page offset. Each process has one segment table; each segment has its own page table; the segment entry holds the length and points to that page table. Example (1 KB pages): segment 1, offset 1,500 = page 1, offset 476; page 1 in frame 21 gives $21 \times 1,024 + 476 = 21,980$.

Protection and sharing

Length checks stop a program running past a segment; a segment listed in several processes' tables is **shared**. **Ring protection**: ring 0 (most privileged, the kernel) outward to user rings. A program may access **data** only in its own or a **less** privileged ring, and may **call services** only in its own or a **more** privileged ring, through controlled entry points (gates). x86 has 4 rings; most systems use only rings 0 and 3.

Key terms

Resident set

The pages (or segments) of a process that are in main memory right now. The rest of the process waits on disk until it is needed.

Real memory

Main memory: the physical RAM. The word "real" contrasts it with virtual memory, the larger memory each process believes it has.

Virtual address space

The whole range of addresses a process may use, as the process sees it. It can be far larger than real memory; the parts not in memory are kept on disk.

Page table entry (PTE)

One row of a page table: the number of the frame that holds the page, plus control bits such as present, modified and protection.

Present bit (P)

A control bit in a page or segment table entry that says whether that piece is in main memory right now. A reference to a piece with $P = 0$ causes a fault.

Modified bit (M)

A control bit the hardware sets when a page is written. If it is still 0 when the page is replaced, the copy on disk is current, so the page need not be written out.

Page table pointer register

A processor register holding the address where the running process's page table starts. The OS reloads it on every process switch.

Root page table (page directory)

The top level of a two-level page table: a small table, kept in main memory at all times, whose entries point to the pages that hold the rest of the page table.

Multilevel page table (two-level page table)

A page table split into levels, where each entry of an upper level points to one page of the level below, so only the parts of the table in use need to be in main memory.

Inverted page table

One table for the whole system with one entry per frame of real memory instead of one per virtual page. It is searched by hashing the page number, and its size depends only on the size of real memory.

Hash function

A quick rule that turns a key, such as a page number, into a small table index. Different keys can give the same index (a collision), so entries that collide are linked in a chain.

Translation lookaside buffer (TLB)

A small, fast hardware cache of recently used page table entries inside the processor, so most address translations need no extra trip to memory.

TLB miss

A translation whose page has no entry in the TLB, so the page table entry must be read from memory first. The opposite, a TLB hit, gets the frame number straight from the TLB.

Associative lookup

A search in which the key is compared with every entry at the same moment by parallel hardware, instead of being used as an index. A TLB is searched this way.

Effective access time (EAT)

The average time per memory reference once address translation is counted. With TLB hit ratio h , TLB time t and memory time m : $EAT = h(t + m) + (1 - h)(t + 2m)$.

Address-space identifier (ASID)

A small tag naming the process each TLB entry belongs to, so entries of several processes can share the TLB. Without it the TLB must be flushed at every process switch.

TLB reach

The amount of memory the TLB can map at once: number of TLB entries $\times$ page size.

Large page (huge page)

A page far bigger than the usual 4 KB, such as 2 MB or 1 GB, offered by many processors so that one TLB entry covers much more memory.

Protection ring

One of a set of privilege levels numbered from 0 (most privileged, the kernel) outward. Code may use data in its own or a less privileged ring, and may call services in its own or a more privileged ring.

Review questions with answers

1. Which pair of properties of paging and segmentation makes it possible to run a process with only some of its pieces in main memory?

- A. Pages all have one fixed size, while each segment can be sized to fit the code or data it holds.
- B. Every address is translated at the moment it is used, and the pieces of a process need not be contiguous.
- C. The OS loads every piece when the process starts, and the hardware caches page table entries to speed up lookups.
- D. Programs are compiled with absolute addresses, and main memory is divided into fixed partitions set up at boot.

Answer: B. Every address is translated at the moment it is used, and the pieces of a process need not be contiguous.

Run-time translation lets any piece live in any frame and come back somewhere else, and non-contiguity makes the pieces independent. A piece that is not in memory is then just a translation that fails, which the OS can repair on demand.

2. Put the handling of a page fault in order.

Correct order:

1. The process references an address in a page that is not in main memory
2. The hardware raises a page fault interrupt and the OS takes over
3. The OS blocks the process and issues a disk read for the page
4. The OS dispatches another process while the disk works
5. The disk interrupts; the OS updates the page table and makes the process Ready
6. The process is dispatched again and retries the instruction

The fault blocks only the faulting process. The processor stays busy with another process during the slow disk read, and the original instruction is re-executed once the page is present.

3. The resident set of a process is the set of pages the process will need in the near future.

Answer: False

The resident set is simply the part of the process that is in main memory right now. Which pages will be needed soon is what the OS tries to predict, using locality; the two are not the same thing.

4. A system has 32-bit virtual addresses, 4 KB pages and 4-byte page table entries. The user page table is itself stored in 4 KB pages. How many pages does it occupy?

Answer: 1024 pages

$2^{32} \div 2^{12} = 2^{20}$ entries; $\times 4$ bytes = 4 MB = 2^{22} bytes; $\div 2^{12} = 2^{10} = 1,024$ pages. A root table with 1,024 four-byte entries (exactly 4 KB) can point to all of them.

5. A TLB lookup takes 2 ns, a main memory access takes 100 ns, and the TLB hit ratio is 95%. Ignoring page faults, what is the effective access time?

Answer: 107 ns

$EAT = h(t + m) + (1 - h)(t + 2m) = 0.95 \times 102 + 0.05 \times 202 = 96.9 + 10.1 = 107$ ns. Without a TLB every reference would cost $2 \times 100 = 200$ ns.

6. Which items are stored in an entry of a conventional page table (one table per process, indexed by page number)?

- A. The number of the frame that holds the page
- B. A present bit
- C. A modified bit
- D. Protection bits
- E. The virtual page number
- F. The identifier of the process that owns the page

Answer: A, B, C, D

The page number is the index into the table, so it need not be stored, and the owner is implied by whose table it is. An inverted page table, shared by all processes, is the one that must store both the page number and the process identifier.

7. The OS picks a page to replace and finds that its modified (M) bit is 0. What does that tell it?
- A. The page has not been referenced at all since it was loaded, so it is the best page to replace right now.
 - B. The page differs from its copy on disk, so it must be written back to disk before its frame can be reused.
 - C. The page has not been written since it was loaded, so its frame can be reused without writing the page to disk.
 - D. The page is not in main memory at the moment, so its page table entry does not point to any real frame.

Answer: C. The page has not been written since it was loaded, so its frame can be reused without writing the page to disk.

M = 0 means the copy on disk is still identical to the one in memory, so replacing the page costs one disk transfer (reading the new page) instead of two.

8. Does each statement describe a conventional per-process page table or an inverted page table?

Per-process page table	Inverted page table
One entry for every virtual page of a process Its size grows with the virtual address space Indexed directly by the page number	One entry for every frame of real memory Its size is a fixed fraction of real memory Searched by hashing the page number and following a chain Each entry must record which process owns the page

A conventional table maps every virtual page of one process and is indexed by page number. An inverted table maps every frame, is shared by all processes, and is searched by hash, so each entry must say which page of which process it holds.

9. Match each term to its meaning.

Resident set	The part of a process that is in main memory now
Thrashing	Most of the time goes to moving pages instead of running programs
TLB	A hardware cache of recently used page table entries
Root page table	A small, always-resident table that points to pages of the user page table
Address-space identifier	A tag that lets TLB entries of several processes coexist

These are the core pieces of virtual memory hardware: what is in memory, what goes wrong when too little is, how translation is sped up, and how page tables are kept small.

10. Compared with 4 KB pages, what is the usual effect of switching to much smaller pages?
- A. Less internal fragmentation, but larger page tables
 - B. More internal fragmentation, but smaller page tables
 - C. A larger TLB reach
 - D. More efficient disk transfers

Answer: A. Less internal fragmentation, but larger page tables

With small pages, less of each process's last page is wasted, but a process needs more pages, so its page table grows and may itself have to be paged.

11. Code running in ring 3 tries to read data that lives in ring 1. Under ring protection, what happens?
- A. It is allowed: code may always read data in a more privileged ring, but it may not write it.
 - B. It is allowed, but only through a gate, the controlled entry point into the inner ring.
 - C. It is allowed as long as the segment's present bit is 1, meaning the data is in main memory.
 - D. The access is refused: data may be used only in the same or a less privileged ring.

Answer: D. The access is refused: data may be used only in the same or a less privileged ring.

Ring 1 is more privileged than ring 3. Code may use data in its own ring or a less privileged (higher-numbered) one, and may call services in its own or a more privileged ring through a gate.

12. In a system that combines segmentation and paging, what does a segment table entry provide on the way to the data?
- A. The frame number of every page in the segment
 - B. A pointer to one page table shared by all segments of all processes
 - C. The length of the segment and the address of that segment's page table
 - D. The real address of the segment's first byte, as in pure segmentation

Answer: C. The length of the segment and the address of that segment's page table

The length check comes first; then the segment offset is split into a page number and a page offset, and the page number indexes the page table that the segment entry points to.

8.2 Operating System Software

You should be able to

- Name the six policy areas of virtual-memory software and the question each one answers.
- Contrast demand paging with prepaging, and explain why placement barely matters for paging.
- Run OPT, LRU, FIFO and clock (including the modified-bit variant) on a reference string, count page faults, and recognise Belady's anomaly.
- Explain page buffering, fixed vs variable allocation, local vs global scope, the working set and its approximations (PFF, VSWS).
- Describe cleaning policies, load control and the choices for which process to suspend.

Why the software decisions matter

Designers choose whether to use virtual memory, whether to use paging, segmentation or both (mostly set by the hardware), and the algorithm for each policy (pure OS software). Every page fault costs OS code (choosing a victim, updating tables), disk I/O (milliseconds) and a process switch, so the goal of every policy is a **low page fault rate**. With 100 ns per access and 8 ms per fault, one fault per 100,000 references nearly doubles the run time.

Policy	Question it answers
Fetch	When does a page come in?
Placement	Where in real memory does it go?
Replacement	Which page leaves to make room?
Resident set management	How many frames per process, and whose page goes?
Cleaning	When is a modified page written back?
Load control	How many processes are kept in memory?

Fetch and placement

Demand paging reads a page only when it is referenced: a burst of faults at start-up, then few, thanks to locality. **Prepaging** also reads other pages, usually the ones stored after it on disk, in the same operation; positioning the disk costs far more than transferring one more page. Unused pages are wasted. Prepaging is not swapping, which moves a whole process. Example (8 ms per disk operation + 0.1 ms per page): demand paging 10 faults, 81 ms; 3 pages per read 4 faults, 33.2 ms; 8 per read 3 faults, but 10 of 20 pages read are never used.

Placement matters for pure segmentation (first-fit, best-fit). For paging any frame is as good as any other, except on NUMA machines, where memory near the processor running the process is faster.

Replacement

The ideal victim is the page least likely to be referenced soon, predicted from past behaviour. **Frame locking**: a lock bit marks frames that may never be replaced (kernel, key control structures, I/O buffers, time-critical code).

- **OPT**: evict the page whose next reference is farthest in the future. Fewest possible faults, but needs the future: a yardstick only.
- **LRU**: evict the page unused for the longest time. Close to OPT thanks to locality, but exact LRU needs a time stamp per reference or a stack of pages.
- **FIFO**: treat the frames as a circular buffer and evict the page in memory longest. Simple, but evicts busy pages just because they are old.
- **Clock**: a use bit per frame, set when the page is loaded and on every reference. To replace, the pointer advances: a frame with use bit 1 has it cleared and is passed; the first frame with use bit 0 is the victim; the pointer then moves to the next frame. If every bit is 1 the pointer goes all the way round and evicts the page where it started.

Worked example, 3 frames, references 3 4 5 2 4 3 4 1 4 2 3 2 4 1: OPT 7 faults, LRU 9, clock 10, FIFO 11 (counts include the first loads into empty frames). **Belady's anomaly**: FIFO on 1 2 3 4 1 2 5 1 2 3 4 5 gives 9 faults with 3 frames and 10 with 4. LRU and OPT never do this, because the pages they keep with n frames are always among those they keep with $n + 1$.

Modified pages

Clock with the modified bit sorts frames into $(u, m) = (0, 0)$ best, $(0, 1)$, $(1, 0)$, $(1, 1)$ worst. (1) Scan from the pointer for $(0, 0)$, changing no bits; take the first. (2) If none, scan for $(0, 1)$, clearing the use bit of each frame passed; take the first. (3) If none, every use bit is now 0: repeat step 1 and, if needed, step 2, which must succeed. It prefers pages needing no write-back.

Page buffering: an evicted page loses its page table entry but stays in its frame, on the tail of the **free page list** (clean) or the **modified page list** (changed). A page read in takes the frame at the head of the free list. A page referenced again before its frame is reused is reclaimed with no disk read; modified pages are written in batches. This makes even FIFO perform well.

Cleaning policy: demand cleaning writes a page only when it is selected for replacement, so a fault with a modified victim waits for two transfers. **Precleaning** writes modified pages ahead of time in batches, but a page may be modified again, wasting the write. Best: page buffering, cleaning only replaceable pages in batches and keeping cleaning separate from replacement. (Model: 8 ms transfers, half the victims modified, 30% re-modified: average wait 12 ms with demand cleaning, 9.2 ms with precleaning but 15 wasted writes per 100 replacements, 8 ms with buffering.)

Resident set management

Fewer frames per process lets more processes fit (one is likelier to be ready) but raises each one's fault rate; past some size, more frames barely help. **Fixed allocation** gives a process a set number of frames; **variable allocation** lets it change. **Local scope** picks a victim among the faulting process's own pages; **global scope** among all unlocked pages.

- Fixed + local: evict an own page; the size must be chosen in advance (too small: many faults; too large: wasted memory).
- Fixed + global: impossible (taking another process's frame changes both sizes).
- Variable + global: the faulting process gets a free frame; when none is left, any process may lose a page. Easiest and common; page buffering softens bad victim choices.
- Variable + local: start with an allocation based on the program type, replace within the process, and periodically re-evaluate the allocation.

Working set and its approximations

$W(t, \Delta)$ is the set of pages referenced in the last Δ units of **virtual time** (the process's own references). A larger Δ never gives a smaller set, and $1 \leq |W| \leq \min(\Delta, \text{number of pages})$. Example: for 1 2 3 1 2 3 1 2 4 5 4 6 ..., $W(12, 4) = \{4, 5, 6\}$. Policy: track each working set, drop pages that leave it, and run a process only if its working set is resident. Problems: the past may not predict the future, exact measurement is impractical, and the best Δ is unknown and varies.

Page fault frequency (PFF): a use bit per page. On a fault, if fewer than F references have passed since the last fault, add the page; otherwise first drop every page with use bit 0, reset the others' use bits, then add the page. Weak when locality changes: faults come fast, so old and new pages pile up.

VSWS (variable-interval sampled working set) uses M (minimum interval), L (maximum) and Q (faults allowed). Use bits are reset at each sampling instant; faulting pages are added; a sample drops pages

whose use bit is still 0. Sample when L references have passed; if Q faults happen first, sample at once if at least M have passed, otherwise wait until M.

Load control and suspension

Too few processes in memory: often all are blocked and the processor idles. Too many: each has too few frames and the system thrashes. Utilization rises, peaks, then collapses (in this section's model: 19% with 1 process, 87% at 7, 16% at 14). Guides: working set or PFF (implicit load control); $L = S$ (mean time between faults equal to mean fault service time); keep the paging device about **50% busy**; watch the **clock sweep rate** (slow sweeps: room for more).

To lower the level, suspend: the **lowest-priority** process; the **faulting** process (probably lacks its working set and is about to block); the **last activated** (least likely to have its working set resident); the one with the **smallest resident set** (cheapest to reload, but penalises good locality); the **largest** process (frees the most frames); or the one with the **largest remaining execution window** (like shortest-processing-time scheduling).

Key terms

Fetch policy

The rule that decides when a page is brought into main memory: only when it is referenced (demand paging), or ahead of time together with others (prepaging).

Demand paging

A fetch policy that reads a page from disk only when the process references it and takes a page fault. A new process faults often at first, then less as the pages it is using arrive.

Prepaging

A fetch policy that, along with the page that faulted, reads other pages the process has not asked for yet, usually its neighbours on disk, because one larger disk read is cheaper than several small ones.

Placement policy

The rule that decides where in real memory a piece of a process goes. It matters for pure segmentation (first-fit, best-fit) but hardly at all for paging, where every frame is as good as any other.

Replacement policy

The rule that chooses which resident page to evict when a new page must come in and no frame is free. It aims to remove the page least likely to be used soon.

Frame locking

Marking a frame with a lock bit so its page can never be chosen for replacement; used for the kernel, key control structures, I/O buffers and time-critical code.

Optimal policy (OPT)

The replacement policy that evicts the page whose next reference lies farthest in the future. It causes the fewest possible faults but needs knowledge of the future, so it serves only as a yardstick.

Least recently used (LRU)

The replacement policy that evicts the page that has gone unreferenced for the longest time, betting that the recent past predicts the near future.

First-in-first-out (FIFO)

The replacement policy that evicts the page that has been in memory the longest, no matter how often it is used. Simple, but it throws out busy pages just because they are old.

Clock policy

A cheap approximation of LRU: frames form a circle with a pointer, and each frame has a use bit. The pointer passes over frames whose use bit is 1 (clearing it) and evicts the first page whose use bit is 0.

Use bit

A one-bit flag per frame (also called the reference bit) set to 1 when a page is loaded and every time it is referenced; replacement and working-set policies clear it to see which pages are still being used.

Belady's anomaly

The surprising case in which giving FIFO replacement more frames produces more page faults on the same reference string. LRU and OPT never behave this way.

Page buffering

Keeping recently evicted pages in their frames on a free page list (clean pages) or a modified page list (changed pages) so that a page referenced again soon can be reclaimed without a disk read.

Resident set

The pages of a process that are in main memory right now. Resident set management decides how many frames each process gets and whose page is replaced.

Variable allocation

A resident set policy in which the number of frames a process holds can change over its lifetime. Its opposite, fixed allocation, gives a process a set number of frames for its whole run.

Replacement scope

Which pages are candidates when a process faults: local scope considers only that process's own resident pages; global scope considers every unlocked page in memory.

Working set

$W(t, \Delta)$: the set of pages a process referenced during its last Δ units of virtual time (its last Δ memory references). It estimates the pages the process needs right now.

Page fault frequency (PFF)

A working-set approximation that sizes a resident set from how quickly faults arrive: a fault soon after the previous one adds a page; a fault after a long gap first drops the pages that went unused.

Cleaning policy

The rule for when a modified page is written back to disk: only when it is chosen for replacement (demand cleaning) or ahead of time in batches (precleaning).

Load control

Deciding how many processes are resident in main memory at once (the multiprogramming level), so that the processor stays busy without the system thrashing.

Review questions with answers

1. Match each virtual-memory policy to the question it answers.

Fetch policy	When is a page brought into main memory?
Placement policy	Where in real memory does a piece go?
Replacement policy	Which resident page is removed to make room?
Resident set management	How many frames does each process get?
Cleaning policy	When is a modified page written to disk?
Load control	How many processes are kept in main memory?

Six separate decisions, all serving one goal: keep the page fault rate low, because every fault costs OS work, disk I/O and a process switch.

2. Why can no real operating system use the optimal (OPT) replacement policy?

- A. It needs a use bit in every page table entry, which most hardware lacks.
- B. It must know when each page will next be referenced, which lies in the future.
- C. It suffers from Belady's anomaly, so adding memory can make it worse.
- D. It takes too long to scan all the frames on every fault.

Answer: B. It must know when each page will next be referenced, which lies in the future.

OPT evicts the page whose next reference is farthest away, which requires knowing future references. It is used only as a yardstick to judge real policies.

3. A process has 3 frames, all empty at the start, and makes the references 2 6 1 2 5 6 2 1 4 6 2 5. How many page faults does FIFO replacement cause, counting the first three loads?

Answer: 8 faults

Faults on 2, 6, 1; 2 hits; 5 evicts 2; 6 hits; 2 evicts 6; 1 hits; 4 evicts 1; 6 evicts 5; 2 hits; 5 evicts 2. That is $3 + 5 = 8$ faults.

4. A process has 3 frames, all empty at the start, and makes the references 2 6 1 2 5 6 2 1 4 6 2 5. How many page faults does LRU replacement cause, counting the first three loads?

Answer: 10 faults

After 2, 6, 1 load: 2 hits; 5 evicts 6; 6 evicts 1; 2 hits; 1 evicts 5; 4 evicts 6; 6 evicts 2; 2 evicts 1; 5 evicts 4. $3 + 7 = 10$ faults. On this string FIFO happens to beat LRU (8 faults): neither is always better, and OPT beats both (7 faults).

5. Under FIFO replacement, giving a process more frames can never increase the number of page faults it suffers.

Answer: False

Belady's anomaly: with FIFO, the string 1 2 3 4 1 2 5 1 2 3 4 5 causes 9 faults with 3 frames but 10 with 4. LRU and OPT never behave this way, because the pages they hold with n frames are always among those they would hold with $n + 1$.

6. A clock policy manages four frames holding pages A, B, C and D, in that order around the circle, with use bits 0, 1, 1, 1. The pointer is at the frame holding C. Page E must be loaded. Which page is replaced?
- A. A
 - B. B
 - C. C
 - D. D

Answer: A. A

From C: use bit 1, clear it and move on; D: use bit 1, clear and move on; A: use bit 0, so A is replaced. E goes into that frame with its use bit set, and the pointer moves on to B.

7. Put the steps of the clock policy that also uses the modified bit (u = use bit, m = modified bit) in order.

Correct order:

1. Scan from the pointer for a frame with u = 0 and m = 0, changing no bits
2. If none, scan for u = 0 and m = 1, clearing the use bit of each frame passed
3. If still none, repeat the first scan: every use bit is now 0
4. If needed, repeat the second scan, which is now sure to find a frame

The first scan looks for the cheapest victim: idle and clean. The second accepts an idle but modified page and ages every page it passes. If both fail, all use bits are 0, so repeating them must succeed.

8. Sort each statement under the resident set strategy it describes.

Fixed allocation, local scope	Variable allocation, global scope	Variable allocation, local scope
The number of frames is chosen when the process starts and never changes Too small an allocation means many faults; too large wastes memory	A page fault can take a frame away from another process The simplest scheme to implement, and a widely used one	A fault replaces one of the process's own pages, but its allocation is re-evaluated from time to time

Fixed + local never changes a size. Variable + global grows the faulting process and can take any unlocked page. Variable + local replaces within the process and adjusts sizes periodically. Fixed + global cannot exist.

9. A process makes the references 2 4 2 3 4 5 5 3 2 6 at virtual times 1 to 10. What is the size of its working set $W(10, 5)$?

Answer: 4 pages

$W(10, 5)$ holds the distinct pages referenced at times 6 to 10: 5, 5, 3, 2, 6, which is the set {5, 3, 2, 6}, so $|W| = 4$.

10. Which statements about page buffering are true?

- A. An evicted page stays in its frame until that frame is given to another page.
- B. A page reclaimed from the free page list needs no disk read.
- C. Pages on the modified page list are written to disk in batches.
- D. An evicted page keeps a valid page table entry, so referencing it causes no fault at all.
- E. Page buffering only works together with LRU replacement.

Answer: A, B, C

The page table entry is removed, so a reference does fault, but the fault is cheap: the page is still in memory and is simply moved back. It works with any replacement policy and makes even FIFO perform well.

11. As more and more processes are loaded into memory, processor utilization rises and then drops sharply. What causes the drop?

- A. Each process gets too few frames, so faults pile up at the paging device while the processor waits: thrashing.
- B. The scheduler spends so much time choosing among all the ready processes that little time is left to run them.
- C. Each process spends more of its time waiting for ordinary I/O, such as keyboard input, so fewer are ready to run.
- D. The page tables grow so large that they no longer fit in the TLB, so every address translation slows down.

Answer: A. Each process gets too few frames, so faults pile up at the paging device while the processor waits: thrashing.

Past the peak, each process has fewer frames than its working set, so it faults constantly. The paging device saturates, processes queue for it, and the processor idles. Load control keeps the level below that point.

12. Load control must suspend one process. Under the "last process activated" rule, which process goes, and why?

- A. The most recently activated process, because new processes are always given the lowest priority.
- B. The process that caused the most recent page fault, because it is about to block for the disk anyway.
- C. The most recently activated process, because it is the least likely to have its working set in memory yet.
- D. The oldest process in memory, because it has already had more processor time than any other process.

Answer: C. The most recently activated process, because it is the least likely to have its working set in memory yet.

A newly activated process has not yet built up its working set, so suspending it throws away little useful resident memory. The other rules use priority, the faulting process, resident set size, process size or remaining run time.

8.3 UNIX and Solaris Memory Management

You should be able to

- Explain why SVR4 UNIX and Solaris run two separate memory schemes: a paging system and a kernel memory allocator.
- Name the four paging data structures (page table, disk block descriptor, page frame data table, swap-use table), list their fields, and follow one page through all four.
- Trace copy on write after fork(): which pages are shared, which get copied, and when.
- Run the two-handed clock and compute how handsread, scanrate and free memory set the time a page has to be used again before it is paged out.
- Explain the lazy buddy system (locally vs globally free blocks) and use the slack count to decide what happens when a block is freed.

Two memory managers in one kernel

UNIX runs on many kinds of machines, so its memory management varies. Early UNIX had no virtual memory: each process sat in one variable-sized partition, and whole processes were swapped out. **SVR4** (System V Release 4, late 1980s) and **Solaris** (Sun's UNIX, built on SVR4) use paged virtual memory, with two separate schemes in the kernel:

- The **paging system** hands out whole page frames, to the pages of user processes and to **disk block buffers** (cached copies of disk blocks).
- The **kernel memory allocator** takes frames from the paging system and cuts them into small pieces for the kernel's own tables and buffers.

The four paging data structures

Page table: one per process, one entry per virtual page. Fields:

- **Page frame number:** the physical frame holding the page (meaningful only when valid).
- **Age:** how long the page has been in memory without being referenced; older pages are better replacement candidates.
- **Copy on write:** set when processes share the page (as after fork()); the first write makes a private copy for the writer.
- **Modify:** set on a write; the page must then be written to disk before its frame is reused.
- **Reference:** set when the page is used; cleared by the replacement algorithm.
- **Valid:** the page is in main memory (touching it while Valid = 0 causes a page fault).
- **Protect:** whether writing is allowed (code pages are read-only).

Disk block descriptor: one per virtual page, beside its entry. Fields: **swap device number**, **device block number**, **type of storage** (swap, executable file, or demand zero: no disk copy, start from a zeroed frame). The entry stays small because the hardware reads it on every access.

Page frame data table: one entry per physical frame, indexed by frame number. Fields: **page state** (available, or holding a page whose copy is on swap, in an executable file, or in a DMA transfer); **reference count** (processes using the frame); **logical device** and **block number** of its disk copy; **pointers** into the

free-page list and a **hash queue**. Hashing a disk page's (device, block) finds quickly whether a frame already holds it; a freed frame keeps its contents until reused, so a fault can reclaim the page with no disk read.

Swap-use table: one per swap device, one entry per page on it: a **reference count** of the page table entries (via their descriptors) that point at that disk page, and the page's **identifier** on the device. Count 0: a free slot.

Example. After P forks Q, their shared data page (frame 1, swap block 120, copy on write) has frame count 2 and swap-use count 2; their code page (exec file) has frame count 2 and no swap-use entry, since code is reread from the program file.

fork() and copy on write

fork() copies the parent's page table entries, not its pages: every frame's reference count rises by one, and writable pages are marked copy on write in **both** tables (read-only code is simply shared). On a write:

- Protect = read-only → refused (an error, never a copy).
- Copy on write = 0 → write in place; the hardware sets Modify.
- Copy on write = 1 and frame count > 1 → copy the page into a free frame for the writer only, clear its COW bit, lower the old frame's count.
- Copy on write = 1 and frame count = 1 → last user: just clear the bit, no copy.

Reads never copy. A child that calls exec() at once has copied almost nothing.

The two-handed clock

When free memory runs low, the **page daemon** sweeps the page frame data table entries of the replaceable (not locked) pages as a circular list with two hands:

- The **front hand** clears each page's reference bit as it passes.
- The **back hand**, a fixed distance behind, checks the bit. Still 0: not used since the front hand passed, so the page goes on the page-out list (written to swap first if modified). 1: used, so it stays.

Scanrate is how many pages per second the hands move; **handspread** is the gap between the hands in pages. Together they set the time a page has to be used again:

window = handspread ÷ scanrate. Example: 2,000 pages at 500 pages/s gives 4 s; a page used at least every 4 s is never paged out. A larger handspread or slower scan gives pages longer; a shorter window frees memory faster but evicts pages still wanted, which fault back in. With one hand (plain clock) a bit is checked only after a full lap, so on a huge memory nearly every bit is set again by then.

How fast the hands move

The scanrate depends on free memory. At or above **lotsfree** the page daemon sleeps (no scanning). Below it the scanrate starts at **slowscan** and rises in a straight line as free memory falls, reaching **fastscan** at **minfree**.

$\text{scanrate} = \text{slowscan} + (\text{fastscan} - \text{slowscan}) \times (\text{lotsfree} - \text{free}) \div (\text{lotsfree} - \text{minfree})$. Example: lotsfree 4,000, minfree 1,000, slowscan 100, fastscan 1,000 pages/s and 2,500 free pages: $100 + 900 \times 1,500 \div$

3,000 = 550 pages/s; with handsread 2,000 the window is $2,000 \div 550 \approx 3.64$ s. Less free memory → faster hands → shorter window → more pages freed.

Kernel memory: the lazy buddy system

Kernel objects are small (tens to hundreds of bytes), short-lived and constantly allocated and freed, so paging is a poor fit. SVR4 uses a **buddy system**: blocks of 2^k bytes, halved to fit a request and merged with their buddy when both are free. A plain buddy system merges at once, though the same size is often wanted again moments later. The **lazy buddy system** defers coalescing until it looks necessary, then coalesces as much as possible. A free block is **locally free** (kept for reuse at its size, may not merge) or **globally free** (may merge).

For each size 2^i : N_i blocks exist, A_i allocated (a split block counts as allocated until its halves merge back; the rebuilt block is then freed by the same rules), G_i globally free, L_i locally free; $N_i = A_i + G_i + L_i$. The **slack** is $D_i = N_i - 2L_i - G_i = A_i - L_i$, allocated minus locally free. A local free moves a block from A to L, so D drops by 2; the rules keep $D \geq 0$.

Event	Action	D
Free, $D \geq 2$	free locally	$D - 2$
Free, $D = 1$	free globally and coalesce	0
Free, $D = 0$	free globally and coalesce; also free one locally free block globally and coalesce	0
Allocate a locally free block	reuse it	$D + 2$
Allocate a globally free block	take it	$D + 1$
Split a larger block	one half allocated, the other locally free	unchanged

Example. 64-byte blocks: $A = 5$, $G = 1$, $L = 2$, so $N = 8$ and $D = 8 - 4 - 1 = 3$. The next free is local (D becomes 1); the one after is global (D becomes 0). In a burst of 18 requests on a 256-byte pool (two 32-byte records held, then four rounds of two allocated and freed), a plain buddy system made 7 splits and 4 merges, the lazy one 4 splits and 0 merges.

Solaris

Solaris keeps the SVR4 page structures and the two-handed clock. For kernel memory it later introduced the **slab allocator**: a cache per kind of kernel object holding ready-made objects, so an allocation just takes one from its cache. Linux adopted the idea.

Key terms

SVR4 (System V Release 4)

A late-1980s version of AT&T's UNIX that combined features of the main UNIX lines of its day. Its paged virtual-memory design was carried into Solaris and other commercial UNIX systems.

Paging system

The part of an SVR4 or Solaris kernel that hands out page frames: to the pages of user processes and to disk block buffers that cache file data. It runs virtual memory.

Kernel memory allocator

The part of the kernel that hands out the small pieces of memory the kernel itself needs (tables, records, buffers), most of them much smaller than a page.

Copy on write (COW)

A way to share pages after fork(): parent and child map the same frames, marked copy on write, and a page is copied only when one of them first writes to it, and only for the writer.

Disk block descriptor

A record kept beside each page table entry that says where the page lives on disk: swap device number, block number, and the type of storage (swap, executable file, or fill with zeros).

Page frame data table

A kernel table with one entry per physical page frame, indexed by frame number: what state the frame is in, how many processes use it, where its disk copy is, and links into the free list and a hash queue.

Swap-use table

A table kept for each swap device, with one entry per page on the device: how many page table entries point at that disk page (a reference count) and the page's identifier on the device.

Swap device

A disk area (a partition or a file) set aside to hold pages of processes that are not in main memory.

Reference count

A counter of how many users share one object. When it drops to zero the object can be freed; while it is above one, changing the object needs care.

Hash queue

A short linked list reached by hashing a key. The kernel hashes a disk page's device and block number to find, without a full search, whether some frame already holds that page.

Reference bit

A bit in a page table entry that the hardware sets whenever the page is used. Replacement algorithms clear it and later check whether it was set again (also called the use bit).

Page daemon

A kernel process (also called the pageout daemon) that wakes up when free memory runs low, runs the replacement algorithm and puts unused pages on the list of pages to be paged out.

Two-handed clock

The SVR4 and Solaris page replacement algorithm: a front hand sweeps the frames clearing reference bits, and a back hand a fixed distance behind pages out every frame whose bit is still 0.

Handspread

The gap, in pages, between the front hand and the back hand of the two-handed clock.

Scanrate

How many pages per second the two hands of the two-handed clock move. Divided into the handspread it gives the time a page has to be used again.

Free-memory thresholds (lotsfree, minfree)

Two tuning levels for free memory. Below lotsfree the page daemon starts scanning, slowly; as free memory falls toward minfree the scan speeds up to its maximum.

Lazy buddy system

The SVR4 kernel memory allocator: a buddy system that does not merge a freed block with its buddy right away, because a block of that size is likely to be wanted again soon.

Locally free block

In the lazy buddy system, a free block that is kept ready for reuse at its own size and is not allowed to merge with its buddy.

Globally free block

In the lazy buddy system, a free block that is allowed to merge with its buddy, exactly as in an ordinary buddy system.

Slab allocator

A kernel allocator, introduced by Solaris and later adopted by Linux, that keeps caches of ready-made objects of one size and type, so allocating one is just taking it from its cache.

Review questions with answers

1. Why do SVR4 UNIX and Solaris use a separate kernel memory allocator instead of giving the kernel's own tables and buffers whole pages from the paging system?

- A. The paging system can only hand out memory that may later be paged out, and kernel data must always stay in memory.
- B. Most kernel requests are far smaller than a page and come and go very often, so whole pages would waste memory and time.
- C. The paging system serves only user processes and their disk buffers, so it cannot give any frames to the kernel.
- D. The kernel allocator is faster because it uses the two-handed clock to pick which small blocks to hand out next.

Answer: B. Most kernel requests are far smaller than a page and come and go very often, so whole pages would waste memory and time.

Kernel tables and buffers are typically tens to hundreds of bytes and come and go constantly. A 4 KB page per request would waste most of each page, so the allocator cuts frames into small power-of-two blocks (a lazy buddy system in SVR4).

2. Match each SVR4 paging structure with what it keeps one entry for.

Page table	Each virtual page: its frame and status bits
Disk block descriptor	Each virtual page: where its disk copy is
Page frame data table	Each physical page frame
Swap-use table	Each page on one swap device

Two records describe every virtual page (the page table entry and, beside it, the disk block descriptor); the page frame data table describes physical frames; each swap device has its own swap-use table.

3. Which SVR4 structure holds each field? (PTE = page table entry, DBD = disk block descriptor, frame table = page frame data table.)

PTE	DBD	Frame table	Swap-use
Copy on write bit Age	Type of storage Swap device number	Count of processes sharing a frame Free-list and hash-queue links	Count of PTEs naming one disk page

The page table entry holds the per-page bits the hardware and replacement algorithm use; the descriptor says where the page is on disk; the frame table entry describes one physical frame; the swap-use entry counts users of one page on a swap device.

4. With copy on write, fork() immediately copies every writable page of the parent so that the child has its own copy.

Answer: False

fork() copies only the page table entries and marks the writable pages copy on write in both processes. A page is copied later, when one of them first writes it, and only for the writer. Pages never written are never copied.

5. Parent P creates child C with fork(), using copy on write. Then C writes its data page. Which statements are true?

- A. C now has a private copy of the data page in a different frame.
- B. The original frame's reference count drops from 2 to 1.
- C. P is also given a fresh copy of the data page.
- D. If P writes its data page later, the kernel just clears P's copy-on-write bit without copying, because P is the frame's only user.
- E. The shared, read-only code page is copied as well.

Answer: A, B, D

Only the writer gets a copy, so C moves to a new frame and the old frame's count falls to 1. P keeps the original; when P writes, it is the last user, so no copy is needed. Code is read-only and is never copied.

6. Put in order what happens to a page that the program does not use again while the two-handed clock is running.

Correct order:

1. The front hand reaches the page and clears its reference bit
2. The hands move on while the program leaves the page alone
3. A handsread later, the back hand reaches the page and finds the bit still 0
4. The page is put on the list of pages to be paged out
5. Its frame becomes free (after the page is written to swap, if it was modified)

The front hand gives the page a fresh start, the gap between the hands is the time it has to be used, and the back hand acts on the result: still 0 means unused, so out it goes.

7. A two-handed clock runs with a handsread of 3,000 pages and a scanrate of 750 pages per second. After the front hand clears a page's reference bit, how many seconds does the page have to be used again before the back hand checks it?

Answer: 4 s

The back hand reaches the page after the hands have moved 3,000 pages, which takes $3,000 \div 750 = 4$ seconds.

8. A system has lotsfree = 5,000 pages, minfree = 1,000 pages, slowscan = 200 pages/s and fastscan = 1,800 pages/s. The scan rate rises in a straight line from slowscan (at lotsfree) to fastscan (at minfree). Free memory is now 2,000 pages. What is the scan rate?

Answer: 1400 pages/s

scanrate = $200 + (1,800 - 200) \times (5,000 - 2,000) \div (5,000 - 1,000) = 200 + 1,600 \times 0.75 = 1,400$ pages per second.

9. A process faults on a page whose disk block descriptor says swap device 1, block 121. Before reading the disk, the kernel hashes the pair (1, 121) and searches a hash queue of page frame data table entries. Why?

- A. To choose which swap device the page should be read from, since a copy may be kept on more than one device.
- B. To find a free slot on the swap device, so the page has somewhere to go the next time it is paged out.
- C. To find out whether some frame, even one on the free list, still holds that page, so no disk read is needed.
- D. To pick which page the two-handed clock should evict next, so a frame is ready before the disk read starts.

Answer: C. To find out whether some frame, even one on the free list, still holds that page, so no disk read is needed.

Frames are linked into hash queues by the disk address of the page they hold. A freed frame keeps its contents until it is reused, so a quick hash lookup can rescue the page and skip the slow disk read.

10. In a lazy buddy system, the 64-byte blocks are counted as A = 5 allocated, G = 1 globally free and L = 2 locally free. The slack is $D = N - 2L - G = A - L$. What is D?

Answer: 3

$N = 5 + 1 + 2 = 8$, so $D = 8 - 2 \times 2 - 1 = 3$, which equals $A - L = 5 - 2$. Because $D \geq 2$, the next 64-byte block freed will be freed locally, and D will drop to 1.

11. In a lazy buddy system the slack for one block size is $D = 1$, and a block of that size is freed. What happens?

- A. It is freed locally, and D rises to 3.
- B. It is freed globally, and one locally free block of that size is made global as well.
- C. It is freed locally, and D drops to -1.
- D. It is freed globally, so it may merge with its buddy, and D becomes 0.

Answer: D. It is freed globally, so it may merge with its buddy, and D becomes 0.

A local free would push $D = A - L$ down to -1, so the block is freed globally instead: A falls by 1, D falls to 0, and the block merges with its buddy if the buddy is globally free too.

12. If the scanrate stays the same, a larger handsread gives each page more time to be used again before the back hand checks it, so fewer pages are paged out.

Answer: True

The window is $\text{handsread} \div \text{scanrate}$. A bigger gap at the same speed means a longer window, so more pages get used in time and survive.

8.4 Linux Memory Management

You should be able to

- Describe Linux's processor-independent page-table tree (page global directory, page middle directory, page table) and split a virtual address into its index fields and offset.
- Walk an address through two-, three-, four- and five-level page tables, and explain how folded levels let one piece of code run on every processor.
- Use the buddy allocator's free lists (orders 0, 1, 2, ...) to allocate and free runs of contiguous page frames, splitting and merging buddies.
- Contrast the old page-aging scheme with today's split LRU (active and inactive lists, PG_referenced and PG_active flags) and trace pages between the lists.
- Explain how the slab allocator serves small kernel objects from caches of equal-size slots, and name the SLAB, SLOB and SLUB implementations.

Two jobs for one memory manager

Linux runs on about twenty processor families, so its memory manager is portable. It has two parts: **process virtual memory** (page tables, finding frames, choosing pages to evict) and **kernel memory allocation**. Frames are shared by kernel code and static data (never paged out), kernel objects (from the **slab allocator**), the **page cache** (copies of file data: **file-backed pages**, droppable if clean), user process pages (heap and stack are **anonymous pages**, needing swap to evict), and free frames, kept by the **buddy allocator** in runs of 1, 2, 4, 8, ... frames.

Page tables: one tree, described the same way on every processor

A flat table per process would be huge, so Linux keeps a tree of small tables, classically in **three levels**:

- **Page global directory (PGD)**: one per process, one page in size, in main memory whenever the process is active (every translation starts there). A register (CR3 on x86) holds its location and is reloaded at each process switch. Each entry points to a page of the middle directory.
- **Page middle directory (PMD)**: may span several pages; each entry points to a page of a page table.
- **Page table**: may span several pages; each entry refers to one virtual page (frame number plus control bits: present, dirty, accessed, access rights).

A virtual address splits into **global directory index | middle directory index | page table index | offset**.

Tables exist only for regions in use. The code is written against this generic tree; each processor supplies a few definitions (table sizes, bit positions).

Folding. A level the hardware lacks is defined with one entry and zero address bits, so the call that reads it hands back the entry it was given: no memory read, no cost, and the same code runs everywhere. Today's

Linux has five named levels, **PGD → P4D → PUD → PMD → PTE** (the PUD level was added in 2005, kernel 2.6.11; the P4D level in 2017).

Each table fills one 4 KB page: 512 eight-byte entries (9 index bits) or 1,024 four-byte entries (10 bits). So 32-bit x86 has 2 real levels, $10 + 10 + 12 = 32$ bits (4 GB); 64-bit ARM or RISC-V in 39-bit mode has $3, 9 + 9 + 9 + 12 = 39$ bits (512 GB); x86-64 has 4 (48 bits, 256 TB), or 5 with 5-level paging (57 bits, 128 PB).

Worked example (39-bit, three levels): address 0x55_5555_6A3C has offset 0xA3C and page number 0x55555556, whose 9-bit fields are 341 | 170 | 342. The walk reads PGD[341], PMD[170], PTE[342] to get frame f ; physical address = $f \times 4,096 + 0xA3C$. Three table reads precede the data read, which is why the TLB caches translations. Most MMUs walk the tree on a TLB miss; the kernel walks it in software to change mappings.

An **empty entry** at any level raises a **page fault**. If the address is inside a region of the process (a heap page never touched), the kernel zero-fills a free frame, records it in the PTE and restarts the instruction; if no region covers it, the process gets SIGSEGV.

Page allocation: the buddy allocator

Devices copying straight into memory, huge pages and big buffers need **contiguous** frames. Linux keeps free memory as blocks of 2^k contiguous frames (k = the block's **order**), one free list per order (orders 0 to 10: up to 1,024 frames = 4 MB).

1. A request for n pages is served from the smallest order with $2^k \geq n$ (5 pages → order 3 = 8 frames, 3 unused).
2. If that list is empty, take a block from the next larger non-empty list and halve it repeatedly; each spare half joins the list one order down.
3. On a free, the block's **buddy** starts at $f \text{ XOR } 2^k$. If the buddy is free and of the same order, merge them and repeat one order up.

Worked example (16 free frames): A asks for 1 page → splits $16 \rightarrow 8, 8 \rightarrow 4, 4 \rightarrow 2, 2 \rightarrow 1$; A gets frame 0 and the lists hold 1, 2–3, 4–7, 8–15. B asks for 3 → order 2: frames 4–7 (one unused). C asks for 4 → split 8–15; C gets 8–11. A frees frame 0 → merges with 1, then 2–3, giving 0–3; its buddy 4–7 is in use, so it stops. Neighbours need not be buddies: 2–3 and 4–5 touch, but the buddy of 2 is 0. Real Linux keeps these lists per memory zone, with per-CPU caches of single pages in front.

Page replacement then: page aging

Older kernels (last in 2.4.0–2.4.9, 2001) used a clock-like sweep in which each page's single use bit grew into an **8-bit age**. Each use raised the age, a background sweep lowered it, and a page whose age had fallen to 0 (the lowest) was the victim. The age is a running score of uses minus sweeps, not just "used or not". Because it counts uses, the scheme is a form of **least frequently used (LFU)**: a page used 5 times and then left alone keeps a positive age for several sweeps, while a page used once is back to 0 after one sweep. Busy pages are protected; stale ones linger.

Page replacement now: the split LRU

Since 2.4.10 (2001) Linux has kept an **active list** (pages that proved busy) and an **inactive list** (eviction candidates); since 2.6.28 (2008), one pair for **file-backed** and one for **anonymous** pages. Two page flags drive them: **PG_referenced** (one recent use) and **PG_active** (on an active list).

- A newly read file page starts at the head of the inactive list; its first use sets **PG_referenced**.
- A page on the inactive list that is used again while **PG_referenced** is set is **promoted** to the head of the active list (**PG_active** set, **PG_referenced** cleared). So a page must be used twice to become active.
- A use of an active page sets **PG_referenced**. Aging the active list looks at its tail: a referenced page has the flag cleared and goes round again; an unreferenced one is **deactivated** to the inactive head. Idle pages drift down.
- Reclaim takes the page at the **tail of the inactive list**, topping that list up from the active tail when it gets too short.

Why two lists: pages of a big file read once enter the inactive list, are never promoted and leave from its tail, so the active list (the working set) is untouched. Since kernel 6.1 (2022) Linux can instead use a multi-generational LRU with several age generations.

Kernel memory allocation: the slab allocator

Whole runs of pages come from the buddy allocator, but most kernel objects are a few bytes to a few KB, so the **slab allocator** sits on top of it:

- It keeps a **cache** for each object type and for each `kmalloc` size class (8, 16, 32, 64, 96, 128, 192, 256, 512 bytes, 1, 2, 4 and 8 KB). Larger `kmalloc` requests go straight to the buddy allocator.
- A cache takes pages from the buddy allocator and cuts each into equal slots; such a page (or small run of pages) is a **slab**, which is **full**, **partial** or **empty**.
- An allocation takes a free slot, from a partial slab first (so pages fill up); a free returns the slot; empty slabs go back to the buddy allocator when memory is tight.

Fast (no searching, splitting or merging) and thrifty (slots fit their objects). **Example:** `kmalloc(294)` uses the 512-byte cache: 218 bytes (43%) of the slot go unused and a 4 KB page holds 8 objects; a whole page would be about 7% used.

Implementations: **SLAB** (the original 1990s design, modelled on Solaris, with per-CPU queues; removed in 6.8, 2024), **SLOB** ("simple list of blocks" for tiny-memory machines; removed in 6.4, 2023) and **SLUB** (simpler, "unqueued", scales to many CPUs; default since 2.6.23, 2007, and the only one since 6.8).

All three share one interface (`kmem_cache_alloc` , `kmalloc` , `kfree` , ...), so other code never changed.

Key terms

Page global directory (PGD)

The top table of a Linux process's page-table tree. Each process has exactly one, a single page in size, and it must be in main memory whenever the process runs, because every translation starts there.

Page middle directory (PMD)

The level below the global directory in Linux's classic three-level layout. It may span several pages; each of its entries points to one page of a page table.

Page upper directory (PUD)

A level Linux inserted above the middle directory in 2005 (kernel 2.6.11), so that 64-bit processors with four levels of page tables run the same code.

Page 4th-level directory (P4D)

A level Linux inserted just below the global directory in 2017 for processors with five levels of page tables, such as recent x86-64 chips with 57-bit addresses. Elsewhere it is folded away.

Folded level

A page-table level that the processor does not really have. Linux defines it with one entry and zero address bits, so the walk passes straight through it and the same code works on 2-, 3-, 4- and 5-level hardware.

Buddy allocator

Linux's page allocator. It keeps free physical memory as blocks of 1, 2, 4, 8, ... contiguous frames, halves a bigger block when no block of the right size is free, and merges a freed block with its buddy whenever both are free.

Order (of a block)

The size class of a block in the buddy allocator: a block of order k is 2^k contiguous page frames and starts at a frame number that is a multiple of 2^k .

Page aging

The replacement scheme of older Linux kernels, used for the last time in 2.4.0 to 2.4.9 (2001): each page carried an 8-bit age that rose when the page was used and fell during a background sweep; pages whose age had fallen to 0 were evicted first.

Least frequently used (LFU)

A replacement idea that evicts the page used the fewest times over a recent period, instead of the page that has gone unused the longest.

Split LRU

Linux's page-replacement design since kernel 2.6.28 (2008): file-backed pages and anonymous pages each get their own active list and inactive list, and reclaim takes pages from the tail of an inactive list. It split the single active/inactive pair that Linux had used since 2.4.10 (2001).

Active list

The list of pages that have shown they are in use (referenced at least twice). Pages that stop being used drift from its tail to the inactive list.

Inactive list

The list of pages that are candidates for eviction. Newly read file pages start at its head, a page used again while on it is promoted to the active list, and reclaim takes pages from its tail.

PG_referenced

A flag in each page's descriptor that remembers one recent use. When a page whose flag is already set is used again, Linux treats that as proof the page is busy.

PG_active

A flag in each page's descriptor that says the page is on an active list rather than an inactive one.

File-backed page

A page whose contents come from a file on disk, such as program code or file data in the page cache. A clean one can be evicted for free, because it can be read from the file again.

Anonymous page

A page with no file behind it, such as heap or stack memory. Before its frame can be reused, its contents must be written to swap space.

Page cache

The main memory Linux uses to keep copies of file data recently read or written, so the next access to that data needs no disk I/O.

Slab allocator

The kernel's allocator for small objects. It takes whole pages from the buddy allocator and cuts them into equal-size slots, keeping one cache of slots per object type or size.

Slab

One page (or a few contiguous pages) that a slab cache has cut into equal-size object slots. A slab is full, partial or empty depending on how many of its slots are in use.

SLUB

The slab implementation that became Linux's default in 2007 (kernel 2.6.23) and its only one since kernel 6.8 (2024), after the older SLOB and SLAB were removed.

Review questions with answers

1. In the classic three-level Linux page-table layout, what does each entry of the page middle directory point to?

- A. The page frame that holds the process's data
- B. One page of the page global directory
- C. One page of a page table
- D. The page table pointer register

Answer: C. One page of a page table

A global directory entry points to a page of the middle directory, a middle directory entry points to a page of a page table, and a page table entry gives the frame that holds the page.

2. Put the fields of a virtual address in the classic three-level Linux layout in order, from the leftmost (most significant) bits to the rightmost.

Correct order:

- 1. Global directory index
- 2. Middle directory index
- 3. Page table index
- 4. Offset

The walk starts at the top of the tree, so the leftmost bits index the global directory, then the middle directory, then the page table; the rightmost bits are the offset inside the page.

3. Because 32-bit x86 has only two levels of page tables, Linux needs a separate version of its page-table code for it that skips the middle directory.

Answer: False

Linux folds the middle level: it is defined with one entry and zero address bits, so the call that reads it simply hands back the entry it was given. The same generic code runs on two-, three-, four- and five-level hardware.

4. A 64-bit processor uses 4 KB pages and four levels of page tables. Every table fills exactly one 4 KB page with 8-byte entries. How many bits long are the virtual addresses these tables translate?

Answer: 48 bits

Each table holds $4,096 \div 8 = 512 = 2^9$ entries, so each level uses 9 bits: $4 \times 9 = 36$, plus a 12-bit offset for 4 KB pages, gives 48 bits (as on x86-64). A fifth level adds 9 more: 57 bits.

5. Match each Linux page-table level to its description.

PGD	The top table: one per process, kept in memory while it runs
P4D	The level added for five-level hardware, folded elsewhere
PUD	The level added so four-level 64-bit hardware fits
PMD	The middle level of the classic three-level layout
PTE	An entry describing one virtual page and its frame

The classic tree was PGD → PMD → PTE. The PUD (2005) and P4D (2017) levels were inserted for deeper hardware; on machines without them they are folded.

6. Linux's buddy allocator receives a request for 5 contiguous pages. How many page frames does the block it hands out contain?

Answer: 8 frames

Blocks come only in powers of two. The smallest order with $2^k \geq 5$ is $k = 3$, a block of 8 frames, so 3 of them go unused.

7. In a buddy allocator, a free block of order 2 (4 frames) starts at frame 12. At which frame does its buddy start?

Answer: 8

The buddy of an order- k block at frame f starts at $f \text{ XOR } 2^k = 12 \text{ XOR } 4 = 8$. Frames 8–11 and 12–15 are the two halves of the order-3 block 8–15, so if both are free they merge back into it.

8. Under the page-aging scheme of older Linux kernels, which page is the best candidate for replacement?

- A. The page with the highest age
- B. A page whose age has fallen to 0
- C. The page that was loaded the longest time ago
- D. A page whose use bit was set since the last sweep

Answer: B. A page whose age has fallen to 0

Each use raises a page's age and each background sweep lowers it, so age 0 means its uses have been outweighed by sweeps. Because the age counts uses, a page that was busy a while ago outranks one used once just now: a least-frequently-used flavour.

9. Which statements about Linux's split LRU page replacement (kernel 2.6.28 and later) are true?
- A. Pages are reclaimed from the tail of an inactive list.
 - B. File-backed pages and anonymous pages each have their own active and inactive lists.
 - C. A page on the inactive list must be used again while its PG_referenced flag is set before it is promoted to the active list.
 - D. A newly read file page goes straight to the head of the active list.
 - E. Active pages that stop being used drift to the inactive list.
 - F. Reading a big file once pushes the whole active list out of memory.

Answer: A, B, C, E

New file pages start on the inactive list, and only a second use there promotes them. That is why a one-time scan of a big file passes through the inactive list and leaves without disturbing the active pages. Reclaim always works from the inactive tail.

10. A file page is read for the first time and is then used heavily for a while before going idle. Put the stages of its life under the split LRU in order.

Correct order:

1. It is read from its file and joins the inactive list, its first use noted by PG_referenced
2. It is used again while on the inactive list and is promoted to the active list
3. It stops being used and is moved from the active tail back to the inactive list
4. It drifts down to the tail of the inactive list
5. Reclaim evicts it and its frame is reused

Pages enter at the inactive head, earn the active list by a second use, are demoted when they go idle, and are only ever reclaimed from the inactive tail.

11. Sort each statement under the allocator it describes.

Buddy allocator	Slab allocator
Hands out runs of 1, 2, 4, 8, ... contiguous page frames	Keeps a cache of equal-size slots for each object type or size
Halves a larger block when no free block of the right size exists	Serves kmalloc(100) from its 128-byte size class
Merges a freed block with its buddy when both are free	Gets the pages it cuts up from the other allocator

The buddy allocator manages whole page frames in power-of-two runs. The slab allocator sits on top of it, cutting pages into equal slots for small kernel objects.

12. Which slab implementation is Linux's default today, and since kernel 6.8 its only one?
- A. SLAB
 - B. SLOB
 - C. The buddy allocator
 - D. SLUB

Answer: D. SLUB

SLUB, the simpler "unqueued" design, became the default in 2.6.23 (2007). All three implementations offer the same interface, so the rest of the kernel did not change when SLOB and SLAB were removed.

8.5 Windows Memory Management

You should be able to

- Read the 32-bit Windows address map: name each region, give its address range and size, and explain why the two no-access regions exist.
- Explain the three states of an address-space region (available, reserved, committed) and why a program reserves first and commits later.
- Trace a thread stack growing through its guard page, and tell apart reserved addresses, commit charge and physical memory.
- Describe Windows resident set management: variable allocation with local scope, working-set growth and trimming, and how the standby and modified lists make trimmed pages cheap to recover.

The Windows virtual memory manager

The **virtual memory manager (VMM)** is the part of the Windows executive that runs virtual memory. For every page it settles three things: where the process's addresses may go (the address map), which addresses are backed (reserved or committed), and which pages are in physical memory now (the working set). Windows supports page sizes from 4 KB to 64 KB, depending on the processor; on x86 and x64 processors ordinary pages are **4 KB**.

The 32-bit address map

Each 32-bit process sees its own 4 GB of virtual addresses. By default the lower half is **user space** (private to the process: code, data, heap, stacks) and the upper half is **system space** (the same in every process, usable only in kernel mode).

Range	Region	Size	Why it exists
0x00000000–0x0000FFFF	Null-pointer region	64 KB	Never usable, so following a null pointer (or null plus a small offset) faults at once.
0x00010000–0x7FFEFFFF	User space	2 GB – 128 KB	The program's own space; an address works only if its page is committed.
0x7FFF0000–0x7FFFFFFF	Guard region	64 KB	No access. A pointer running off the top of user space faults here instead of reaching kernel memory, which simplifies the kernel's pointer checks.
0x80000000–0xFFFFFFFF	System space	2 GB	Executive, kernel, HAL, device drivers and kernel data. Mapped into every process, so the kernel can serve a system call inside the caller's own address space.

Worked size: user space is $0x7FFEFFFF - 0x00010000 + 1 = 0x7FFE0000$ bytes = 2,097,152 KB – 128 KB = **2,097,024 KB**. A boot option gives programs marked large-address-aware **3 GB** of user space: the line moves to 0xC0000000, the guard region becomes 0xBFFF0000–0xBFFFFFFF, and system space shrinks to **1 GB**. User-mode code touching the null-pointer region, the guard region or system space gets an **access**

violation. On 64-bit Windows each process has **128 TB** of user space (Windows 8.1 and later; earlier 64-bit versions gave 8 TB).

Regions and their three states

The VMM hands out user space in **regions**. A region starts on a **64 KB boundary** (the allocation granularity) and is a whole number of pages long; reserving even 4 KB uses up the rest of that 64 KB of addresses, so programs reserve big pieces and let a heap carve them up. Every page is in one of three states:

- **Available** (Windows calls it free): not claimed; touching it faults.
- **Reserved**: claimed for the process so nothing else lands there, but with nothing behind it: no frame, no paging-file space, no commit charge. Touching it faults.
- **Committed**: backed by storage the VMM has promised. Each page is in physical memory or on disk, in the **paging file** or in a mapped file. Committed private pages count toward the **commit charge**, which may not exceed the commit limit (roughly RAM plus the paging files). A committed page gets a frame only when first touched (a zero-filled frame, with no disk read).

The calls: **VirtualAlloc(MEM_RESERVE)** reserves; **VirtualAlloc(MEM_COMMIT)** commits reserved pages (both flags together reserve and commit in one call); **VirtualFree(MEM_DECOMMIT)** returns the backing but keeps the addresses reserved; **VirtualFree(base, 0, MEM_RELEASE)** returns the whole region, which becomes available again.

Why reserve first? A program can claim one long, contiguous range almost for free (1 MB for a thread stack, or a table that may grow) and commit pieces only as it needs them.

Worked example (4 KB pages): reserve 1 MB → claimed 1,024 KB, commit charge 0, physical 0. Commit the first 64 KB → commit charge 64 KB, physical still 0. Touch byte 0 → page 0 gets a frame, physical 4 KB. Write byte 70000 → page 17 ($70000 \div 4096 = 17.09$), only reserved → access violation; an exception handler commits page 17 (commit charge 68 KB) and the retried write gives it a frame (physical 8 KB). Decommit the first 64 KB → commit charge 4 KB, physical 4 KB. Release → everything back to 0.

How a thread stack grows: the guard page

A thread stack is a reserved region (1 MB by default) that grows toward lower addresses. At first only the top page is committed. Just below the used part sits a **guard page** (PAGE_GUARD): committed memory that raises a one-time exception when touched.

1. A call pushes the stack into the guard page and touches it.
2. Windows catches the exception and turns the page into an ordinary stack page (it gets a zero-filled frame).
3. Windows commits the next page down as the new guard page. Commit charge +4 KB, physical memory +4 KB.
4. When there is no room for another guard page (the bottom page of the reservation is never committed), Windows raises a **stack overflow** exception; if the program does not handle it, the process ends.

A frame bigger than a page could jump over the guard page onto a page that is only reserved, which is an access violation. Compilers prevent this with **stack probes** that touch each new page in order. Returns do

not decommit stack pages, so the commit charge records the deepest point reached. Trimming lowers physical memory, not the commit charge.

Resident set management: variable allocation, local scope

In Windows a process's **working set** is its set of pages in physical memory right now (its resident set). Each working set has a minimum and a maximum size; these are soft limits. On a page fault the VMM brings the page in, and then:

- **Memory plentiful:** the working set simply grows (it may pass its maximum).
- **Memory scarce:** the faulting process gives up one of **its own** pages that has not been used recently (Windows approximates LRU with the accessed bit), so its size stays the same. This is local scope: a busy process cannot steal frames from its neighbours on a fault.

Worked example: one process touches pages 1 2 1 3 2 4 1 2 5 1 3 2. With memory plentiful its working set grows to 5 pages and it takes **5 faults**, all hard first touches. Held to 3 pages with local LRU replacement, it takes **8 faults**: 5 hard plus 3 soft, because each dropped page was still waiting on the standby list.

Trimming and the second-chance lists

Available memory = free frames + standby frames. When it falls too low, the VMM **trims** working sets: it removes the least recently used pages of processes that are above their minimum. Trimmed pages are not thrown away at once (page buffering):

- An unchanged page goes to the **standby list**. Its contents are still valid; its frame can be reused immediately if memory is needed.
- A changed page goes to the **modified list**. The **modified page writer**, a system thread, writes such pages to the paging file (usually in batches) and then moves them to the standby list.
- If the owner touches a page that is still on either list, the VMM moves it back into the working set with no disk read: a **soft page fault**.
- Once a standby frame has been reused for something else, the old copy is gone, and the next touch is a **hard page fault** that reads the page from disk (the paging file or a mapped file).

Once every working set is near its minimum, trimming wins back little and processes fault on almost every touch: the road to thrashing. Since Windows 10, many pages headed for the paging file are first compressed in memory; since Windows 8, a suspended modern app can have its whole working set written out at once.

Key terms

Virtual memory manager (VMM)

The part of the Windows executive that runs virtual memory: it decides how each process's address space is handed out, which pages sit in physical memory, and when pages move between memory and disk.

User space

The lower part of a process's virtual address space. It is private to that process and is where the program's own code, data, heap and stacks live. On 32-bit Windows it is about 2 GB by default.

System space

The upper part of every process's virtual address space (2 GB by default on 32-bit Windows). It holds the executive, the kernel, the HAL and the device drivers, it looks the same in every process, and only kernel-mode code may touch it.

Null-pointer region

The lowest 64 KB of a 32-bit Windows address space (0x00000000–0x0000FFFF). It is never usable, so a program that follows a null pointer, or a null pointer plus a small offset, gets an access violation instead of quietly reading junk.

Guard region

A 64 KB block that can never be accessed, placed just below system space (0x7FFF0000–0x7FFFFFFF by default). A pointer that runs off the top of user space faults there instead of reaching kernel memory, which keeps the kernel's checks on user pointers simple.

Access violation

The exception Windows raises when a program touches an address it may not use: one that is not committed, one in a no-access region, or one in system space while the thread runs in user mode. If the program does not handle it, the process ends.

Allocation granularity

The boundary on which every new region of an address space must start: 64 KB on Windows. A region's size is always a whole number of pages.

Available region

Addresses the process has not claimed at all (Windows itself calls them free). Touching one raises an access violation.

Reserved region

Addresses set aside for a process so that nothing else is placed there, but not yet backed by physical memory or paging-file space. Reserving adds nothing to the commit charge; touching a reserved page raises an access violation.

Committed region

Addresses that have backing storage: each page is either in physical memory or on disk, in the paging file or in a mapped file. Committing private pages adds to the commit charge.

Commit charge

The total amount of committed private memory Windows has promised to back. It may not exceed the commit limit, roughly physical memory plus the paging files, so a commit can fail even while plenty of address space is free.

Paging file (page file)

A file on disk where Windows writes committed pages that must leave physical memory, so that they can be read back when they are needed again.

Guard page (PAGE_GUARD)

A committed page marked to raise a one-time exception the first time it is touched. Windows keeps one just below the used part of each thread's stack to notice that the stack needs to grow.

Working set

In Windows, the set of a process's pages that are in physical memory right now (its resident set). Section 8.2 uses the same words for $W(t, \Delta)$, the pages referenced recently; Windows uses them for the resident

pages.

Working-set trimming (trimming)

Removing pages that have not been used recently from processes' working sets when available memory runs low. The pages are not lost at once: they move to the standby or modified list.

Standby list

The list of frames holding pages that left a working set unchanged (or that have already been written to disk). Their contents are still valid, so the owner can take a page back without any disk I/O, but the frame may be reused for something else whenever memory is needed.

Modified list

The list of frames holding pages that left a working set after being changed. They must be written to disk before their frames can be reused; until then the owner can take them back at once.

Soft page fault (soft fault)

A page fault Windows resolves without reading the disk, for example by moving the page back from the standby or modified list into the working set, or by handing out a fresh zero-filled frame.

Hard page fault (hard fault)

A page fault that must read the page from disk, from the paging file or from a mapped file such as the program file. It is far slower than a soft fault.

Modified page writer

A Windows system thread that writes pages from the modified list to disk (the paging file or their mapped file) and then moves them to the standby list.

Review questions with answers

1. On 32-bit Windows with the default layout, a program reads address `0x7FFF0010`. Which region is that, and what happens?

- A. User space: the read works if the page is committed
- B. System space: the read works only in kernel mode
- C. The guard region: the read raises an access violation
- D. The null-pointer region: the read returns zero

Answer: C. The guard region: the read raises an access violation

Default map: null-pointer region `0x00000000–0x0000FFFF`, user space `0x00010000–0x7FFEFFFF`, guard region `0x7FFF0000–0x7FFFFFFF`, system space `0x80000000–0xFFFFFFFF`. The guard region can never be accessed.

2. In the default 32-bit Windows layout, user space runs from `0x00010000` to `0x7FFEFFFF`. How many KB is that? (2 GB = 2,097,152 KB.)

Answer: 2097024 KB

$0x7FFEFFFF - 0x00010000 + 1 = 0x7FFE0000$ bytes: the lower 2 GB minus the 64 KB null-pointer region and the 64 KB guard region. $2,097,152 - 128 = 2,097,024$ KB.

3. Reserving a 1 MB region (VirtualAlloc with MEM_RESERVE) raises the process's commit charge by 1 MB.

Answer: False

Reserving only claims addresses; nothing backs them, so the commit charge does not move. It rises only when pages are committed.

4. A program reserves 1 MB, commits the first 5 pages (4 KB each), and then touches 2 of those 5 pages. By how many KB does its commit charge grow?

Answer: 20 KB

The commit charge counts committed pages whether or not they have been touched: $5 \times 4 \text{ KB} = 20 \text{ KB}$. Only the 2 touched pages (8 KB) take physical memory, and the rest of the reservation costs nothing.

5. Sort each description into the state of the pages it describes.

Available	Reserved	Committed
Addresses no part of the program has claimed A range right after VirtualFree with MEM_RELEASE	The range VirtualAlloc with MEM_RESERVE just returned The part of a thread stack below its guard page	Pages counted in the commit charge A page whose contents now sit in the paging file

Available addresses are unclaimed. Reserved ones are claimed but have no backing, like the unused part of a stack below its guard page. Committed pages are backed, in memory or in the paging file, and count toward the commit charge.

6. Put the life of one region in order.

Correct order:

1. The addresses are available
2. VirtualAlloc with MEM_RESERVE claims the range
3. VirtualAlloc with MEM_COMMIT backs the first pages
4. The first touch of a committed page gets it a zero-filled frame
5. VirtualFree with MEM_DECOMMIT returns the backing but keeps the addresses
6. VirtualFree with MEM_RELEASE makes the range available again

Reserve, then commit, then touch: frames arrive last, only for pages actually used. On the way back, decommit returns the backing and release returns the addresses.

7. A function with a 16 KB local array first writes at the far end of its frame, jumping over the thread's guard page onto a stack page that is reserved but not committed. What happens?

- A. Windows commits the page automatically, as it does for the guard page
- B. A soft page fault brings the page in from the standby list
- C. The write works, because the page belongs to the stack's reservation
- D. An access violation: a reserved page has nothing behind it

Answer: D. An access violation: a reserved page has nothing behind it

A stack grows one page at a time because the guard page is always touched first. For frames bigger than a page, compilers insert stack probes that touch each page in order.

8. Which resident set policy does Windows use?

- A. Fixed allocation, local scope
- B. Variable allocation, local scope
- C. Variable allocation, global scope
- D. Fixed allocation, global scope

Answer: B. Variable allocation, local scope

Working sets grow and shrink (variable allocation), and a faulting process that must give up a page gives up one of its own (local scope).

9. Available memory has fallen too low. Which of these does the Windows VMM do? Select all that apply.

- A. Trims working sets, removing pages that have not been used recently
- B. Moves unchanged trimmed pages to the standby list
- C. Moves changed trimmed pages to the modified list, to be written to disk before their frames are reused
- D. Gives back the trimmed pages' commit charge
- E. Ends the process with the largest working set

Answer: A, B, C

Trimming only takes pages out of physical memory; they stay committed, so the commit charge does not change. Unchanged pages go to the standby list, changed ones to the modified list, and no process is ended.

10. A soft page fault has to read the page from the paging file.

Answer: False

A soft fault is resolved without disk I/O, for example by moving the page back from the standby or modified list into the working set. A fault that must read the disk is a hard fault.

11. Match each Windows memory structure to what it holds.

Working set	A process's pages that are in physical memory now
Standby list	Pages removed unchanged, whose frames can be reused at once
Modified list	Pages removed after being changed, not yet written out
Paging file	Disk space for committed pages that left physical memory

The working set is what is resident. Trimmed pages wait on the standby list (clean) or the modified list (dirty) until the modified page writer saves the dirty ones to the paging file.

12. On 32-bit Windows booted with the 3 GB user-space option, how many GB of system space does each process's address map keep?

Answer: 1 GB

The 4 GB address space is split at 0xC0000000 instead of 0x80000000: 3 GB of user addresses below the line, leaving 0xC0000000–0xFFFFFFFF, which is 1 GB, for the system.

8.6 Android Memory Management

You should be able to

- Explain what Android keeps from Linux memory management and which three phone constraints (small RAM, flash that wears out, constant app switching) push it to add more.
- Describe how ashmem (today memfd) shares memory by passing a file descriptor and lets the kernel purge unpinned parts, and how ION (today DMA-BUF heaps) gives devices shared buffers from pools set aside at boot.
- Explain why phones do not swap to flash, and how dropping clean pages, zram compression, trim requests and the low memory killer free RAM instead.
- Predict which process the low memory killer ends first, and what the user sees when a killed app is reopened.
- Describe an app's side of the bargain: stay under its heap limit, let the garbage collector work, shrink caches in `onTrimMemory`, and save state for a cold start.

Android memory management: Linux, stretched for phones

Android runs on Linux and keeps all of its memory management: virtual address spaces, page tables, demand paging, the page cache, the buddy and slab allocators. It adds a few pieces because a phone differs from a PC in three ways:

- **Small RAM, many apps.** A few gigabytes are shared by the system and dozens of apps the user switches between; every switch back should feel instant.
- **Flash storage, not a disk.** Each flash block survives a limited number of rewrites (its **write endurance**), and writing costs time and battery, so phones normally have no swap area on storage.
- **Hardware that shares big buffers.** Camera, GPU, video engine and display pass large images to each other by DMA, many times a second.

Piece	Where	Job
Activity Manager	framework (system server)	Ranks every app process by importance, writes an <code>oom_score_adj</code> for it, sends <code>onTrimMemory</code>
lmkd	user-space daemon	Watches memory pressure and kills the process with the highest <code>oom_score_adj</code>
ashmem, now memfd	kernel	Shared memory passed between processes as a file descriptor
ION, now DMA-BUF heaps	kernel	Buffer pools for hardware, shared without copying

Freeing RAM without a swap area

What the kernel can do with a page depends on its kind:

- **File-backed page** (code, resources, mapped files): if clean, it is simply dropped, because the file on flash still holds a copy; it is read back when needed. A changed (dirty) one is first written back to its

file. Android does this constantly, so “Android does no paging” is false.

- **Anonymous page** (heap, stack): no file holds it. A desktop writes it to a swap area on disk. A phone avoids writing it to flash, so it can only stay in RAM, be compressed into **zram**, be freed by the app itself, or vanish when its process is killed.

zram is a swap device made of RAM: swapped-out pages are compressed, often to about a third, and kept in memory. It costs processor time but no flash writes. Worked example: 600 MB of cold pages compressed 3:1 become 200 MB, freeing $600 - 200 = 400$ MB; likewise 6 heap pages fit in 2 frames, freeing 4.

ashmem: shared memory by descriptor (now memfd)

1. Process A asks the kernel for a named region and receives a **file descriptor** for it (say number 23 in A's table).
2. A maps the region into its address space and writes into it with ordinary stores.
3. A passes the descriptor to process B, usually inside a Binder call. B receives its own descriptor number (say 41): different number, same region. No data is copied.
4. B maps the region and sees the very same physical frames, so a write by either is seen by both.

The owner can **unpin** parts it could rebuild; under memory pressure the kernel may **purge** them (they then read as zeros). When the owner pins them again, the kernel reports whether they were purged, so the owner knows to regenerate the contents. The region is freed when the last holder lets go of it, even after a crash. System V shared memory, which outlives its processes unless removed, would leak on a phone that kills processes constantly, so Android omits it. Newer Android versions use the standard Linux **memfd** (`memfd_create`) instead; it has no pinning.

ION: buffers for hardware (now DMA-BUF heaps)

Some devices need one **physically contiguous** block, or memory from a particular region. After hours of use, free frames are scattered (external fragmentation): for example 10 free frames whose longest unbroken run is 3 cannot satisfy a request for 6 contiguous frames. ION manages several pools (**heaps**), some **set aside at boot** for a device, so the camera's block always exists; the price is that apps cannot use that memory (Linux's CMA softens this by lending it to movable pages while the device is idle). ION hands out each buffer as a file descriptor, so camera, GPU and display pass one buffer along (**zero-copy sharing**). Since Android 12 the mainline Linux **DMA-BUF heaps** do this job, and ION has been removed from the mainline kernel.

Why copies matter: one frame = width × height × 4 bytes. A 1920×1080 frame is 8,294,400 bytes ≈ 8.3 MB; copying it twice per frame at 30 frames per second moves about $8.3 \times 2 \times 30 \approx 498$ MB every second. Sharing one buffer moves 0 MB/s.

Importance, trimming and the low memory killer

The Activity Manager ranks every app process and writes its rank as an **oom_score_adj** (−1000 to 1000; higher means more expendable):

Importance	Typical oom_score_adj	Example
Foreground	0	the app the user is touching
Visible	100	a navigation app in a floating window
Perceptible	200	music playing
Service	about 500	a background upload
Cached	900–999, older = higher	apps the user left, kept for a quick return

As free memory falls, Android responds in order: (1) the kernel drops clean pages and compresses cold anonymous pages into zram; (2) apps are asked to shrink through **onTrimMemory**; (3) **lmkd** kills the process with the highest oom_score_adj (the cached app used longest ago) and repeats until enough memory is free; only in deeper trouble does it reach service, then visible processes, and the foreground app last. The killer began as a driver inside the kernel with fixed free-memory thresholds; it now runs as the user-space daemon **lmkd**, which since Android 10 uses the kernel's **pressure stall information (PSI)**.

Worked example (apps share 2,700 MB; trim line 600 MB free, kill line 300 MB free): the user opens a Game needing 800 MB with 650 MB free, leaving 150 MB short. The cached Camera trims 150 MB (0 MB free). **lmkd** kills Chat (cached, adj 920, 250 MB → 250 MB free), then Browser (adj 910, 350 MB → 600 MB free).

Reopening an app whose process still exists lets Android skip creating one: a **hot start** if the existing activity just comes back to the front, a **warm start** if the activity (the screen) must be recreated. Either is usually much faster, though the app may still rebuild caches or redraw its screen. Reopening a killed app is a **cold start**: a new process whose activity receives its **saved instance state**, so the screen looks as the user left it.

An app's four memory duties

- **Stay under the heap limit.** Each app's Java or Kotlin heap has a device-set ceiling (`ActivityManager.getMemoryClass()`, often 128–512 MB). Allocating past it throws `OutOfMemoryError`. It protects the phone from one leaky app.
- **Let the garbage collector work.** ART frees objects that nothing references. An object whose last reference is dropped is garbage: it still takes space until the next collection. Before giving up, the runtime collects garbage: with a 256 MB limit, 200 MB live and 30 MB garbage, a 40 MB request succeeds ($200 + 40 = 240$ MB). Only if live data plus the request exceeds the limit is `OutOfMemoryError` thrown.
- **Answer onTrimMemory.** Release caches that can be rebuilt. `TRIM_MEMORY_UI_HIDDEN` means the app's screens just left the display; `TRIM_MEMORY_BACKGROUND` means it is in the cached list and may be killed next. Higher levels are more serious.
- **Save state early.** A kill runs no app code at all (not even `onDestroy()`). Save screen state in `onSaveInstanceState()` or `onStop()`, keep the bundle small, and keep real data in a database or file.

Standard Linux vs added by Android

Standard Linux: demand paging, the page cache, the buddy allocator, zram, PSI, `memfd_create`, DMA-BUF heaps. Added by Android: `ashmem`, `ION`, the low memory killer (now **lmkd**), and app callbacks such as

onTrimMemory.

Key terms

Write endurance

The limited number of times each block of flash storage can be erased and rewritten before it wears out. Constant heavy writing, such as paging to a swap area, uses it up.

File-backed page

A page whose contents came from a file on storage, such as program code or an image resource. If it has not been changed (a clean page), the kernel can simply drop it and read it back from the file later.

Anonymous page

A page of data created while a program runs, such as heap objects, with no file behind it on storage. It cannot just be dropped: it must stay in RAM, go to swap (on a phone usually zram), or vanish with its process.

zram

A Linux feature that makes a swap device out of RAM itself: pages being swapped out are compressed (often to about a third of their size) and kept in memory, so cold data takes less room and nothing is written to flash.

Anonymous shared memory (ashmem)

An Android addition to the Linux kernel: a named block of shared memory handed out as a file descriptor. A process that receives the descriptor can map the same memory, and parts the owner has unpinned can be reclaimed by the kernel under pressure.

Unpinned region

Part of an ashmem block that its owner has marked as not needed for now. Under memory pressure the kernel may purge (discard) it; when the owner pins it again, the kernel says whether the contents survived.

memfd

The standard Linux way (the `memfd_create` call) to make an anonymous file that lives in RAM and get a file descriptor for it, which can be passed and mapped just like ashmem. Newer Android versions use it in place of ashmem.

ION

An Android memory allocator for buffers shared by processes and hardware such as the graphics processor, display controller and camera. It manages several pools (heaps), some set aside at boot, and hands out each buffer as a file descriptor.

DMA-BUF heaps

The mainline Linux mechanism that replaced ION from Android 12 on: named memory pools that hand out shareable DMA-BUF buffers. Same idea as ION, in standard Linux form.

Zero-copy sharing

Handing a buffer from one process or device to the next by sharing the same physical memory (usually through a file descriptor) instead of copying its bytes.

Memory pressure

How hard the system is struggling to find free RAM: how little is free, how fast the kernel must reclaim, and how long tasks stall waiting for memory.

Pressure stall information (PSI)

A Linux kernel feature that reports the share of recent time in which tasks were stalled waiting for a resource such as memory. Since Android 10, Imkd uses it to decide when to kill.

Low memory killer (LMK)

The part of Android that, when RAM runs short and asking apps to free memory is not enough, ends whole processes, least important first. It began as a driver inside the Linux kernel; today the user-space daemon Imkd does the job.

Imkd (low memory killer daemon)

The user-space program that replaced the kernel low-memory-killer driver. It watches the kernel's memory pressure signals and, when pressure is too high, kills the process with the highest oom_score_adj.

oom_score_adj (OOM adjustment score)

A number from -1000 to 1000 that Android's framework sets for every app process to say how expendable it is: 0 for the foreground app, 100 for a visible one, 200 for one the user can perceive (music playing), about 500 for a background service, 900 to 999 for cached apps. The killer picks the highest score first.

onTrimMemory

A callback Android makes into an app when its screens become hidden or when it sits in the cached list, so the app can release caches it is able to rebuild later.

Heap limit (memory class)

The largest size an app's managed (Java or Kotlin) heap may grow to, fixed per device and often a few hundred megabytes. Going past it throws an OutOfMemoryError.

Garbage collector (GC)

The part of the Android runtime that finds objects the app can no longer reach and frees their memory automatically, so programmers never free objects by hand.

Cold start

Launching an app whose process is not in memory: Android must create a new process and the app must rebuild its screens, using any saved state. If the process still exists, Android skips creating it: a warm start must still recreate the activity (the screen), while a hot start just brings the existing activity back to the front. Both are usually much faster, though the app may still rebuild caches or redraw its screen.

Saved instance state

A small bundle of screen state (text being typed, scroll position) that an activity hands to Android before it may be destroyed, and gets back when it is recreated after its process was killed.

Review questions with answers

1. Why do Android phones normally have no swap partition on their flash storage?

- A. The Linux kernel used on phone processors has no support for moving pages in and out
- B. Android gives each app a fixed physical memory area, so it never needs virtual memory
- C. Flash wears out with repeated rewriting, and heavy paging would also cost battery
- D. Flash storage can only be read in order from start to end, so random page reads fail

Answer: C. Flash wears out with repeated rewriting, and heavy paging would also cost battery

Flash has limited write endurance, and writing costs time and energy. So phones avoid writing anonymous pages to a swap area on flash, and use zram, trimming and killing instead.

2. Android never moves pages between RAM and flash storage.

Answer: False

Clean file-backed pages (program code, resources, mapped files) are dropped under pressure and read back from flash when needed. Only writing anonymous pages to a flash swap area is avoided.

3. A phone compresses 600 MB of cold anonymous pages into zram, which shrinks them to one third of their size. How many MB of RAM does this free?

Answer: 400 MB

The 600 MB become $600 \div 3 = 200$ MB of compressed data, still kept in RAM. RAM freed = $600 - 200 = 400$ MB.

4. Put Android's responses to falling free memory in order, from the first and gentlest to the last resort.

Correct order:

- 1. The kernel drops clean file-backed pages and compresses cold anonymous pages into zram
- 2. Apps are asked to shrink through `onTrimMemory`
- 3. `lmkd` kills the cached process with the highest `oom_score_adj`
- 4. If memory is still short, `lmkd` kills more important processes (service, then visible)

Cheap reclaim that loses nothing comes first, then voluntary shrinking by apps, and only then kills, starting with the process the user will miss least.

5. Memory is critically low. Which process does `lmkd` kill first?

- A. A mail app cached five minutes ago (`oom_score_adj` 900)
- B. A browser cached two hours ago (`oom_score_adj` 930)
- C. A music app playing in the background (`oom_score_adj` 200)
- D. The game the user is playing right now (`oom_score_adj` 0)

Answer: B. A browser cached two hours ago (`oom_score_adj` 930)

`lmkd` kills the process with the highest `oom_score_adj`. Among cached apps, the one used longest ago gets the highest score, so the two-hour-old browser goes first.

6. Sort each mechanism: standard Linux feature, or something Android added?

Standard Linux	Added by Android
Demand paging Buddy page allocator memfd_create DMA-BUF heaps	ashmem ION lmkd onTrimMemory callbacks

Android keeps the whole Linux memory system. It added ashmem, ION, the low memory killer (now lmkd) and app callbacks. Newer versions swap its own pieces for mainline ones: memfd for ashmem, DMA-BUF heaps for ION.

7. Match each mechanism to its job.

ashmem	Shared memory passed between processes as a file descriptor
ION	Buffer pools for camera, GPU and display, some reserved at boot
zram	Compressed swap area kept in RAM
lmkd	Kills the process with the highest oom_score_adj
onTrimMemory	Lets an app release caches it can rebuild

Two mechanisms share memory (ashmem between processes, ION with hardware), one stretches RAM (zram), one asks apps to shrink (onTrimMemory), and one is the last resort (lmkd).

8. Which statements about ashmem are true?

- A. A process that receives the descriptor maps the same physical frames as the creator
- B. Sharing a region means passing its file descriptor, often in a Binder call
- C. The receiver always gets the same descriptor number as the sender
- D. Unpinned parts of a region may be purged by the kernel under memory pressure
- E. The memory stays allocated after every process holding it has exited

Answer: A, B, D

Descriptor numbers are per process, so the receiver usually gets a different number for the same region. The region is freed once the last holder closes it, which is why Android prefers it to System V shared memory.

9. A user returns to a chat app that lmkd killed while it sat in the cached list. What happens?

- A. The old process wakes up and continues from the exact instruction where it was stopped
- B. Android tells the user that the app crashed, then opens it on its very first screen
- C. Android reads the old process's memory back from the swap area on flash storage
- D. A new process is started (a cold start) and receives the saved instance state

Answer: D. A new process is started (a cold start) and receives the saved instance state

A kill destroys the process, but Android keeps the activity's saved instance state. On return it creates a new process and passes that state to onCreate, so only a short delay gives the kill away.

10. When Imkd kills a cached app, Android first calls the app's `onDestroy()` so it can save its data.

Answer: False

A kill runs no app code at all: the process simply vanishes. That is why apps must save state in `onSaveInstanceState()` or `onStop()`, and keep real data in a database or file.

11. A 1920×1080 image at 4 bytes per pixel is about 8.3 MB. If a camera pipeline copied every frame twice and ran at 30 frames per second, about how many MB would it copy per second?

Answer: 498 MB/s

$8.3 \text{ MB} \times 2 \text{ copies} \times 30 \text{ frames} = 498 \text{ MB every second}$. Sharing one ION (DMA-BUF) buffer through a file descriptor cuts this to zero.

12. An app's heap limit is 256 MB. It holds 200 MB of live objects and 30 MB of garbage, then asks for a 40 MB object. What happens?

- A. It fails at once with `OutOfMemoryError`, since $230 + 40 = 270 \text{ MB}$ is over the limit
- B. The runtime collects the garbage first; $200 + 40 = 240 \text{ MB}$ fits, so it succeeds
- C. Imkd kills the app at once, because its heap is almost completely full
- D. The 30 MB of garbage is compressed into zram to make room for the object

Answer: B. The runtime collects the garbage first; $200 + 40 = 240 \text{ MB}$ fits, so it succeeds

When an allocation would pass the limit, the garbage collector runs first. $200 \text{ MB live} + 40 \text{ MB} = 240 \text{ MB} \leq 256 \text{ MB}$, so it fits. `OutOfMemoryError` comes only if the live data plus the new object would still exceed the limit.

Chapter 9 · Uniprocessor Scheduling

One processor, many ready processes. Who runs next decides how fast, fair and responsive the whole system feels.

On a computer with one processor, only one process can run at a time, so the operating system must keep choosing which one runs next. This chapter separates scheduling into long-term, medium-term and short-term decisions, then focuses on the short-term scheduler (the dispatcher). It defines the goals a scheduler is judged by, such as response time, turnaround time, throughput and fairness, and then lets you run the classic algorithms side by side: first-come-first-served, round robin, shortest process next, shortest remaining time, highest response ratio next and feedback. It ends with fair-share scheduling and the traditional UNIX scheduler.

Learning objectives

- Distinguish long-term, medium-term and short-term scheduling.
- Define user-oriented and system-oriented scheduling criteria and the idea of priorities.
- Simulate FCFS, round robin, SPN, SRT, HRRN and feedback scheduling and compute turnaround and normalized turnaround times.
- Explain preemptive versus nonpreemptive scheduling and the effect of the time quantum.
- Describe fair-share scheduling and the traditional UNIX priority calculation.

How Chapter 9 fits together

One processor runs one process at a time. Section 9.1 splits the choice into three levels, each tied to seven-state model transitions:

- **Long-term scheduling** admits new programs as processes (New → Ready or Ready/Suspend). It runs rarely and sets the degree of multiprogramming.
- **Medium-term scheduling** is part of swapping: it suspends processes to disk and brings them back (Ready ↔ Ready/Suspend, Blocked ↔ Blocked/Suspend).
- **Short-term scheduling**, the **dispatcher**, picks which ready process runs next (Ready → Running). It runs most often, on interrupts, system calls and signals.

Time-outs, event waits, event occurrences and exits are not scheduling decisions, though most of them call the dispatcher. In the overview game, A needs 4 ticks, B 1 and C 2, all ready at time 0. Running the shortest first (B, C, A) gives waiting times 0, 1 and 3, an average of $4 \div 3 \approx 1.33$ ticks; the worst of all 105 schedules averages $11 \div 3 \approx 3.67$.

The algorithms (9.2)

Criteria are user-oriented (turnaround, response time, deadlines) or system-oriented (throughput, utilization, fairness). A policy has a selection function and a decision mode: **nonpreemptive** (runs until it finishes or blocks) or **preemptive**.

Policy	Picks	Preemptive?
FCFS	the process that has waited longest	no
Round robin	the next in the queue, for one time quantum	yes, at the end of a quantum
SPN	the shortest expected service time	no
SRT	the least expected work left	yes, when a shorter job arrives
HRRN	the highest ratio (waiting + service) ÷ service	no
Feedback	the head of the highest non-empty queue; a used quantum drops a level	yes, at the end of a quantum

For the workload A (arrives 0, needs 5), B (1, 3), C (2, 2), D (4, 2), one tick per letter: FCFS gives AAAAABBBCCDD; round robin ($q = 1$) ABACBADCBADA; SPN AAAAACCDDBBB; SRT ABBBCCDDAAAA (B cuts in at $t = 1$); HRRN AAAAACCBBD (ratios at $t = 5$: B 2.33, C 2.50, D 1.50; at $t = 7$: B 3.00, D 2.50); feedback ($q = 1$) ABCADBCDABAA. Fair-share scheduling divides the processor between groups of users.

Traditional UNIX (9.3)

Classic UNIX ran round robin within priority levels and each second recomputed every priority from a base value, recent processor use (decayed over time) and the nice value, so processor-hungry processes sink and interactive ones stay responsive.

Why it matters

Process states, lock waits and page faults all end up as processes entering and leaving the ready queue: scheduling ties the course together.

9.1 Types of Processor Scheduling

You should be able to

- Explain what processor scheduling is for and name the four kinds of scheduling decision.
- Place long-term, medium-term and short-term scheduling on the seven-state process model and the queuing diagram.
- Explain how long-term scheduling controls the degree of multiprogramming, and when and which jobs it admits.
- Explain why medium-term scheduling is part of swapping, and list the events that invoke the short-term scheduler.
- Compare how often each scheduler runs, and why the short-term scheduler must be fast.

What processor scheduling is for

A system may hold many processes but only one processor. **Processor scheduling** assigns processes to the processor over time so that the system meets its goals, such as quick **response time**, high

throughput (work finished per unit of time) and efficient use of the processor. The OS does this with four different kinds of decision, made on very different time scales.

Kind	The question it answers	State moves it makes	How often
Long-term	Should a new program be added to the pool of processes to be executed?	New → Ready, New → Ready/Suspend	Rarely
Medium-term	Should a process be partly or fully in main memory, or swapped out to disk?	Ready ↔ Ready/Suspend, Blocked ↔ Blocked/Suspend	Now and then
Short-term	Which ready process does the processor run next? (the dispatcher)	Ready → Running	Most often: many times a second
I/O	Which pending I/O request does an available device handle next?	none (it orders a device's request queue)	Each time a device finishes a request

The schedulers on the seven-state model

The three processor schedulers nest like rings around the seven-state process model. The **innermost** ring is short-term scheduling: Ready, Running and Blocked, the processes already in main memory. Around it is **medium-term** scheduling: the suspended states Ready/Suspend and Blocked/Suspend. The **outermost** ring is long-term scheduling: New and Exit. To reach the processor a process passes every ring from the outside in: it is admitted, it is in main memory, and only then is it chosen to run. The inner the ring, the more often its decision is made.

Other arrows are **events**, not scheduling choices: *time-out* (Running → Ready), *event wait* (Running → Blocked, e.g. an I/O request or semWait), *event occurs* (Blocked → Ready, or Blocked/Suspend → Ready/Suspend on disk) and *release* (Running → Exit). Most of them wake the short-term scheduler. Only processes in main memory can be dispatched, so a Ready/Suspend process must first be swapped in.

The queuing view

The same system can be drawn as a network of queues:

- **Batch jobs** wait (usually on disk) until long-term scheduling admits them into the **ready queue**, or into the **Ready/Suspend queue** when main memory is full.
- **Interactive users** do not wait in a batch queue: a login creates a process that goes straight into the ready queue, as long as the system is not saturated.
- Short-term scheduling dispatches from the ready queue. A **time-out** returns the process to the back of the ready queue, an **event wait** moves it to a **blocked queue**, and **release** sends it to Exit. When the event occurs it rejoins the ready queue.
- Medium-term scheduling moves processes between the ready queue and the Ready/Suspend queue, and between the blocked queue and the Blocked/Suspend queue. If the event a swapped-out process waits for occurs, it moves from Blocked/Suspend to Ready/Suspend.

Long-term scheduling

Long-term scheduling decides which programs are admitted to the system, so it controls the **degree of multiprogramming**: how many processes are active at once. More processes give the processor more

chances to find work, but each process then gets a smaller share of it, and every process needs main memory.

Worked example. In a simplified model where each process waits for I/O a fraction p of the time, independently of the others, the processor is idle only when all n processes wait at once, so utilization = $1 - p^n$. With $p = 0.6$: $n = 3$ gives $1 - 0.216 = 78.4\%$ busy, about 26.1% per process; $n = 5$ gives $1 - 0.078 = 92.2\%$ busy, about 18.4% per process. Each extra process adds less while every share shrinks. ($p = 0.5$, $n = 4$: $1 - 0.0625 = 93.75\%$.)

The long-term scheduler makes two decisions:

- **When** can the OS take on another process? For example each time a job ends, or when the processor has been idle for more than some fraction of the time (a rule such as “admit while idle more than 10%”).
- **Which** job comes next? Possible rules: first come first served, priority, expected execution time, or I/O requirements. A common goal is a **balanced mix** of processor-bound jobs (long processor bursts, little I/O) and I/O-bound jobs (short bursts, much I/O), or favouring jobs that need devices nobody is using right now.

The mix in numbers. One processor and one disk, each serving jobs in arrival order, for 30 time units; processor-bound jobs compute 5 units then use the disk for 1, I/O-bound jobs compute 1 then use the disk for 4. Three processor-bound jobs: processor 100% busy, disk only 17%. Three I/O-bound jobs: processor 33%, disk 97%. One processor-bound then two I/O-bound jobs (in that order): processor 90%, disk 83%. The mix keeps both resources working.

Time-sharing. A user logging in creates a process. The OS accepts every authorized user until the system is **saturated**, then refuses new logins with a message such as “system full, try later”. A batch job can wait to be admitted later; a person at a keyboard cannot.

Common mistake: more jobs is not always more work done. Once the processor is nearly always busy, extra jobs mostly shrink every share and crowd memory.

Medium-term scheduling

Medium-term scheduling is part of the **swapping** function: it decides when to swap a process out to disk and when to swap one back in. Its decisions manage the degree of multiprogramming and, on a system without virtual memory, main memory itself, because a process must then fit entirely in free memory before it can be swapped in. The usual way to free memory is to swap out a **Blocked** process, which cannot use its memory while it waits anyway. When everything in memory is blocked and ready work waits on disk, swapping keeps the processor busy.

Short-term scheduling

Short-term scheduling, carried out by the **dispatcher**, makes the fine-grained decision of which ready process runs next. It runs far more often than the other two. It is invoked whenever an event may block the current process or offer a chance to preempt it:

- **Clock interrupts** (for example, the end of a time slice)
- **I/O interrupts** (a device finished, so a waiting process may now be ready)
- **Operating system calls** (the caller may have to wait)

- **Signals**, for example semaphore operations such as semWait and semSignal

It may run hundreds of times a second, so it must be fast: time spent choosing is time no process gets.

How often each one runs

Long-term scheduling runs rarely (when jobs arrive or finish, or users log in), medium-term scheduling more often (when memory pressure changes), and short-term scheduling most often. In a small run of six jobs (three memory slots, 3-unit time slice) a sensible policy made 6 long-term, 2 medium-term and 13 short-term decisions: each job is admitted once, but every time-out, I/O request and exit needs a new dispatch.

Key terms

Processor scheduling

Deciding which process gets the processor, and when, so the system meets its goals, such as quick response, high throughput and efficient use of the processor.

Long-term scheduling

The decision whether to add a newly submitted program to the pool of processes the OS will run. It controls the degree of multiprogramming and runs rarely.

Medium-term scheduling

The decision whether a process should be partly or fully in main memory, or swapped out to disk. It is the swapping part of the OS.

Short-term scheduling

The decision which ready process in main memory runs on the processor next. It is carried out by the dispatcher and runs many times a second.

I/O scheduling

The decision which pending I/O request an available device (such as a disk) handles next.

Degree of multiprogramming

The number of processes active in the system at the same time, sharing the processor and main memory.

Admission

Accepting a newly created process into the pool that competes for the system: the move from New to Ready, or from New to Ready/Suspend when memory is full.

Batch job

A program submitted to run with no user interacting with it. It waits (usually on disk) until the long-term scheduler admits it.

Processor-bound process

A process that mostly computes: it uses long stretches of processor time and does little I/O. Also called CPU-bound.

I/O-bound process

A process that mostly waits for I/O: it runs for short bursts, then blocks for a device, again and again.

Saturation

The point where a system already serves as many users or processes as it can with acceptable performance. A saturated time-sharing system refuses new logins.

Queuing diagram

A picture of the OS as a network of waiting lines (queues). Arrows show the events and scheduling decisions that move a process from one queue to another.

Review questions with answers

1. Which kind of scheduling decides which Ready process in main memory gets the processor next?

- A. Long-term scheduling
- B. Medium-term scheduling
- C. Short-term scheduling
- D. I/O scheduling

Answer: C. Short-term scheduling

Short-term scheduling, carried out by the dispatcher, makes the Ready → Running decision, many times a second.

2. A newly created batch job goes straight into the Ready/Suspend state instead of Ready. Which kind of scheduling made that decision, and why?

- A. Long-term: it admitted the job while main memory was full
- B. Medium-term: it swapped the job out because it had low priority
- C. Short-term: it preempted the job when its time slice ended
- D. I/O scheduling: the job is waiting for the disk

Answer: A. Long-term: it admitted the job while main memory was full

New → Ready and New → Ready/Suspend are both admissions, made by long-term scheduling. When memory is full, the admitted process starts out on disk.

3. Medium-term scheduling runs more often than short-term scheduling, because processes are swapped in and out constantly.

Answer: False

Swapping is slow disk work, so it happens now and then. Short-term scheduling runs on every clock interrupt, I/O interrupt, system call and signal: by far the most often.

4. Which of these events invoke the short-term scheduler? Select all that apply.

- A. A clock interrupt
- B. An I/O interrupt
- C. A process making an operating system call
- D. A semaphore operation such as semSignal
- E. A new batch job arriving in the batch queue

Answer: A, B, C, D

The short-term scheduler runs whenever an event may block the running process or offer a chance to preempt it: clock interrupts, I/O interrupts, OS calls and signals. A new batch job is a matter for long-term scheduling.

5. Order the three processor schedulers from the one that runs least often to the one that runs most often.

Correct order:

1. Long-term scheduling
2. Medium-term scheduling
3. Short-term scheduling

Long-term decisions happen when jobs arrive or leave, medium-term when memory pressure changes, and short-term many times every second. The inner the ring, the more often its decision.

6. Match each move or decision to the kind of scheduling that makes it.

New → Ready/Suspend	Long-term scheduling
Blocked → Blocked/Suspend	Medium-term scheduling
Ready → Running	Short-term scheduling
A disk chooses which pending request to serve next	I/O scheduling

Admission is long-term, swapping in or out is medium-term, dispatching is short-term, and ordering a device's request queue is I/O scheduling.

7. Sort each transition or event by the kind of scheduling that handles it.

Long-term	Medium-term	Short-term
New → Ready A user logs in to a time-sharing system	Ready/Suspend → Ready Blocked → Blocked/Suspend	Ready → Running A clock interrupt ends a time slice

Admissions and logins are long-term; swapping between memory and disk is medium-term; dispatching and reacting to clock interrupts is short-term.

8. The long-term scheduler admits 4 processes. Each waits for I/O 50% of the time, independently of the others. Using processor utilization = $1 - p^n$, what percentage of the time is the processor busy?

Answer: 93.75 %

$p = 0.5$ and $n = 4$: $1 - 0.5^4 = 1 - 0.0625 = 0.9375$, so the processor is busy 93.75% of the time.

9. Five admitted processes are always ready to compute, and the short-term scheduler shares the processor equally among them. What percentage of the processor's time does each process get?

Answer: 20 %

The processor is busy all the time and is split five ways: $100\% \div 5 = 20\%$ each. Admitting more processes shrinks every share.

10. Three processor-bound jobs are running and the disk sits mostly idle. Which waiting job should the long-term scheduler admit next to make better use of the hardware?

- A. Another processor-bound job
- B. An I/O-bound job
- C. No job: admission has no effect on how busy the devices are
- D. Whichever job needs the most memory

Answer: B. An I/O-bound job

An I/O-bound job uses the disk while the processor-bound jobs compute, so both resources stay busy. Keeping a balanced mix is a classic long-term policy.

11. A time-sharing system is already saturated. What does the OS typically do when one more authorized user tries to log in?

- A. Refuse the login with a "system full, try later" message
- B. Accept it and keep accepting every login, however slow the system becomes
- C. Put the login in the batch queue to start hours later
- D. Accept it by permanently ending the oldest user's session

Answer: A. Refuse the login with a "system full, try later" message

Logins are admitted until the system is saturated; after that the OS refuses new ones so the existing users keep acceptable response times.

12. On a system without virtual memory, medium-term scheduling must consider whether a swapped-out process will fit in the free main memory before swapping it in.

Answer: True

Without virtual memory a process must be wholly in main memory to run, so the swap-in decision depends on its memory needs as well as on the degree of multiprogramming.

9.2 Scheduling Algorithms

You should be able to

- Name the user-oriented and system-oriented scheduling criteria and explain why they pull against each other.
- Describe any scheduling policy by its selection function (written with w , e and s) and its decision mode.
- Simulate FCFS, round robin, SPN, SRT, HRRN and feedback scheduling and compute finish, turnaround and normalized turnaround times.
- Explain the quantum trade-off, virtual round robin, exponential averaging, and which policies can starve long processes.
- Compute fair-share priorities and predict how groups of processes split the processor.

Scheduling algorithms

On one processor only one process runs at a time. The short-term scheduler (the **dispatcher**) chooses which ready process runs next; its rule is the scheduling algorithm, or policy.

Scheduling criteria

	Performance-related (measurable)	Other (qualitative)
User-oriented	Turnaround time: submission to completion (waiting plus running); suits batch jobs. Response time: for an interactive request, submission until the response <i>begins</i> ; what users notice. Deadlines: maximize the percentage met.	Predictability: a job takes about the same time and cost whatever the load.
System-oriented	Throughput: processes completed per unit of time. Processor utilization: percentage of time busy; matters on expensive shared systems.	Fairness: treat processes alike; none starves. Enforcing priorities: favour higher priorities. Balancing resources: keep all resources busy; favour processes that avoid overloaded resources.

Criteria conflict: frequent switching gives good response time but adds overhead, lowering throughput.

Priorities

Ready queues RQ0 (highest priority) to RQn (lowest); the dispatcher serves the highest non-empty queue. Risk: low-priority processes can **starve**. Remedy: let priority change with age (aging) or execution history.

Describing a policy

- **Selection function:** the rule that picks the next process, written with **w** = time spent waiting so far, **e** = time spent executing so far, **s** = total service time required, including e (usually an estimate). $s - e$ is the time still needed.
- **Decision mode.** *Nonpreemptive:* a running process continues until it terminates or blocks itself. *Preemptive:* the OS may interrupt it and move it to Ready when a new process arrives, when an interrupt makes a blocked process ready, or periodically on a clock interrupt. More overhead, but usually better service.

Measuring a schedule

Turnaround T_r = finish time – arrival time. Normalized turnaround = T_r/T_s , where T_s is the service time; 1.0 means the process never waited, and larger values mean a longer delay relative to the work it needed. Standard workload (arrival, service): A (0, 3), B (2, 6), C (4, 4), D (6, 5), E (8, 2). Conventions: time advances in whole units; a process arriving at time t joins the queue before a process that is preempted (or whose quantum expires) at the same t ; ties in SPN and SRT go to the earlier arrival.

First-come-first-served (FCFS)

Selection $\max[w]$; nonpreemptive. Bad for a short process behind a long one: on the standard workload E needs 2 units but finishes at 20, a normalized turnaround of 6.00. It favours processor-bound over I/O-bound processes, which wait behind long bursts while their devices sit idle.

Round robin (RR)

A clock interrupt every quantum q ; the running process goes to the back of the ready queue (behind processes arriving at that instant). Very short q : short processes finish fast but interrupt and dispatch

overhead grows. Rule of thumb: slightly longer than a typical interaction. Longer than every burst: RR becomes FCFS.

RR is unfair to I/O-bound processes, which use part of a quantum, block, then wait behind full quanta.

Virtual round robin: processes returning from I/O join an auxiliary FCFS queue served before the main queue, and run only for the rest of their quantum.

Shortest process next (SPN) and shortest remaining time (SRT)

SPN: selection $\min[s]$; nonpreemptive. Short processes jump ahead, so averages improve (mean T_r 7.60 against 8.60 for FCFS on the standard workload), but long processes wait longer and less predictably, and can starve. It needs service-time estimates; a batch job that runs far past its estimate may be aborted.

SRT: the preemptive version, selection $\min[s - e]$. A new arrival with less time left preempts. No extra clock interrupts, but elapsed times must be recorded. Turnaround is better than SPN (mean T_r 7.20); long processes can still starve.

Estimating service time from past bursts $T(1), T(2), \dots$: the simple average $S(n+1) = (1/n) \sum T(i)$ weights bursts equally. **Exponential averaging** $S(n+1) = \alpha T(n) + (1 - \alpha)S(n)$, $0 < \alpha < 1$, gives weights $\alpha, \alpha(1 - \alpha), \alpha(1 - \alpha)^2, \dots$ to $T(n), T(n - 1), T(n - 2), \dots$; older observations count less and less. A larger α follows changes faster: with $\alpha = 0.8$ the last four bursts carry 99.84% of the weight. Worked example: $\alpha = 0.5$, $S(n) = 10$, $T(n) = 4$ gives $S(n+1) = 0.5 \times 4 + 0.5 \times 10 = 7$.

Highest response ratio next (HRRN)

Selection $\max[R]$ with response ratio $R = (w + s)/s$; nonpreemptive. R starts at 1.0 and grows while a process waits, faster for short ones: short jobs are favoured, yet a long job's ratio eventually wins, so no starvation. Needs estimates. Example: at $t = 9$ on the standard workload C has $R = (5 + 4)/4 = 2.25$, D has $R = (3 + 5)/5 = 1.60$, E has $R = (1 + 2)/2 = 1.50$, so C runs (SPN would have picked E).

Feedback (multilevel feedback queues)

No estimates: penalize processes that have run longer. Preemptive at the quantum. A new process enters RQ_0 and drops one queue each time it is preempted (unless nobody else is waiting). Queues are FCFS, the lowest is round robin, and the highest non-empty queue is served (the simulations in this section use 4 queues, RQ_0 to RQ_3). Short processes finish before sinking far. Long ones can starve. Giving RQ_i a longer quantum of 2^i lets a long process do more per turn; promoting a process that has waited too long (aging) is what prevents starvation.

Results on the standard workload

Policy	A	B	C	D	E	Mean Tr	Mean Tr/Ts
FCFS	3	9	13	18	20	8.60	2.56
RR q=1	4	18	17	20	15	10.80	2.71
RR q=4	3	17	11	20	19	10.00	2.71
SPN	3	9	15	20	11	7.60	1.84
SRT	3	15	8	20	10	7.20	1.59
HRRN	3	9	13	20	15	8.00	2.14
FB q=1	4	20	16	19	11	10.00	2.29
FB q=2 ⁱ	4	17	18	20	14	10.60	2.63

Cells are finish times. Lowest mean Tr/Ts: SRT (1.59); highest: RR q=1 and RR q=4 (2.71).

Summary of characteristics

Policy	Selection	Decision mode	Throughput	Response time	Overhead	Effect on processes	Starvation
FCFS	max[w]	Nonpreemptive	Not emphasized	May be high, especially with large variance in service times	Minimum	Penalizes short and I/O-bound processes	No
RR	constant	Preemptive (at quantum)	May be low if quantum too small	Good for short processes	Minimum	Fair treatment	No
SPN	min[s]	Nonpreemptive	High	Good for short processes	Can be high	Penalizes long processes	Possible
SRT	min[s – e]	Preemptive (at arrival)	High	Good	Can be high	Penalizes long processes	Possible
HRRN	max[(w + s)/s]	Nonpreemptive	High	Good	Can be high	Good balance	No
Feedback	highest non-empty queue	Preemptive (at quantum)	Not emphasized	Not emphasized	Can be high	May favour I/O-bound processes	Possible

Fair-share scheduling

Decisions are based on groups of processes (per user or per application) rather than single processes. Group k gets a weight W_k ($0 < W_k \leq 1$, weights summing to 1), its share of the processor. Once per second:

$CPU_j = CPU_j/2$ and $GCPU_k = GCPU_k/2$ (old use decays), then $P_j = Base_j + CPU_j/2 + GCPU_k/(4 \times W_k)$, with whole-number division. CPU_j counts process j 's recent clock ticks (60 a second) and $GCPU_k$ its group's; the lowest P runs.

Example: A alone in group 1, B and C in group 2, both weights 0.5, base 60. Priorities (A/B/C) at seconds 0 to 3: 60/60/60, 90/60/60, 74/90/75, 96/74/67. The processor goes to A, B, A, C, A, B, A, C, A, B, A, C, ..., so A gets 50% and B and C 25% each: each group gets half, whatever the number of processes in it. Without the group term ($P = Base + CPU/2$) the order becomes A, B, C, A, B, C, ..., and the shares become A 33%, B 33%, C 33%. Because the processor is handed out a whole second at a time, other weights are matched only approximately.

Key terms

Service time

The total processor time a process needs to finish its work (written T_s , or s in a selection function). It usually has to be estimated in advance.

Normalized turnaround time

Turnaround time divided by service time (T_r/T_s). A value of 1.0 means the process never waited; larger values mean a longer delay relative to the work it needed.

Processor utilization

The percentage of time the processor is busy doing useful work rather than sitting idle.

Selection function

The rule a scheduling policy uses to pick which ready process runs next, usually written in terms of w (time waited), e (time executed) and s (total service time).

Decision mode

When a policy is allowed to make its choice: only when the running process ends or blocks (nonpreemptive) or also by interrupting it (preemptive).

Nonpreemptive scheduling

Once a process is running it keeps the processor until it finishes or blocks itself, for example to wait for I/O.

Preemptive scheduling

The OS may interrupt the running process and move it back to Ready, for example when a new process arrives or a clock interrupt marks the end of a quantum.

Processor-bound process

A process that mostly computes: it uses long stretches of processor time and does little I/O. Also called CPU-bound.

I/O-bound process

A process that mostly waits for I/O: it runs for short bursts, then blocks for a device, again and again.

First-come-first-served (FCFS)

A nonpreemptive policy that always runs the process that has waited longest in the ready queue. Also called FIFO.

Round robin (RR)

A preemptive policy that gives each ready process one quantum in turn; when its quantum ends, the running process goes to the back of the ready queue.

Virtual round robin (VRR)

A round robin variant in which processes coming back from I/O wait in an auxiliary queue that is served before the main ready queue, and run only for what was left of their quantum.

Shortest process next (SPN)

A nonpreemptive policy that runs the waiting process with the shortest expected service time.

Shortest remaining time (SRT)

The preemptive version of SPN: the process with the shortest expected remaining time runs, and a new arrival with less time left takes over the processor.

Exponential averaging

Predicting the next value as a weighted blend of the latest observation and the previous prediction, $S(n+1) = \alpha T(n) + (1 - \alpha)S(n)$, so older observations count less and less.

Response ratio

$R = (w + s) / s$, where w is the time a process has waited so far and s its expected service time. It starts at 1.0 and grows while the process waits.

Highest response ratio next (HRRN)

A nonpreemptive policy that runs the waiting process with the largest response ratio, favouring short jobs while letting long ones age their way to the front.

Feedback scheduling

A preemptive policy with several ready queues of falling priority: a new process starts in the top queue and drops one queue each time it is preempted, so long-running processes sink. Also called multilevel feedback.

Aging

Raising the priority of a process the longer it waits, so that it cannot be passed over forever.

Fair-share scheduling

Scheduling that divides the processor among groups of processes (for example users) according to each group's share, instead of treating every process separately.

Review questions with answers

1. An interactive user cares most about how soon the system *starts* to answer a request. Which criterion measures exactly that?

- A. Turnaround time
- B. Response time
- C. Throughput
- D. Processor utilization

Answer: B. Response time

Response time runs from submitting a request until the response begins. It is user-oriented and performance-related, and it is the right yardstick for interactive work.

2. Under FCFS, process E arrives at time 8 and needs 2 units of processor time. Processes that arrived earlier keep the processor busy until time 18, and then E runs. What is E's normalized turnaround time (T_r/T_s)?

Answer: 6

E finishes at 20, so $T_r = 20 - 8 = 12$ and $T_r/T_s = 12 / 2 = 6.00$. A short process stuck behind long ones is exactly where FCFS does worst.

3. Five processes, given as (arrival, service): A (0, 3), B (2, 6), C (4, 4), D (6, 5), E (8, 2). Under shortest process next (SPN, nonpreemptive), at what time does C finish?

Answer: 15

The schedule is A 0–3, B 3–9, E 9–11, C 11–15, D 15–20. At $t = 9$, C, D and E are waiting and E ($s = 2$) is the shortest, so E runs before C.

4. Under HRRN, the processor falls free at time 9. Process C arrived at time 4, needs 4 units and has not run yet. What is its response ratio $R = (w + s)/s$?

Answer: 2.25

C has waited $w = 9 - 4 = 5$, so $R = (5 + 4)/4 = 2.25$. D has 1.60 and E has 1.50, so HRRN runs C.

5. A process's predicted burst was $S(n) = 10$ ms, but its latest burst actually took $T(n) = 4$ ms. With exponential averaging and $\alpha = 0.5$, what is the next prediction $S(n+1)$, in ms?

Answer: 7 ms

$S(n+1) = \alpha T(n) + (1 - \alpha)S(n) = 0.5 \times 4 + 0.5 \times 10 = 7$ ms. The estimate moves halfway toward the newest observation.

6. Fair-share scheduling with base priority 60 and 60 clock ticks per second. Process A is alone in group 1 (weight 0.5) and ran for the whole first second. At the next update the counts are halved, so $CPU_A = 30$ and $GCPU_1 = 30$. What is A's new priority $P = \text{Base} + CPU/2 + GCPU/(4W)$?

Answer: 90

$P = 60 + 30/2 + 30/(4 \times 0.5) = 60 + 15 + 15 = 90$. Processes that have not run still have 60, and a lower number means a higher priority, so one of them runs next.

7. Making the round-robin quantum longer than every process's service time turns round robin into FCFS.

Answer: True

If the clock never cuts a process off, each one runs to completion in the order it reached the queue, which is exactly FCFS.

8. Which of these policies can starve a process, leaving it waiting indefinitely while other processes keep arriving?

- A. FCFS
- B. Round robin
- C. SPN
- D. SRT
- E. HRRN
- F. Feedback

Answer: C, D, F

SPN and SRT keep preferring newly arrived short processes over a long one, and feedback can leave long processes stuck in the bottom queue. FCFS and round robin serve in turn, and a waiting process's HRRN ratio keeps growing until it wins.

9. Match each policy to its selection function (w = time waited, e = time executed, s = total service time).

FCFS	$\max[w]$
SPN	$\min[s]$
SRT	$\min[s - e]$
HRRN	$\max[(w + s)/s]$
Round robin	constant: everyone in turn

FCFS picks the longest waiter, SPN the shortest job, SRT the least remaining time, HRRN the largest response ratio, and round robin simply rotates through the ready queue.

10. Sort each policy by its decision mode.

Nonpreemptive	Preemptive
FCFS	Round robin
SPN	SRT
HRRN	Feedback

FCFS, SPN and HRRN choose only when the running process ends or blocks. Round robin and feedback preempt at the end of a quantum; SRT preempts when a process with less remaining time arrives.

11. In virtual round robin, what happens to a process that comes back from an I/O wait?

- A. It joins an auxiliary queue served before the main ready queue, and runs for at most the rest of its quantum
- B. It goes to the back of the main ready queue, like any other process, and receives a fresh full quantum
- C. It immediately preempts whichever process is running, then uses up the rest of its unfinished quantum
- D. It joins an auxiliary queue that the dispatcher serves only once the main ready queue is completely empty

Answer: A. It joins an auxiliary queue served before the main ready queue, and runs for at most the rest of its quantum

VRR gives an I/O-bound process credit for the part of its quantum it did not use: it is served first, but only for that remainder, so it never gets more than one quantum per turn.

12. Put the once-per-second work of the fair-share scheduler in order.

Correct order:

1. Halve every process's CPU count and every group's GCPU count
2. Recompute each priority $P = \text{Base} + \text{CPU}/2 + \text{GCPU}/(4W)$
3. Give the processor to the process with the lowest P
4. Add the clock ticks it uses to its own CPU count and to its group's GCPU count

Decay comes first, so old use counts less; then every priority is recomputed; the lowest number runs; and the ticks it uses are charged to it and to its whole group, raising their numbers for the next round.

9.3 Traditional UNIX Scheduling

You should be able to

- Explain what the traditional UNIX scheduler was built for, and describe its structure: priority queues with round robin inside each and one-second preemption.
- Compute a process's CPU count and priority with the once-a-second formula, including the nice value and whole-number division.
- Name the priority bands from highest to lowest and explain why work that drives I/O devices sits above ordinary user processes.
- Trace several processes second by second and predict which one the scheduler picks next.
- Explain why a decaying usage count gives I/O-bound processes quick service and keeps busy jobs of equal nice value from starving, and why a large nice value can still starve a job.

Traditional UNIX scheduling

The traditional UNIX scheduler (as in System V Release 3 and 4.3 BSD) was designed for interactive time sharing: many people at terminals plus some long background jobs, all sharing one processor. It had two goals that pull against each other: give interactive users fast response, and make sure low-priority background jobs still make progress instead of starving.

Structure

- **Priority numbers, lower is better.** Every process has a priority number. The scheduler runs the ready process with the **smallest** number, so 63 beats 76.
- **Multilevel feedback queues with round robin.** There is a ready queue for each priority level; the scheduler serves the best non-empty queue, and processes with the same number take turns (round robin). A process moves between levels as its recent behavior changes its number.
- **One-second preemption.** A process that has run for a whole second without blocking or finishing is preempted, and a fresh choice is made. If it blocks sooner (for example to wait for a keystroke), the next process is picked at once.
- **Once-a-second recalculation.** The clock ticks 60 times a second and each tick adds 1 to the running process's CPU count. Once a second every count is halved and every priority recomputed.

The five priority bands

Base priorities divide all processes into fixed bands. From highest priority to lowest:

1. **Swapper:** the kernel process that moves whole processes between memory and disk. If it waited, a memory jam might never clear.
2. **Block I/O device control:** processes waiting for disks and other block devices. Running them at once keeps the disk busy and frees buffers.
3. **File manipulation:** processes waiting inside file-system code; they often hold resources other processes need.
4. **Character I/O device control:** processes waiting for terminals and other character devices; they need only short bursts, so serving them first keeps terminals responsive.
5. **User processes:** ordinary programs running their own code. The formula below sorts processes inside this band.

Putting device-driving work first keeps the I/O devices busy, and costs little because that work runs only briefly each time. The CPU and nice terms are limited so a process can never leave its band. (A process is in an upper band while it waits inside the kernel for that kind of event; once it runs its own code again it is back in the user band.)

The formula

$$\begin{aligned}\text{CPU}_j(i) &= \text{CPU}_j(i - 1) / 2 \\ \text{P}_j(i) &= \text{Base}_j + \text{CPU}_j(i) / 2 + \text{nice}_j\end{aligned}$$

- **j** names the process and **i** the one-second interval. All division rounds down (whole numbers only).
- **CPU_j(i)**: process j's recent processor use: +1 per clock tick while it runs, halved once a second.
- **P_j(i)**: its priority number at the start of interval i (lower = more urgent).
- **Base_j**: its fixed base priority, which picks its band (60 for the user processes in the examples).
- **nice_j**: a user-controlled adjustment. An ordinary user can only raise it (be "nicer", less urgent); only the superuser can lower it.

Worked example. A process (base 60, nice 0) ends a second with count 67. Halve: $\lfloor 67/2 \rfloor = 33$. Then $P = 60 + \lfloor 33/2 \rfloor + 0 = 60 + 16 = \mathbf{76}$.

Usage decay. A process that stops running sees its count halve every second: $60 \rightarrow 30 \rightarrow 15 \rightarrow 7 \rightarrow 3 \rightarrow 1 \rightarrow 0$, so its number drifts back to its base within a few seconds. A process that runs nonstop gains 60 before each halving, so its count levels off just under 60 and its number near base + 29, instead of growing forever. Recent use matters; old use is forgotten. (Halving is the classic form, close to System V's; 4.3 BSD used the same idea with different weights, a 0.1-second time slice and a decay factor tied to the load average, so old use faded more slowly on a busy system.)

The classic trace

A, B and C are created together with base 60 and nice 0; there are 60 ticks a second; ties go to the process that has waited longest (and, if that is equal too, the one created first). Values are shown at the start of each second, after the halving:

t	A count / P	B count / P	C count / P	Runs
0	0 / 60	0 / 60	0 / 60	A
1	30 / 75	0 / 60	0 / 60	B
2	15 / 67	30 / 75	0 / 60	C
3	7 / 63	15 / 67	30 / 75	A
4	33 / 76	7 / 63	15 / 67	B
5	16 / 68	33 / 76	7 / 63	C
6	8 / 64	16 / 68	33 / 76	A
7	34 / 77	8 / 64	16 / 68	B
8	17 / 68	34 / 77	8 / 64	C

At $t = 6$: A's count 16 halves to 8, so $P = 60 + 4 = 64$; B: $33 \rightarrow 16$, $P = 68$; C ran in second 5, so $7 + 60 = 67 \rightarrow 33$, $P = 76$. A has the smallest number and runs. From $t = 3$ the order repeats A, B, C: the arithmetic itself produces round robin, and none of the three starves.

Why I/O-bound processes get quick service

An I/O-bound process such as an editor runs in short bursts and then sleeps, so its count stays tiny and its number stays near its base. Processor-bound processes keep pushing their own numbers up. In the guide's experiment, an editor D joins A, B and C at $t = 3$ and needs 2 ticks per keystroke: its count never passes 5, its number never passes 62, and each of its 16 keystrokes is served at once. (The model's rule: a waking process takes over at once only if its number is strictly smaller than the running process's; otherwise it waits for a whole-second decision.)

Nice values change this. With D at nice +2 its keystrokes wait 12.3 ticks on average; at nice +5 it never runs at all, because the busy processes keep falling back to numbers below D's. If D needs 12 ticks per keystroke instead, its count climbs to 15 and its number to 67, and some keystrokes wait up to 114 ticks. Giving processor-bound A nice +6 cuts its share over 30 seconds to 23%; at nice +10 it gets 0% in this simple model, because B and C alternate and the waiting one is always below 70. Nice is a blunt tool.

What decay does and does not guarantee

The design goal was that low-priority background jobs do not starve, and decay delivers that for jobs that are low only because of their **own recent use**: while such a job waits, its count halves every second until its number is back near its base, below the numbers of the processes that are running. That is why A, B and C, or a compiler running next to other busy jobs with the same base and nice value, all keep getting turns. Decay never touches a **nice value** or a bigger base, though. With three busy processes at nice 0, the one that has waited longest is back to 64 or better at every decision, so a fourth job at nice +5 (never better than 65) does not run at all while they stay busy. Real systems softened this: 4.3 BSD's slower, load-based decay keeps busy processes' numbers higher, which made such lockouts far rarer.

Today

Modern systems use different schedulers: Linux used the Completely Fair Scheduler from version 2.6.23 and replaced it with EEVDF in 6.6, and FreeBSD uses ULE. The idea that recent heavy processor use should

lower a process's priority, so lighter and interactive work goes first, lives on, and the nice command still exists.

Key terms

Priority number

In UNIX, a process's priority written as a number in which a LOWER number means a HIGHER priority. The scheduler runs the ready process with the smallest number.

Multilevel feedback queue

A scheduler with several ready queues, one per priority level. It always serves the best non-empty queue, and a process moves between queues according to how it has behaved recently.

One-second preemption

The traditional UNIX rule that a process that has run for a whole second without blocking or finishing is preempted, so the scheduler can make a fresh choice.

Clock tick

One interrupt from the hardware timer. In the classic UNIX example the clock ticks 60 times a second, and each tick adds 1 to the running process's CPU count.

CPU count

A per-process number, written CPU_j, that measures recent processor use. It goes up by 1 on every clock tick the process runs and is halved once a second, so old use fades.

Usage decay

Shrinking a usage count step by step (here, halving it every second) so that recent processor use weighs much more than use from long ago.

Base priority

The fixed starting priority number of a process, written Base_j. It places the process in a priority band; the CPU and nice terms are added on top of it.

Nice value

A number added to a process's priority so it can be "nice" to other processes. An ordinary user can only raise it (making the process less urgent); only the superuser can lower it.

Superuser

The all-powerful administrator account on a UNIX system (usually named root), which may override the limits placed on ordinary users.

Priority band

A fixed range of priority numbers set aside for one kind of work. A process's base priority picks its band, and its priority is kept inside that band.

Block I/O device

A device that stores and moves data in fixed-size blocks that can be addressed one by one, such as a disk.

Character I/O device

A device that sends or receives data as a stream of single characters, such as a terminal's keyboard and screen.

Interactive process

A process that works with a person in real time, such as a shell or a text editor. What matters most for it is how quickly it reacts to each keystroke or command.

I/O-bound process

A process that spends most of its time waiting for input or output and uses the processor only in short bursts.

Processor-bound process

A process that mostly computes, using the processor for long stretches with little I/O. Also called CPU-bound.

Review questions with answers

1. Under the traditional UNIX scheduler, ready process X has priority number 63 and ready process Y has priority number 76. What happens at the next scheduling decision?

- A. Y runs, because a bigger number means a higher priority
- B. X runs, because a smaller number means a higher priority
- C. X and Y alternate on every clock tick, because both are user processes
- D. Whichever has the smaller nice value runs, whatever the priority numbers say

Answer: B. X runs, because a smaller number means a higher priority

Lower number = higher priority. With 63 against 76, X is more urgent and runs first.

2. A user process has base priority 60 and nice value 0. At the end of a second its CPU count is 50. Using whole-number division, what is its priority number after the once-a-second update?

Answer: 72

Halve: $\text{CPU} = 50 / 2 = 25$. Then $P = 60 + \lfloor 25 / 2 \rfloor + 0 = 60 + 12 = 72$.

3. A user process (base 60, nice 0) has a CPU count of 40 at the end of second 9. It does not run at all during seconds 10 and 11. What is its priority number at $t = 12$?

Answer: 62

The count halves at each update: $40 \rightarrow 20$ ($t = 10$) $\rightarrow 10$ ($t = 11$) $\rightarrow 5$ ($t = 12$). So $P = 60 + \lfloor 5 / 2 \rfloor + 0 = 62$. Waiting lets the number drift back toward the base.

4. Put the traditional UNIX priority bands in order, from highest priority to lowest.

Correct order:

- 1. Swapper
- 2. Block I/O device control
- 3. File manipulation
- 4. Character I/O device control
- 5. User processes

The swapper comes first because everything may depend on memory; then the work that drives disks, then file work, then terminals and other character devices, and finally ordinary user processes.

5. An ordinary user may give their own process a negative nice value so that it runs sooner than other users' processes.

Answer: False

An ordinary user can only make a process "nicer", raising its nice value and so its priority number. Only the superuser may lower a nice value to make a process more urgent.

6. Which statements describe the traditional UNIX scheduler (as in System V Release 3 and 4.3 BSD)?

- A. It keeps several priority queues and uses round robin inside each queue
- B. A process that runs for a whole second without blocking or finishing is preempted
- C. Every priority is recomputed on every clock tick
- D. Each process's recent processor use is shrunk once a second (halved, in the classic formula)
- E. Each process must state its expected running time when it starts

Answer: A, B, D

It is a multilevel feedback scheme with round robin inside each level and one-second preemption. Counts grow on every tick but are shrunk (halved, in the classic formula) and turned into priorities only once a second, and no running-time estimates are needed.

7. Why does an I/O-bound text editor tend to get the processor quickly under this scheduler?

- A. The system gives interactive programs such as editors a permanent negative nice value
- B. I/O-bound processes are moved up into the swapper band, which is served before all others
- C. It uses little processor time, so its CPU count and therefore its priority number stay small
- D. Editors are usually started first, and among user processes the oldest one always wins

Answer: C. It uses little processor time, so its CPU count and therefore its priority number stay small

The count rises only while a process runs. An editor runs in short bursts and then sleeps, so its count and number stay low and it outranks processor-bound processes when it wakes.

8. A user process has been using the processor heavily. Sort each change by its effect on the process's priority number.

Number goes up (less urgent)	Number goes down (more urgent)
The user raises its nice value by 5 Its base priority number is raised by 10 Its CPU count rises from 10 to 50 before the update	The superuser lowers its nice value by 5 It sleeps for several seconds waiting for input

Anything that adds to Base, CPU/2 or nice makes the number bigger. Lowering nice (superuser only) or letting the count shrink by sleeping makes it smaller.

9. Match each part of the UNIX priority formula to its meaning.

$CPU_j(i)$	Recent processor use of process j , in clock ticks, halved each second
$P_j(i)$	Priority number of process j at the start of interval i
$Base_j$	Fixed starting priority that places process j in its band
$nice_j$	Adjustment a user can raise to make process j less urgent

$CPU_j(i)$ is the decaying usage count, $P_j(i)$ the result, $Base_j$ the fixed starting point, and $nice_j$ the user-controlled adjustment.

10. Three user processes all have base priority 60. After the halving at $t = 4$ their CPU counts are $X = 12$, $Y = 9$ and $Z = 20$. X and Z have nice 0; Y has nice +4. Which process runs during second 4?

- A. X
- B. Y
- C. Z
- D. X and Y share the second because they tie

Answer: A. X

X : $60 + \lfloor 12/2 \rfloor + 0 = 66$. Y : $60 + \lfloor 9/2 \rfloor + 4 = 68$. Z : $60 + \lfloor 20/2 \rfloor + 0 = 70$. The smallest number, 66, belongs to X .

11. Why are the bands for processes that drive I/O devices placed above the user band?

- A. Because device-driving code is written by the superuser and has to be trusted more than ordinary user code
- B. Because user processes running ahead of them could otherwise corrupt the data that is stored on the devices
- C. So that device-driving processes can never be preempted once they have started a transfer to a device
- D. So each device gets its next request quickly and stays busy, at little cost: that work needs only short bursts

Answer: D. So each device gets its next request quickly and stays busy, at little cost: that work needs only short bursts

A process waiting on a device needs the processor only briefly to start the next transfer. Running it first keeps disks and terminals busy and responsive while costing everyone else very little.

12. Under the halving formula, three processor-bound user processes with base 60 and nice 0 compute nonstop. A fourth processor-bound process with base 60 is started with nice +5. What happens to it while the other three keep running?

- A. It gets about a quarter of the processor, because usage decay eventually makes every process take its turn
- B. It runs ahead of the other three, because a bigger nice value gives a process a higher scheduling priority
- C. It never runs: the busy processes keep falling back to numbers no worse than 64, and its nice value never decays
- D. It waits a few seconds until halving brings its own priority number down to 64, then joins the rotation

Answer: C. It never runs: the busy processes keep falling back to numbers no worse than 64, and its nice value never decays

With three equal busy processes, the one that has waited longest is back to 64 or better at every decision, while the newcomer can never do better than 65. Usage decay protects a process that is behind only because of its own recent use; a large nice value can still starve a job in this pure halving model.